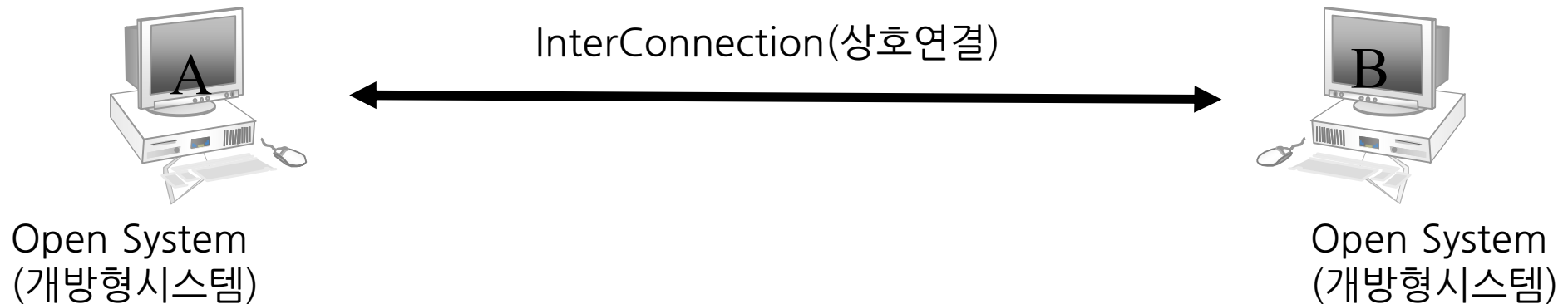


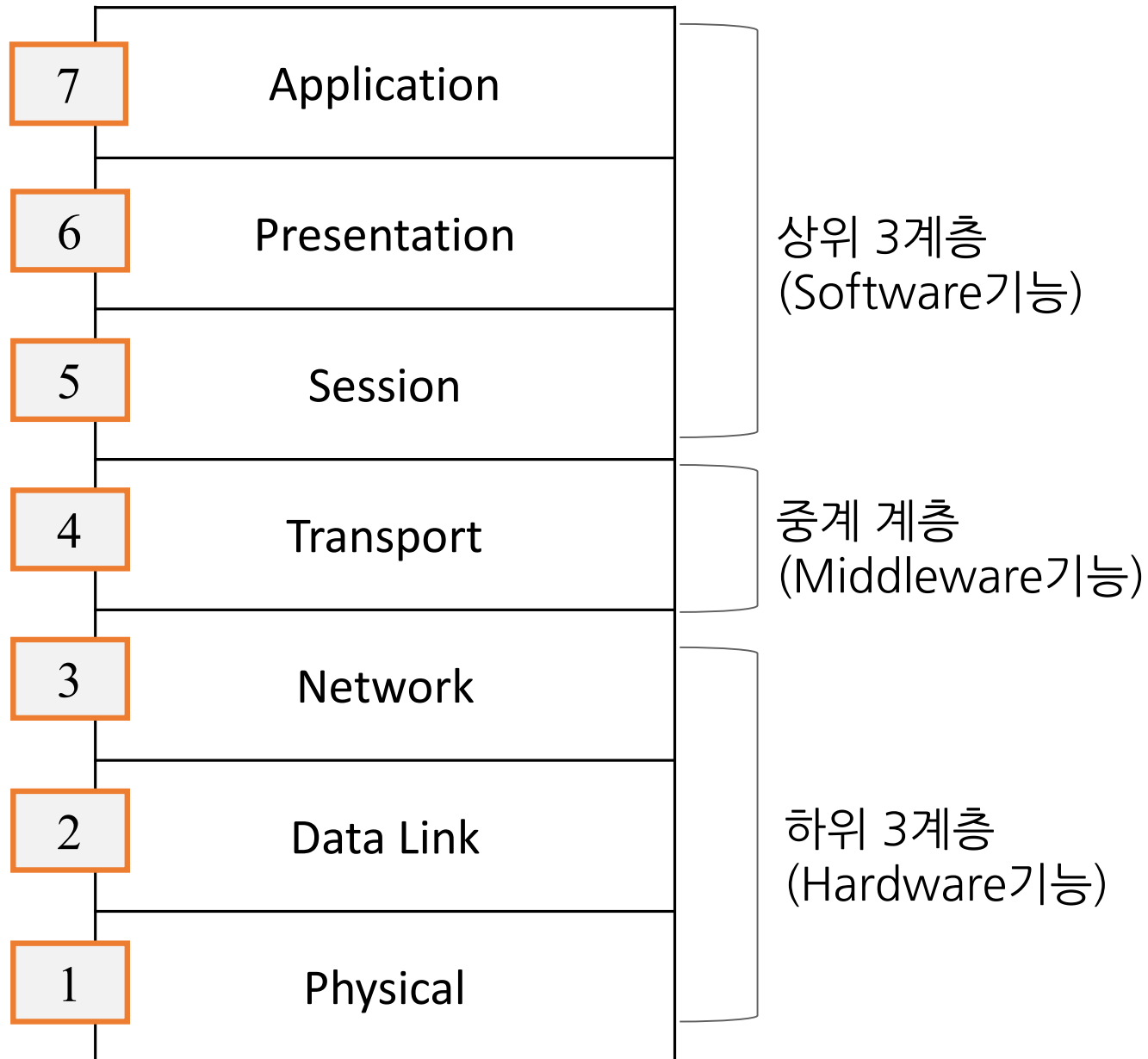
- OSI 참조모델
- TCP/IP Protocol Stack

1. OSI 7 참조모델

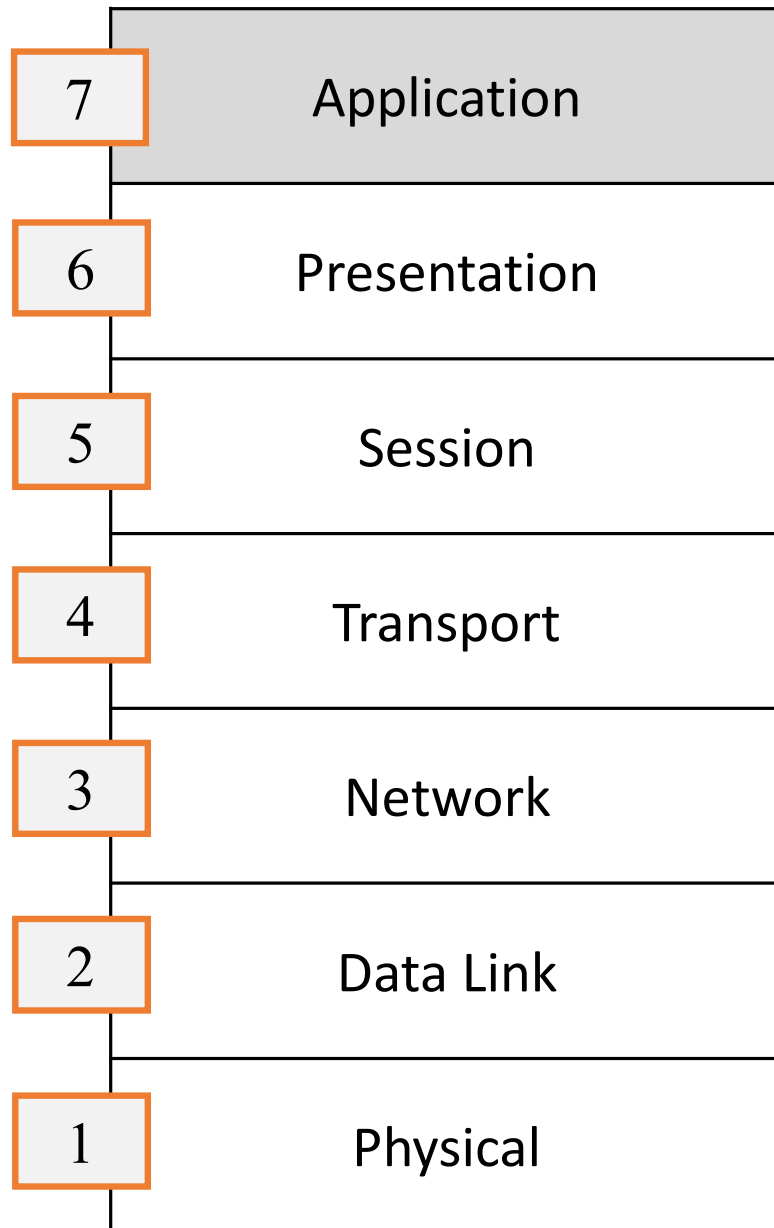
- ISO(국제 표준 기구)는 서로 다른 시스템간의 통신을 허용하기 위해 OSI (Open System Interconnection) 모델을 만듦
- OSI 참조모델은 network가 제공하는 여러 가지의 기능을 7개의 계층으로 나누어 식별



OSI 7 참조모델



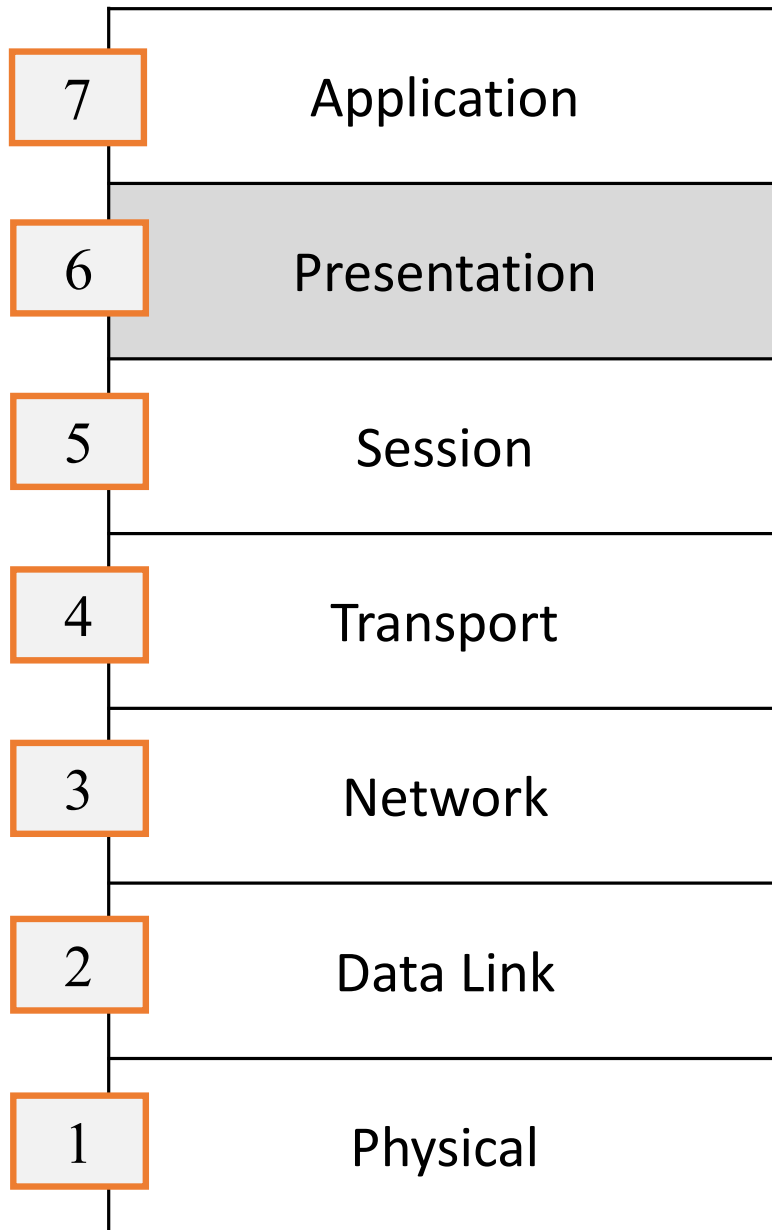
Application Layer



User Interface (UI)제공

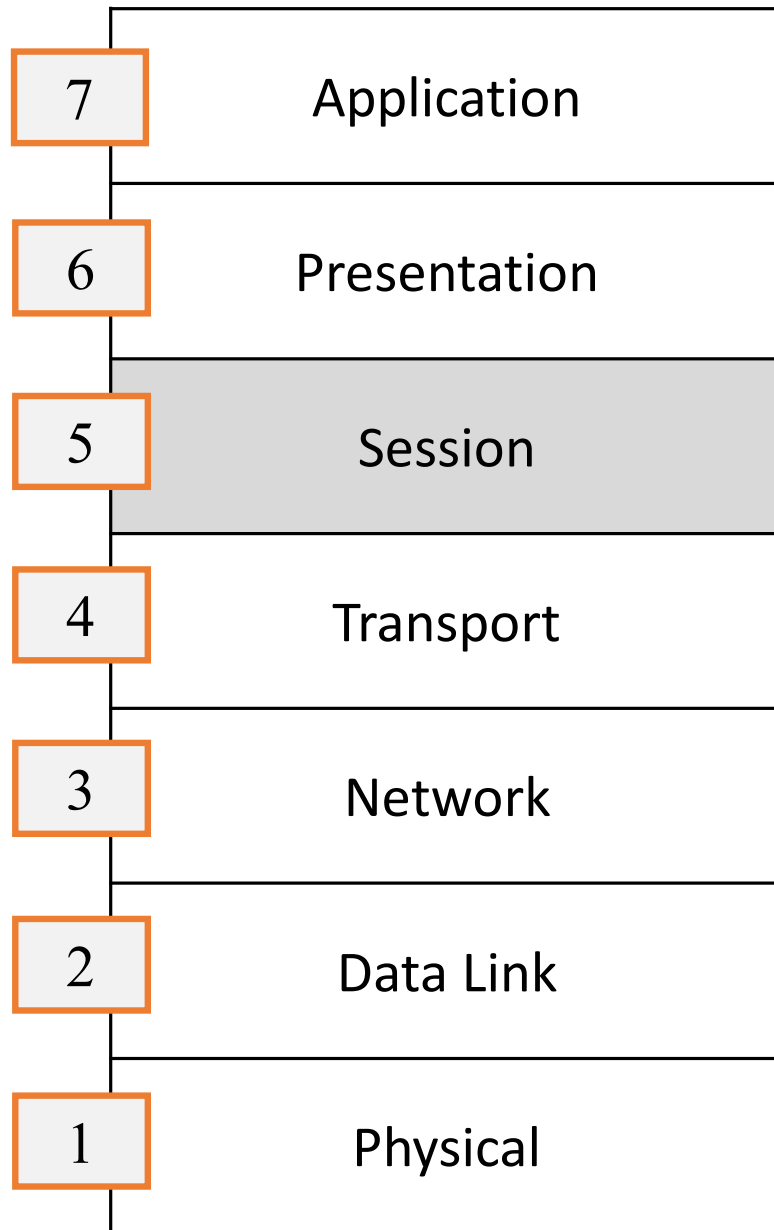
- UI를 통해 데이터 생성
- HTTP, FTP, Telnet, SMTP 등

Presentation Layer



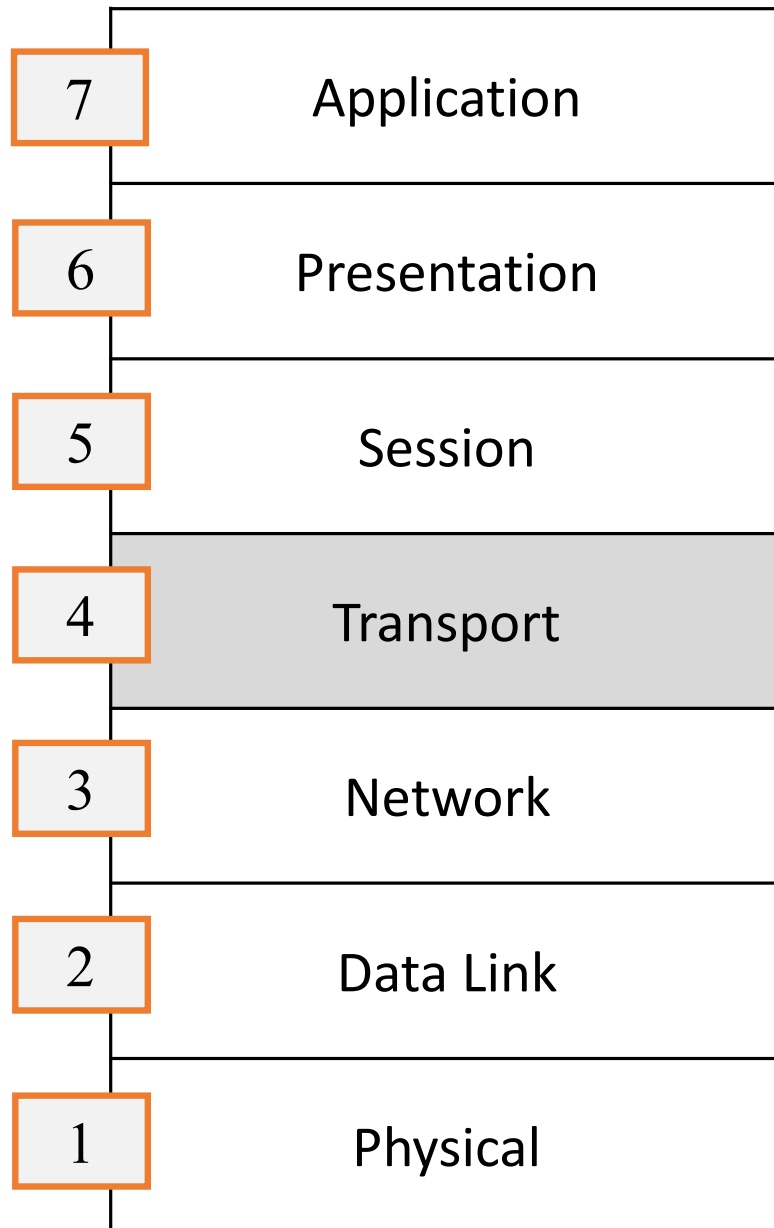
- Code 변환(부호화, encoding)
- 압축(Compression)
- 암호화(Encryption)

Session Layer



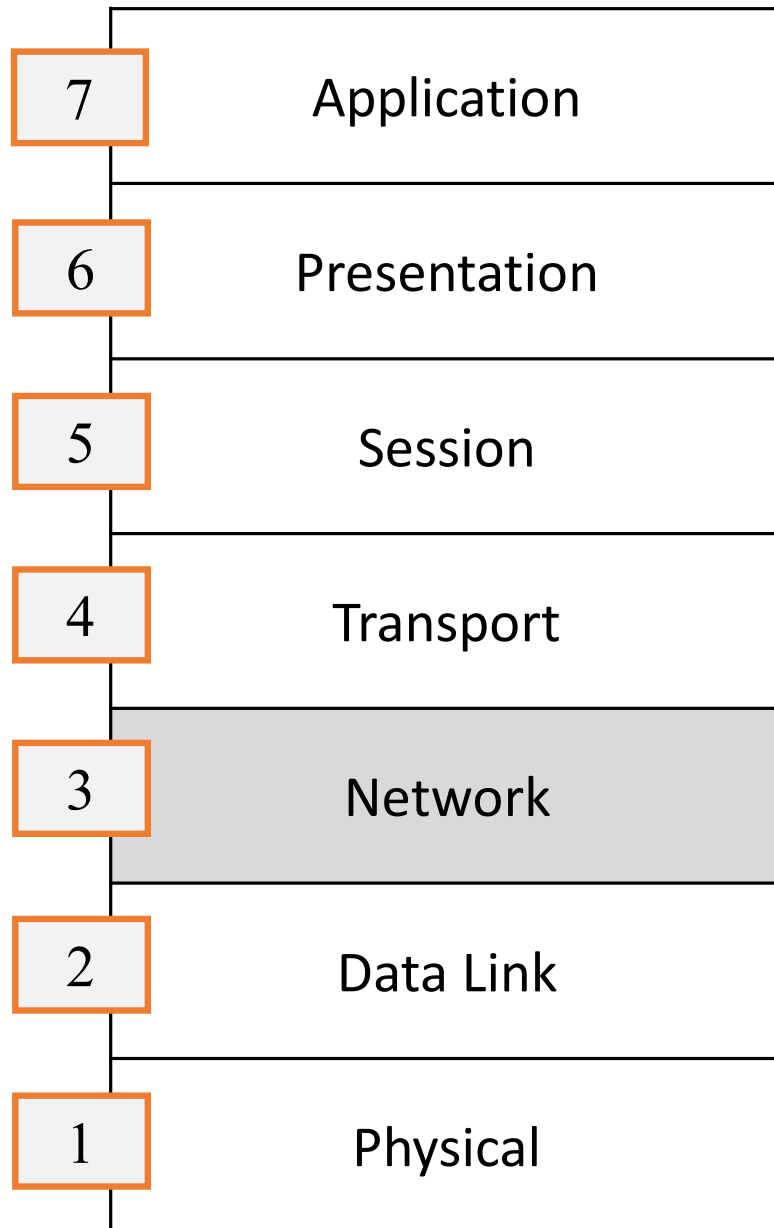
- End-to-End 호스트 사이에서 Application(서비스)의 연결설정(개시), 유지, 해지

Transport Layer



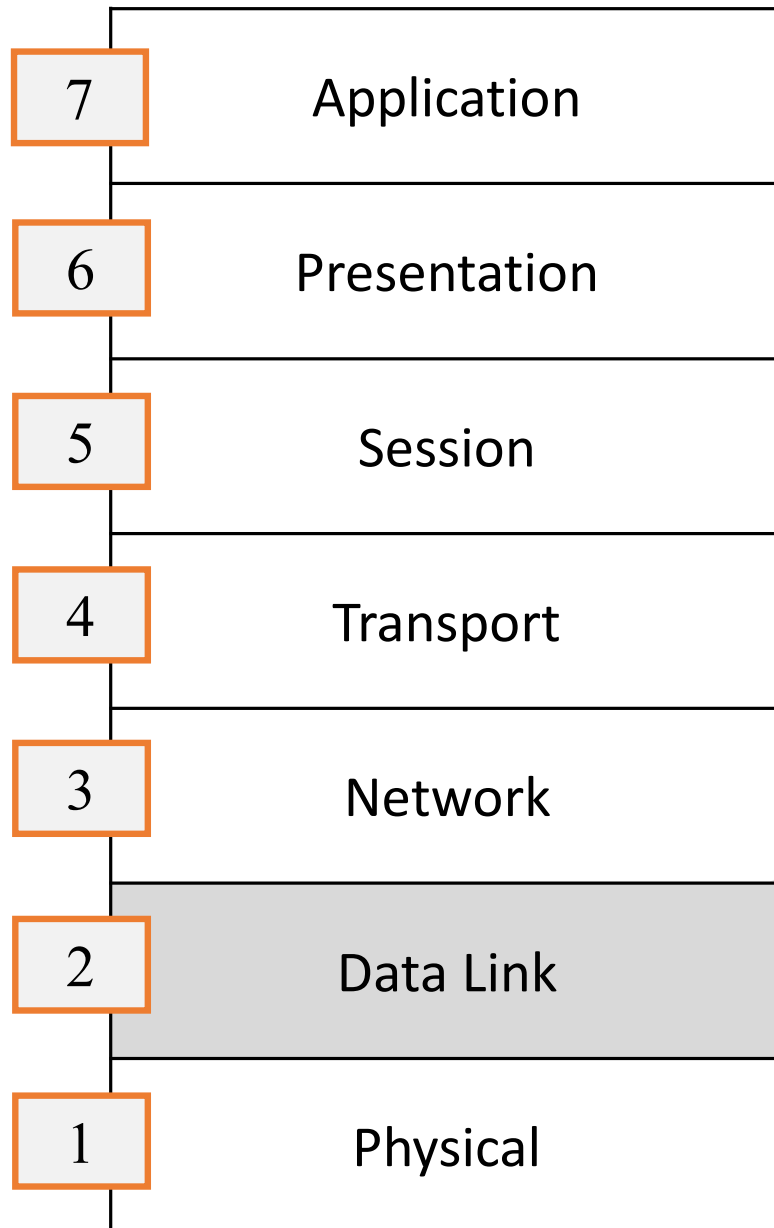
- End-to-End 사이에 가상회선(virtual Circuit) 설정, 유지, 해지

Network Layer



- 최적 경로 결정(Routing)

Data Link Layer



- 데이터 전달(forwarding)
- 오류 제어(error control)
- 매체접근제어(media access control)

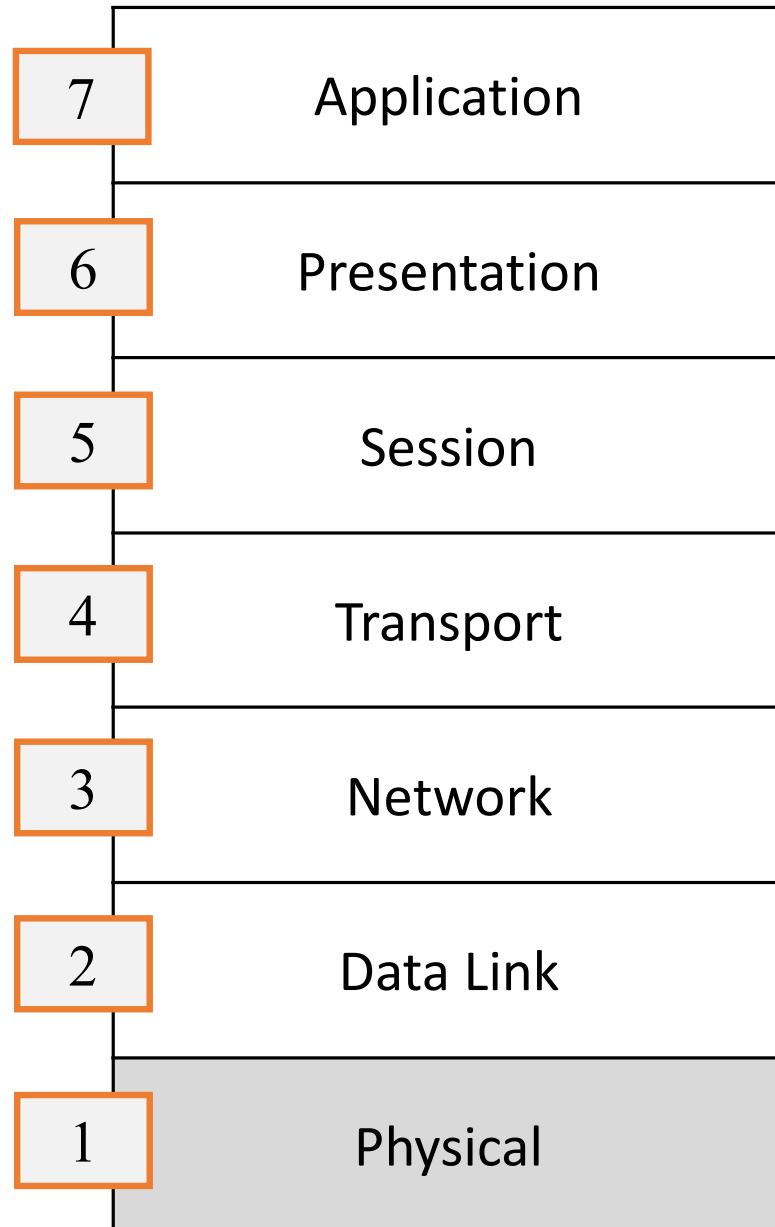
Logical Link Control

LLC

MAC

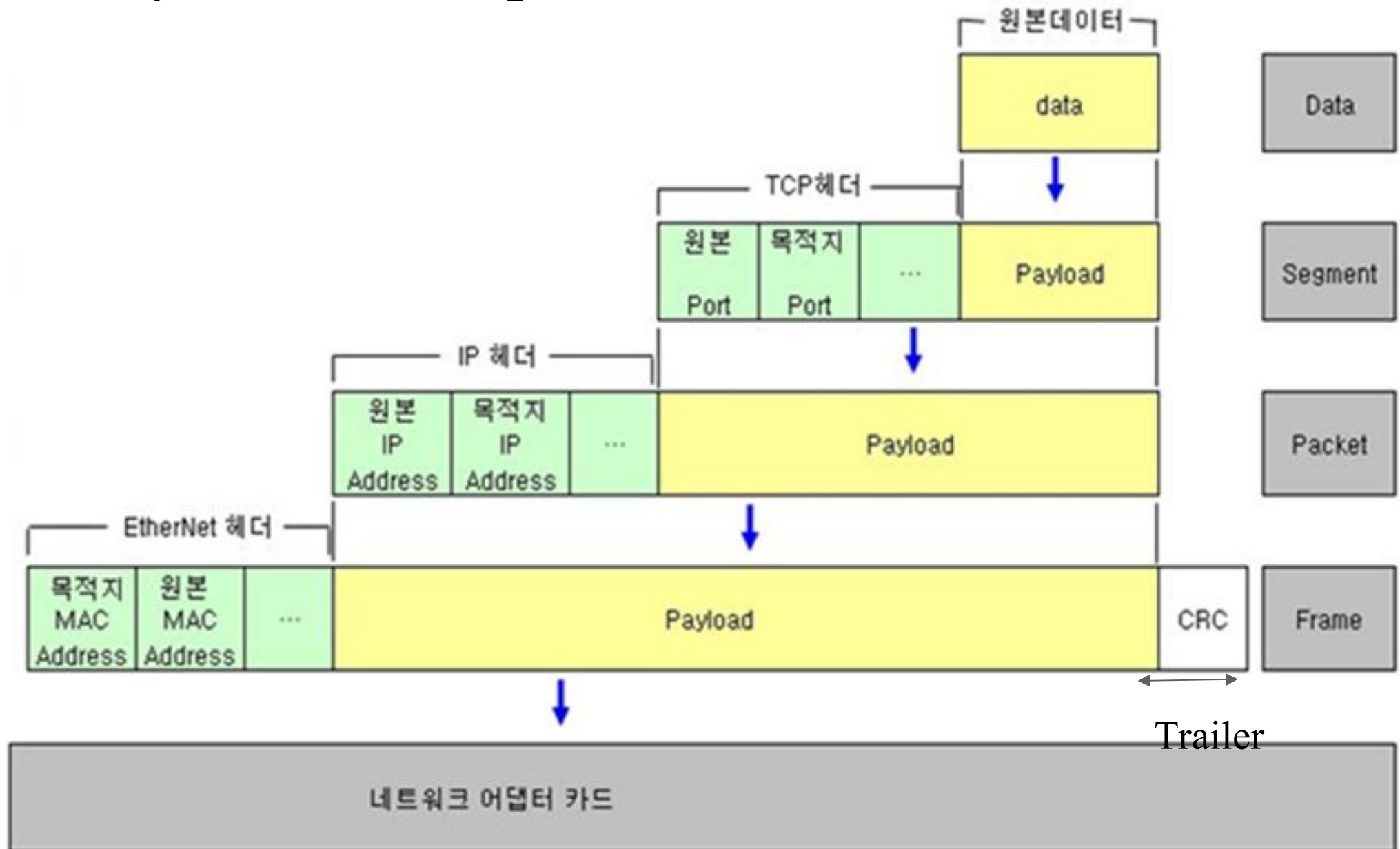
Media Access Control

Physical Layer



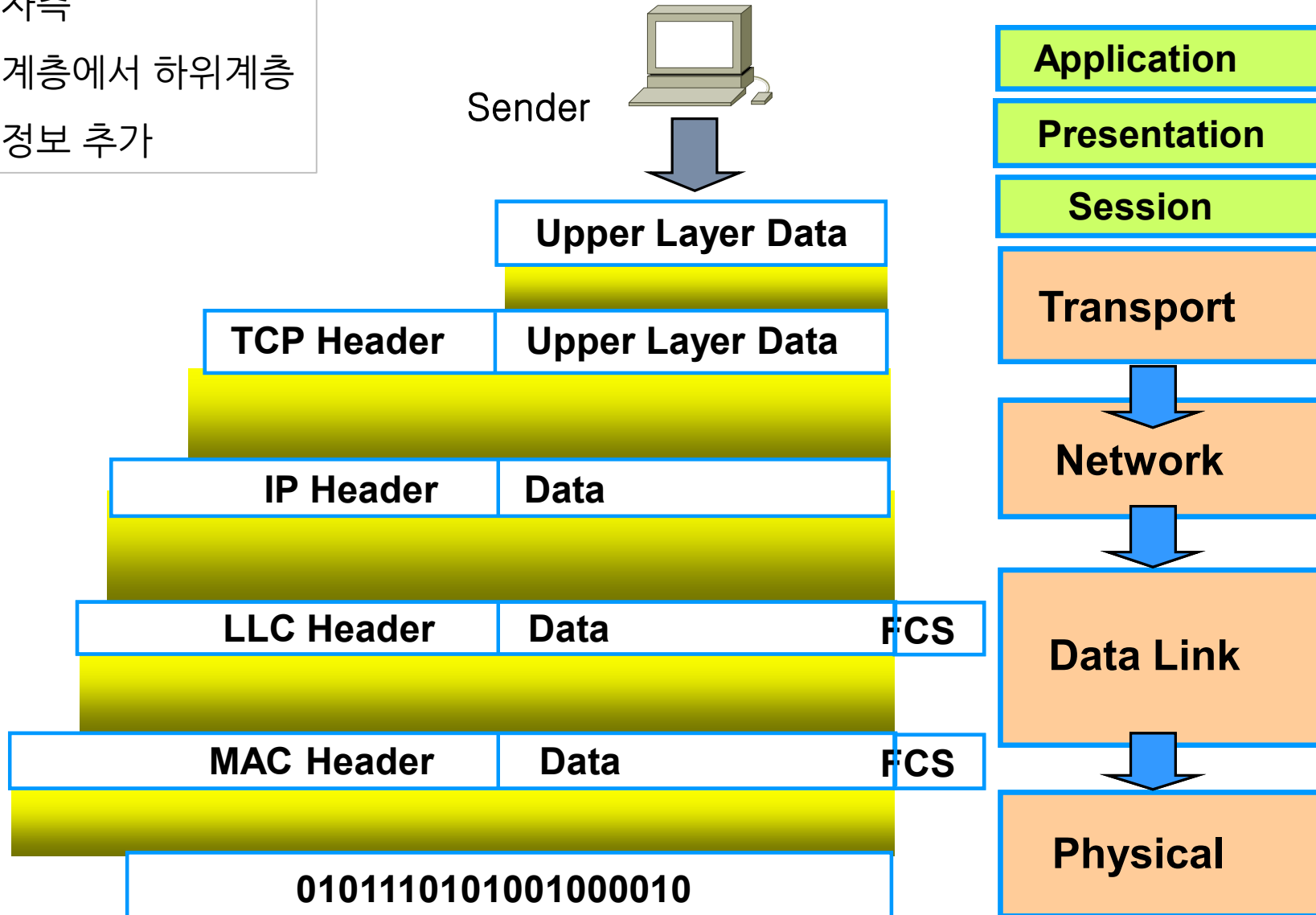
전기적, 기계적, 기능적, 절차적인 수단 제공

Payload & Encapsulation



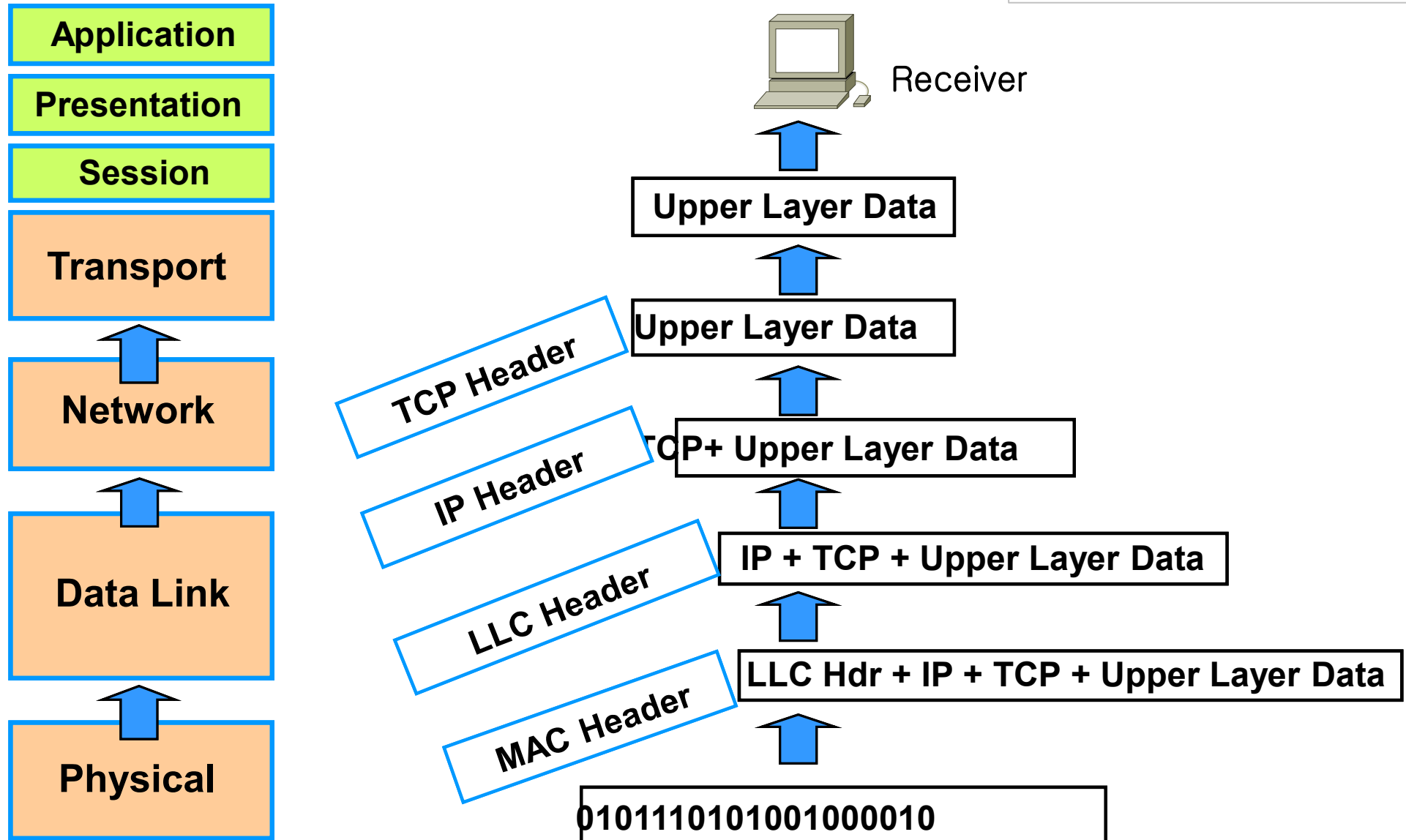
Encapsulation(캡슐화)

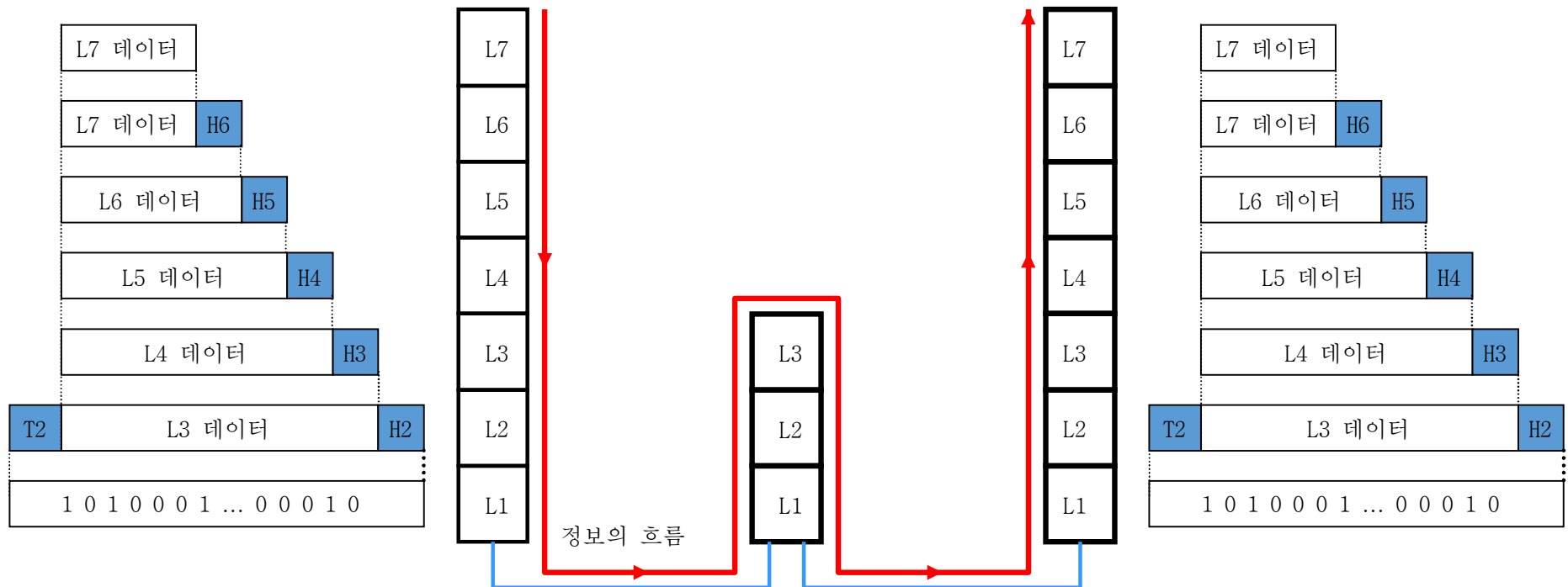
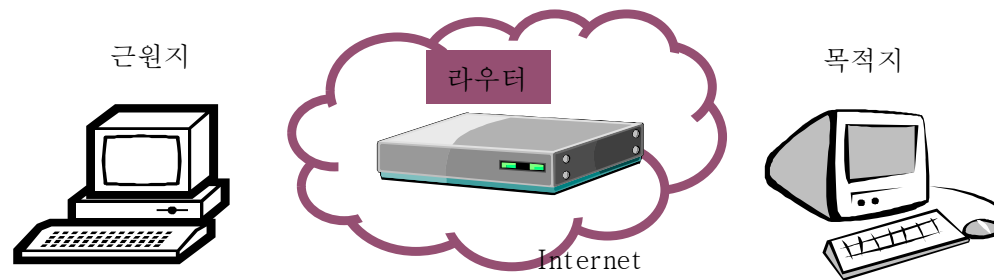
- 송신자측
- 상위계층에서 하위계층
- 제어정보 추가



De-encapsulation(역캡슐화)

- 수신자측
- 하위계층에서 상위계층
- 제어정보 제거



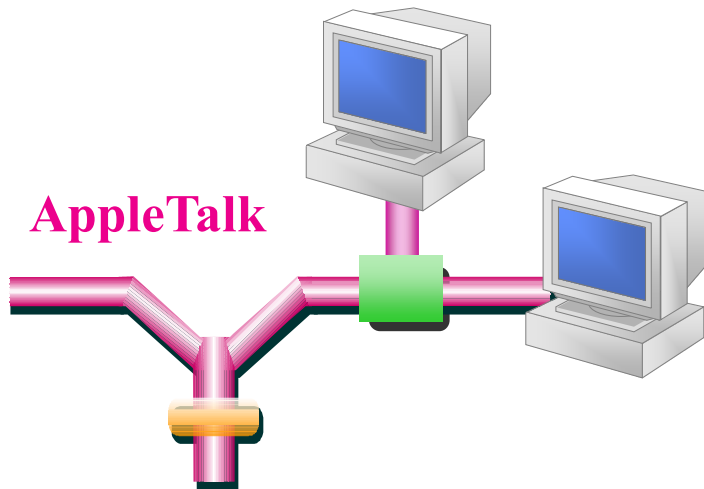


2. 통신 모델과 Protocol

통신체계	프로토콜 종류	용도
TCP/IP	HTTP, Telnet, TCP, UDP...	인터넷과 LAN에서 사용
IPX/SPX	IPX, SPX, NPC...	Novel 사가 개발 및 판매하는 Netware 시스템에서 사용
Appltalk	DDP, RTMP, ATP...	Apple사 제품의 LAN에서 사용
DECnet	DPR, NSP, SCP...	구 DEC사의 미니컴퓨터에서 사용
XNS	IDP, SPP, PEP...	Xerox 사의 네트워크에 사용

통신규약(Protocol)

◎ Vendor-Specific Protocols



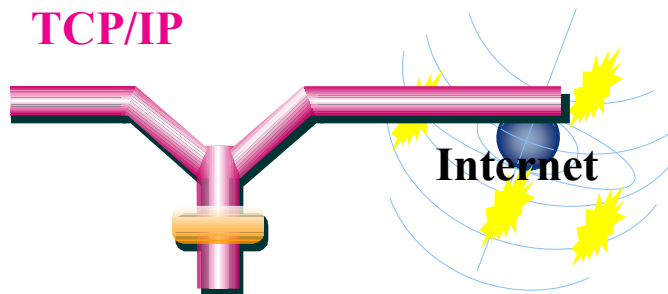
- 독점적 프로토콜(Vendor-Specific Protocols)

- 특정 업체에서 프로토콜 개발
- 타 프로토콜과 호환 불가
- SNA, IPX/SPX, AppleTalk 등

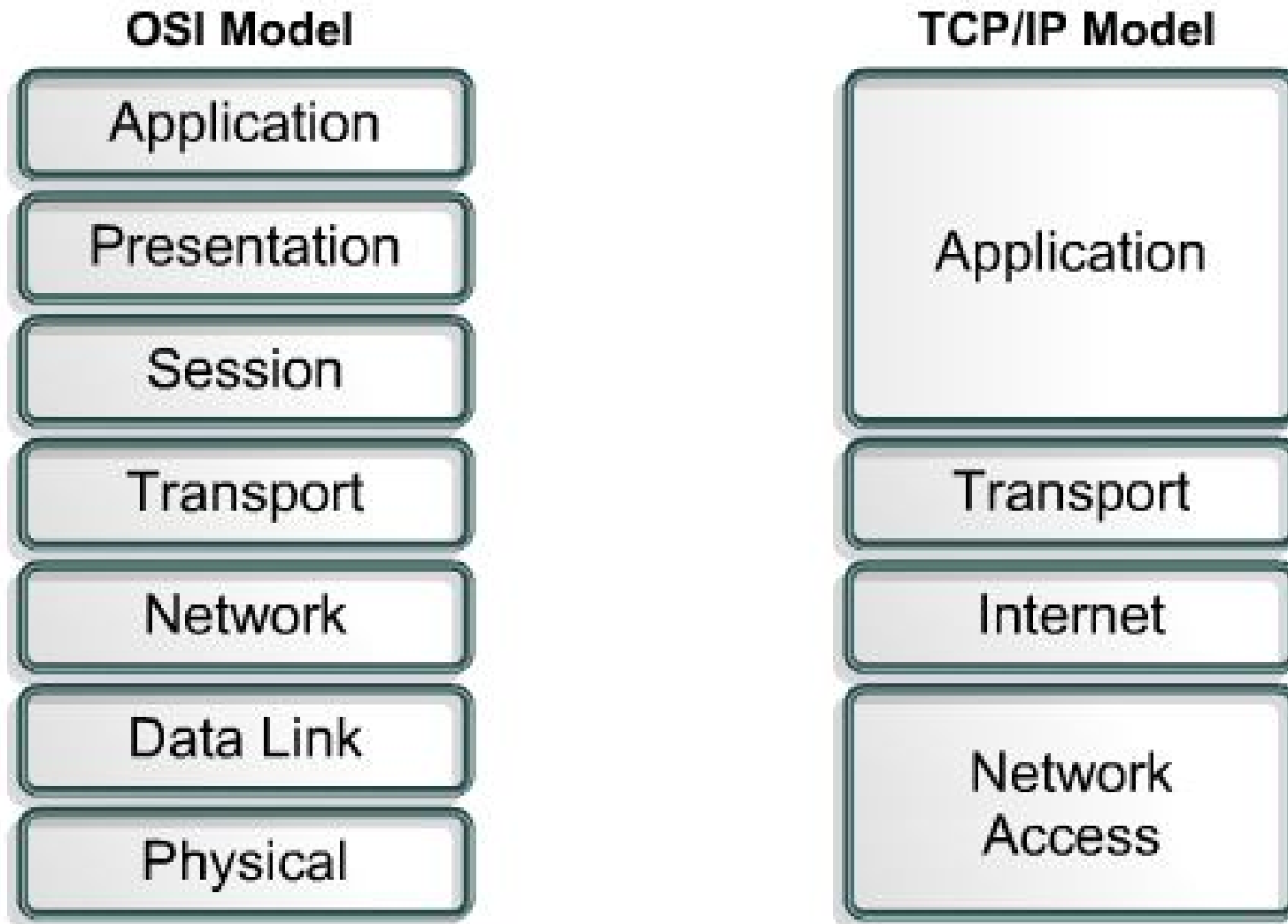
- 비독점적 프로토콜(Open Protocols)

- 학교 및 연구기관에 의해 개발
- 이종 장비간 통신 가능
- TCP/IP, 802.3 등

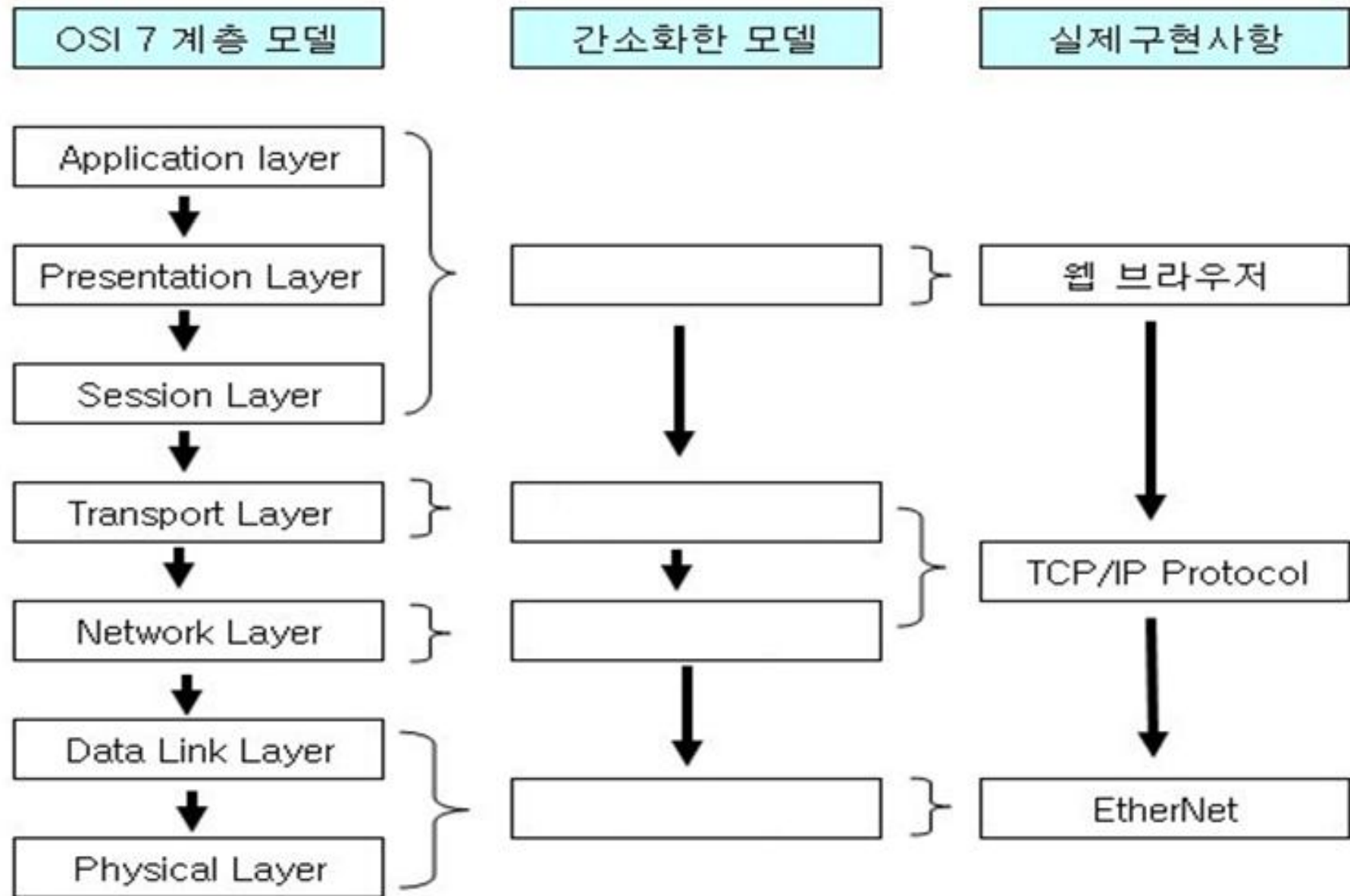
◎ Open Protocols



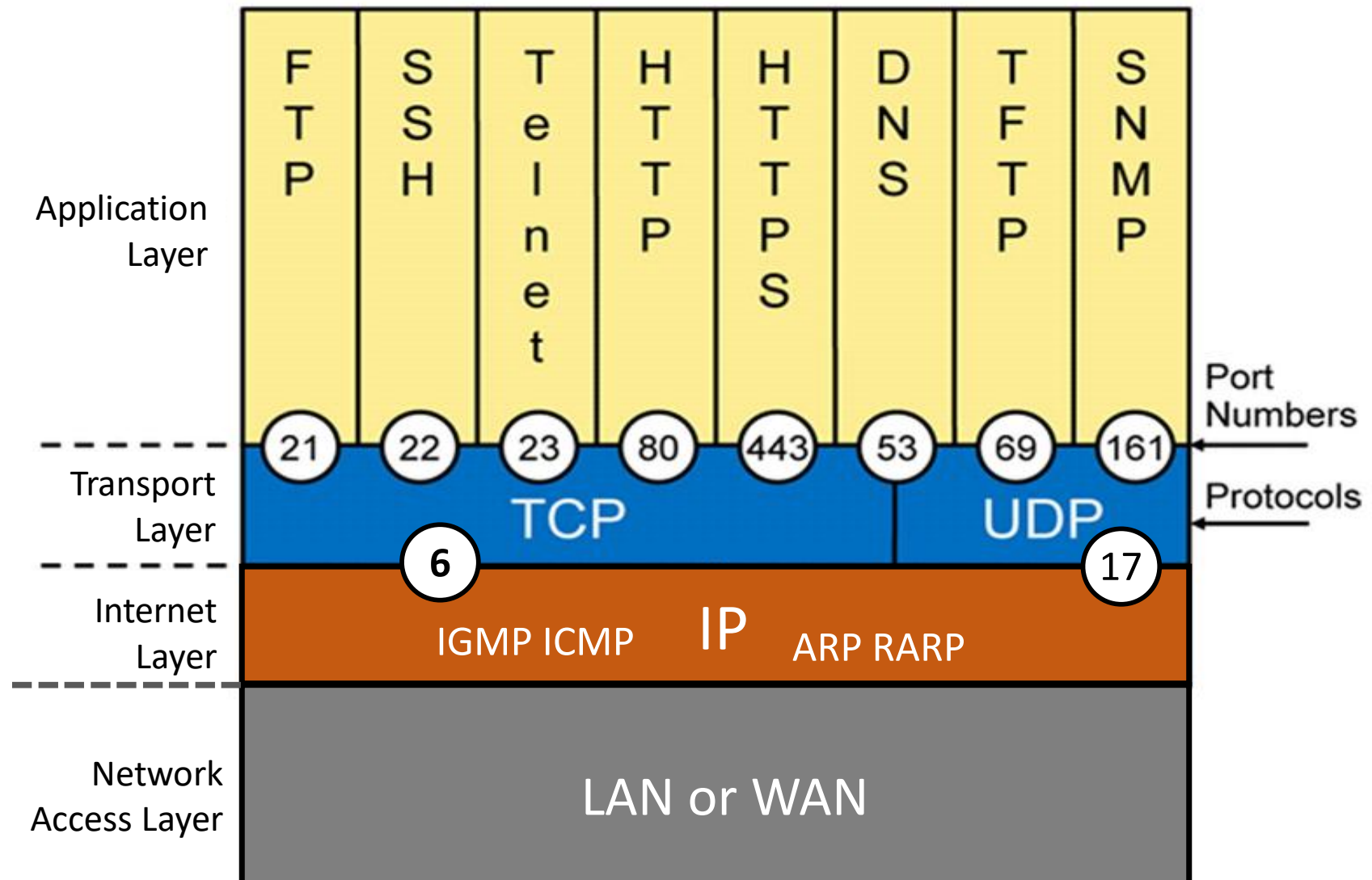
TCP/IP 통신 모델



OSI Reference Model과 TCP/IP 비교

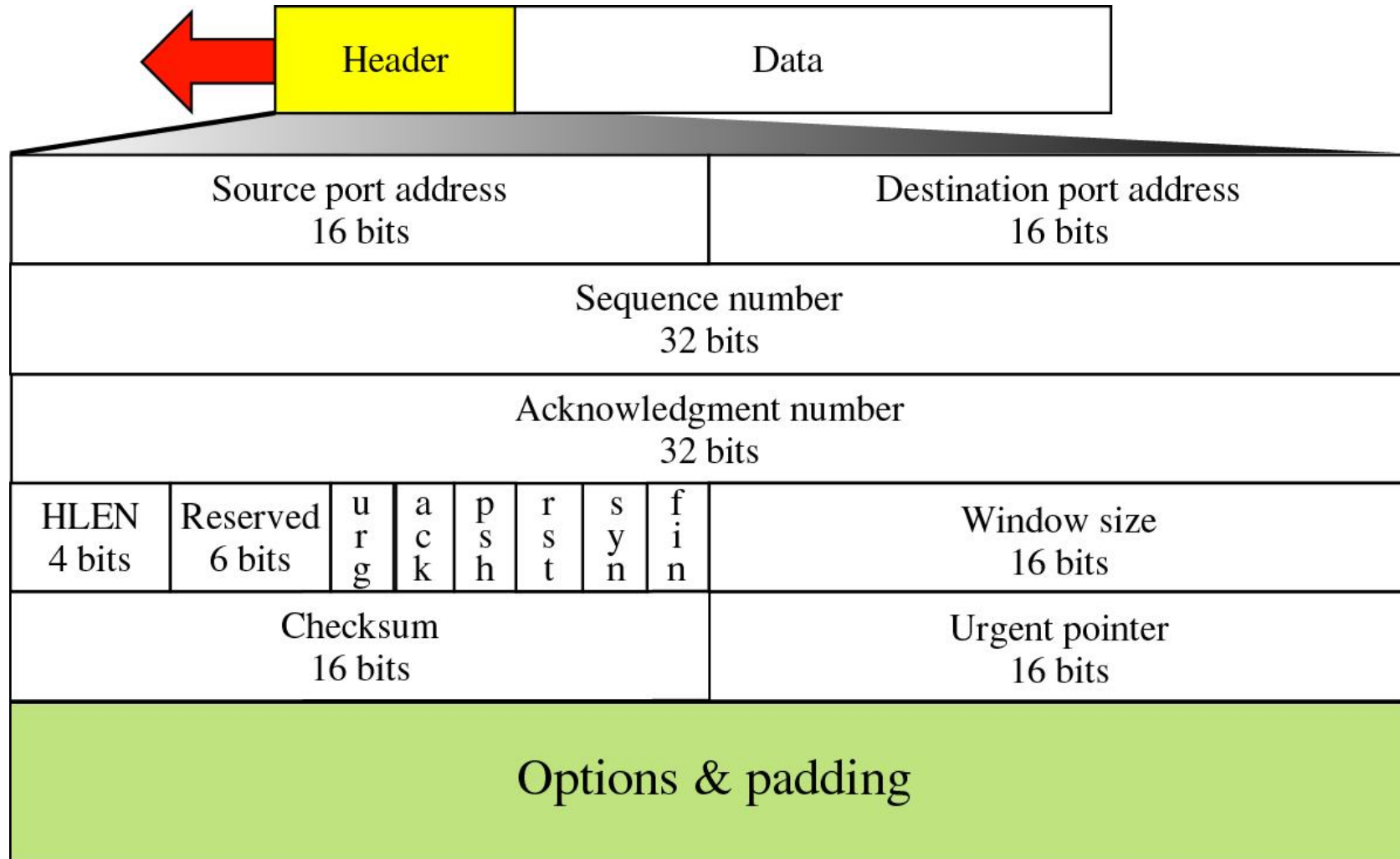


3. TCP/IP Protocol Stack

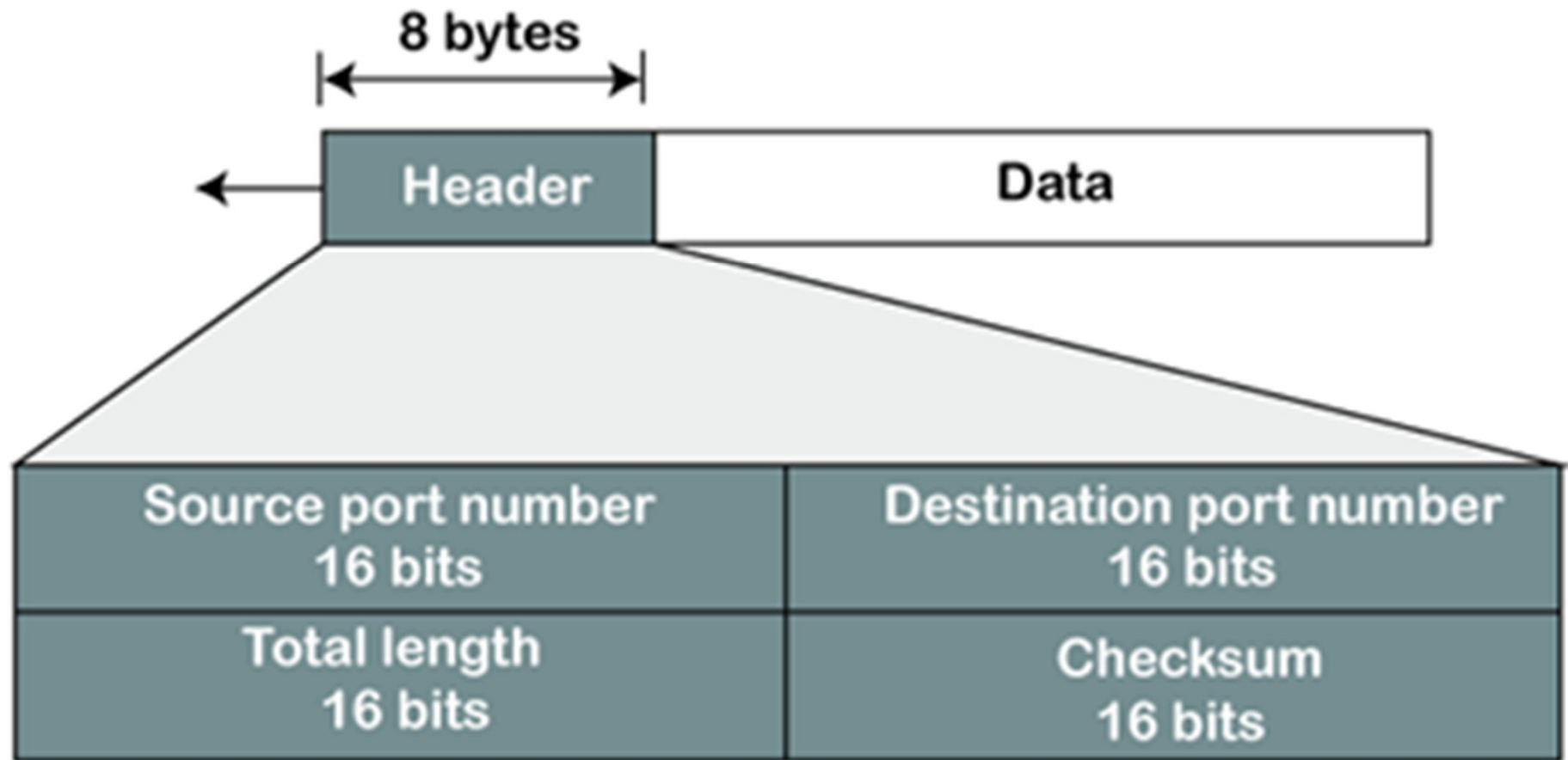


Header 구조

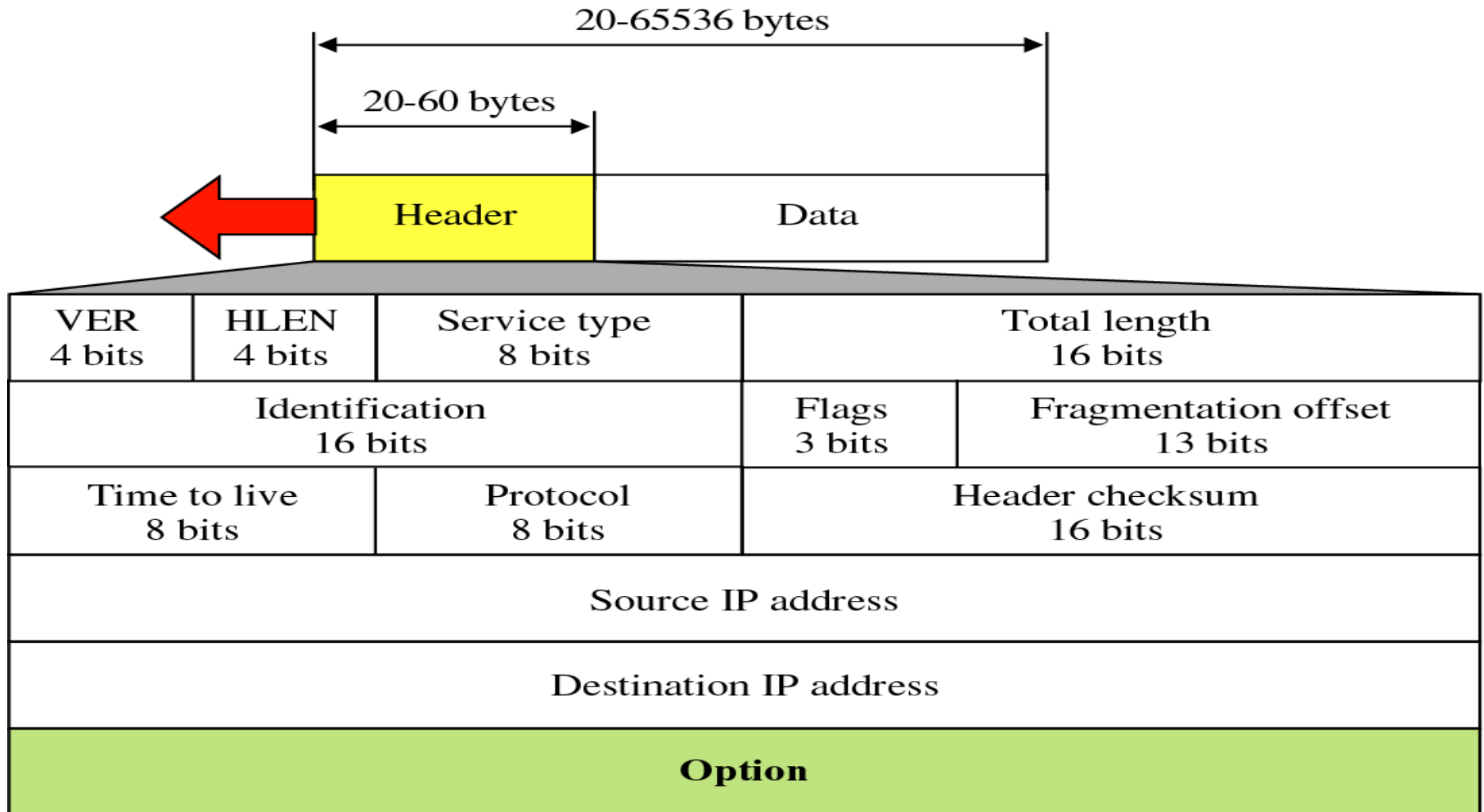
TCP(Transmission Control Protocol) Header



UDP(User Datagram Protocol) Header



IP(Internet Protocol) Header

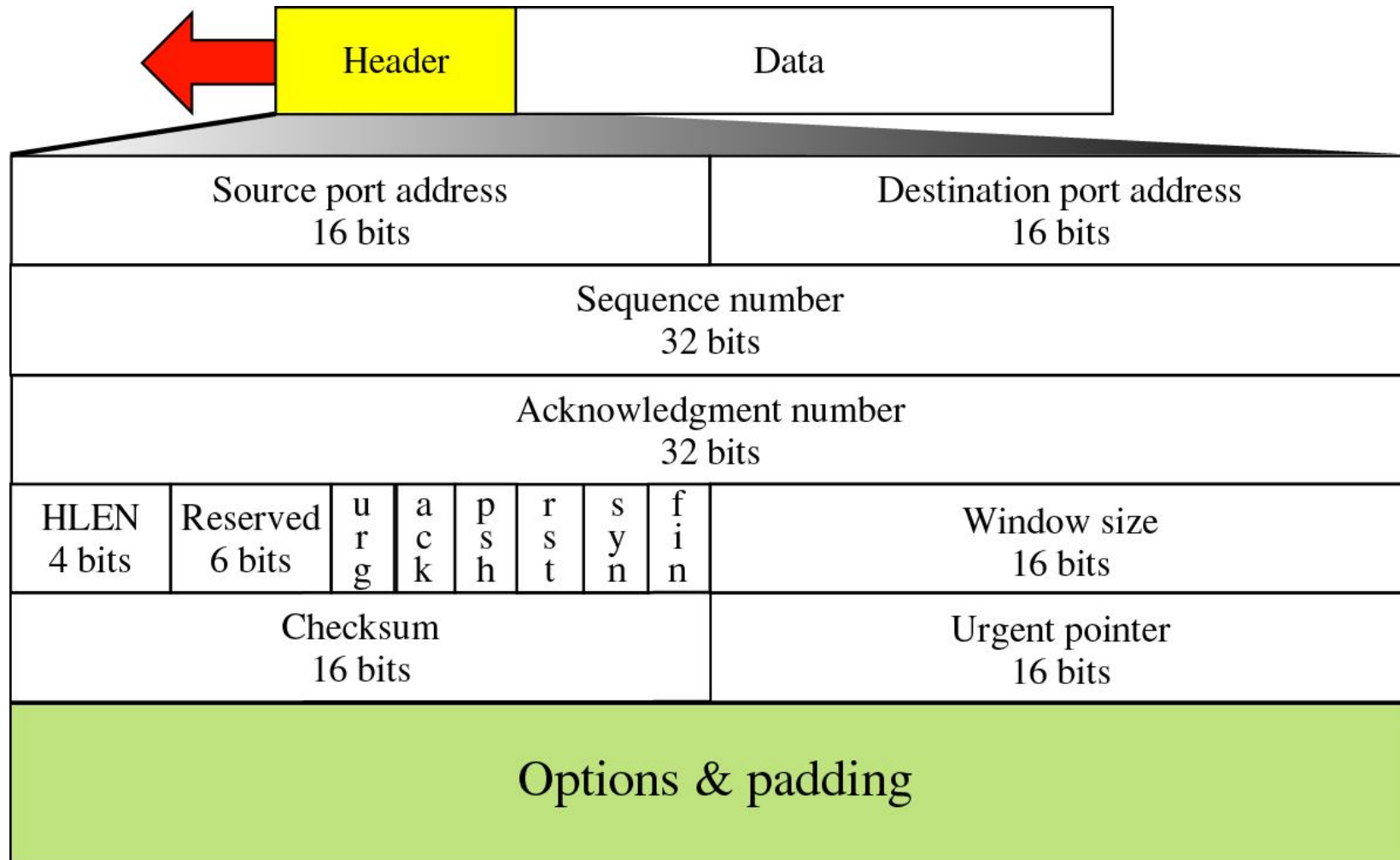


Ethernet Frame



3.1 TCP Header 구조 분석

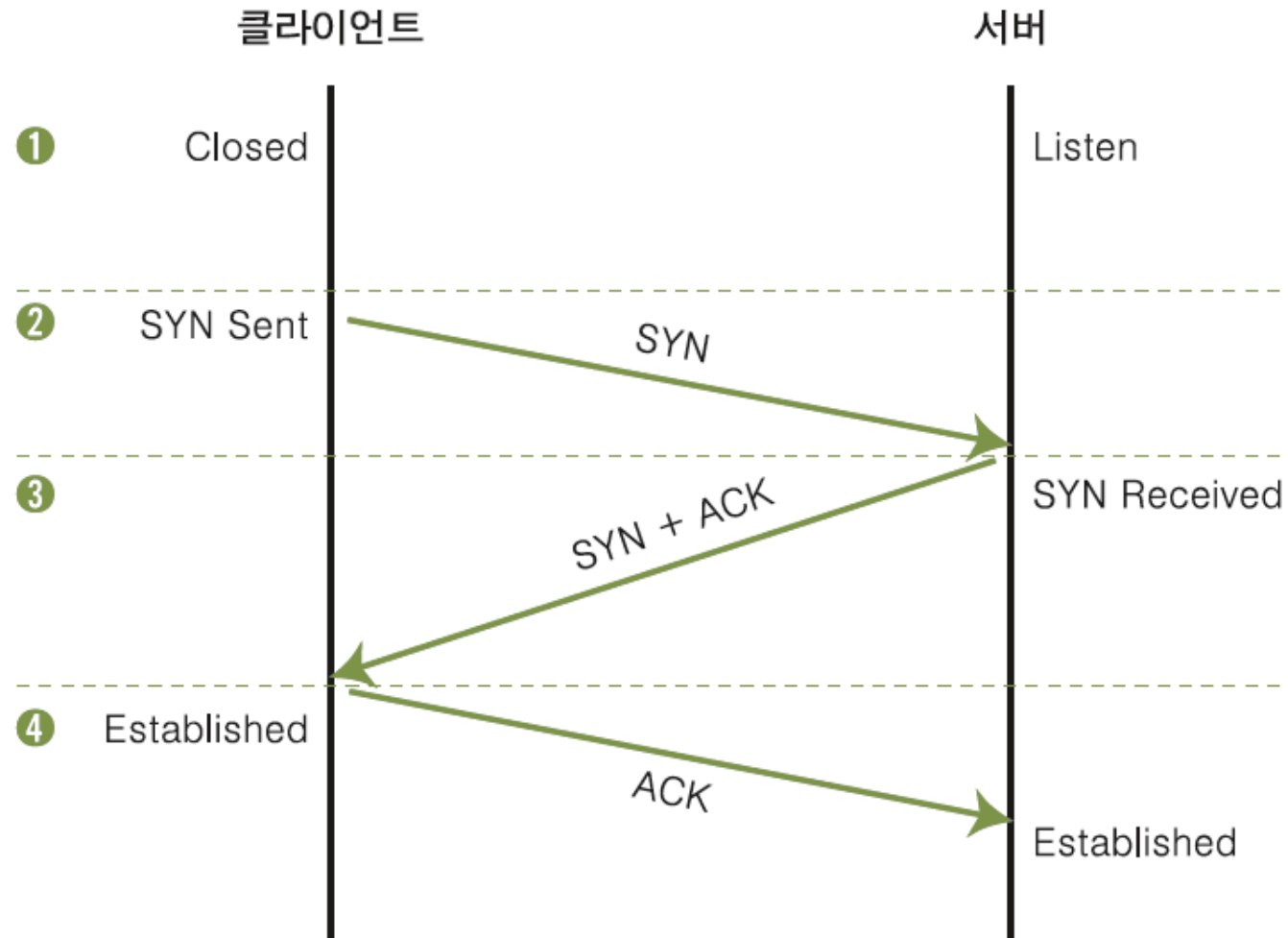
- Transmission Control Protocol
- TCP 헤더는 기본이 20~60바이트로 구성



1) TCP 연결 관리

- ① 데이터 전송 전 : 3 way handshake
- ② 데이터 전송 중 : 신뢰성, 오류제어, 흐름제어
- ③ 데이터 전송 후 : 4 way handshake

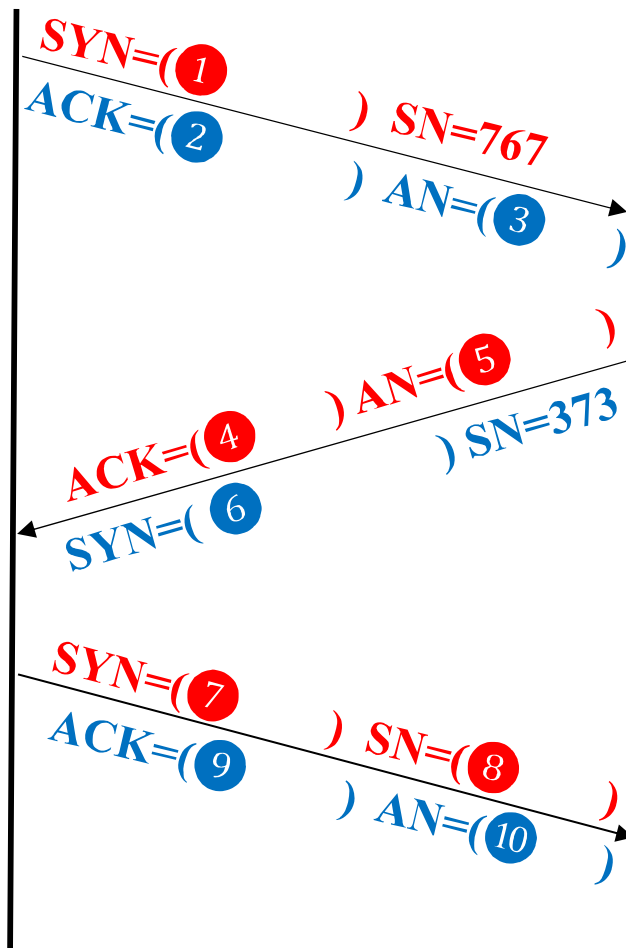
데이터 전송 전 (3-Way Handshaking)



* 3WHS의 일부분이다. 괄호부분을 모두 합한 값은?

Client

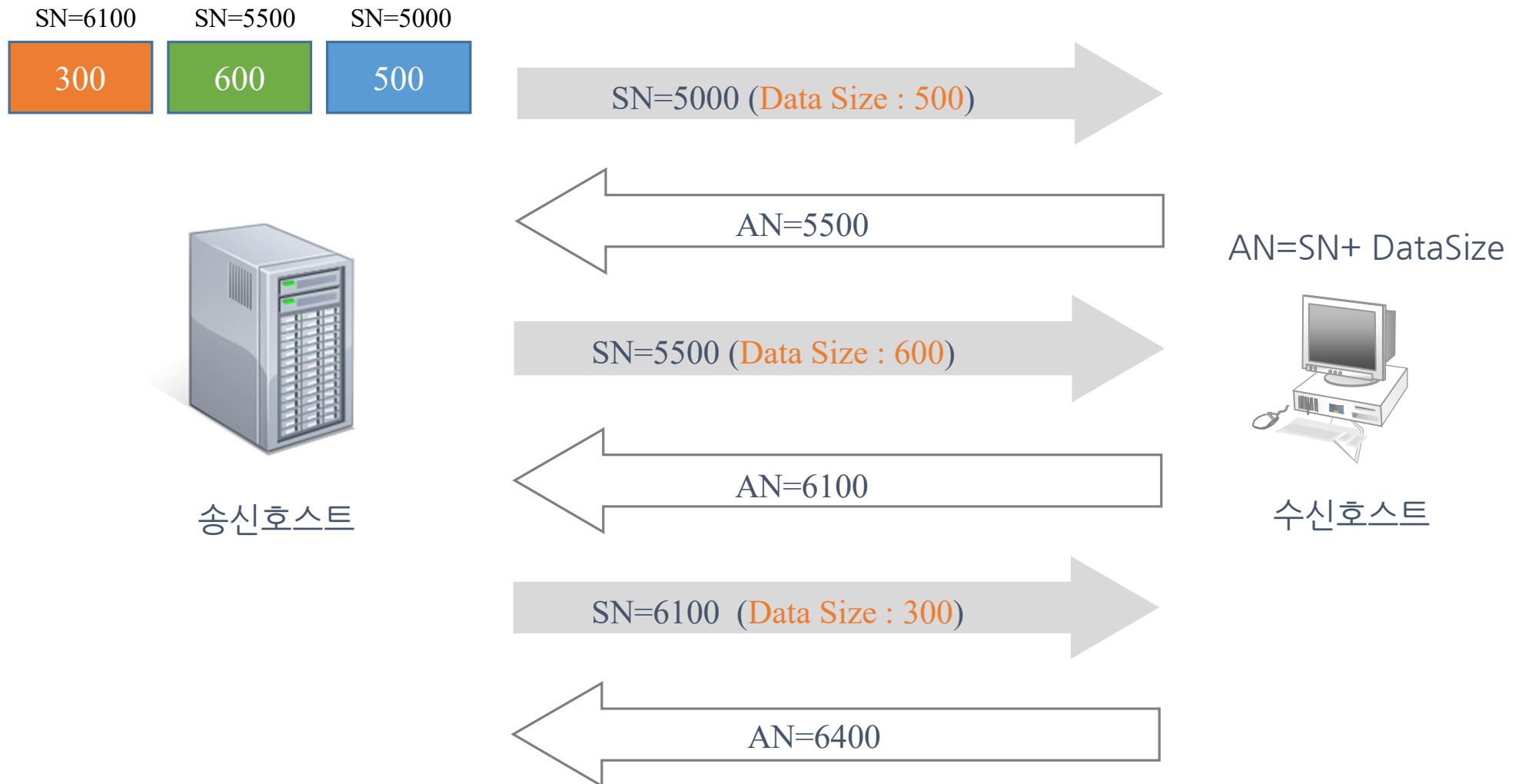
Server



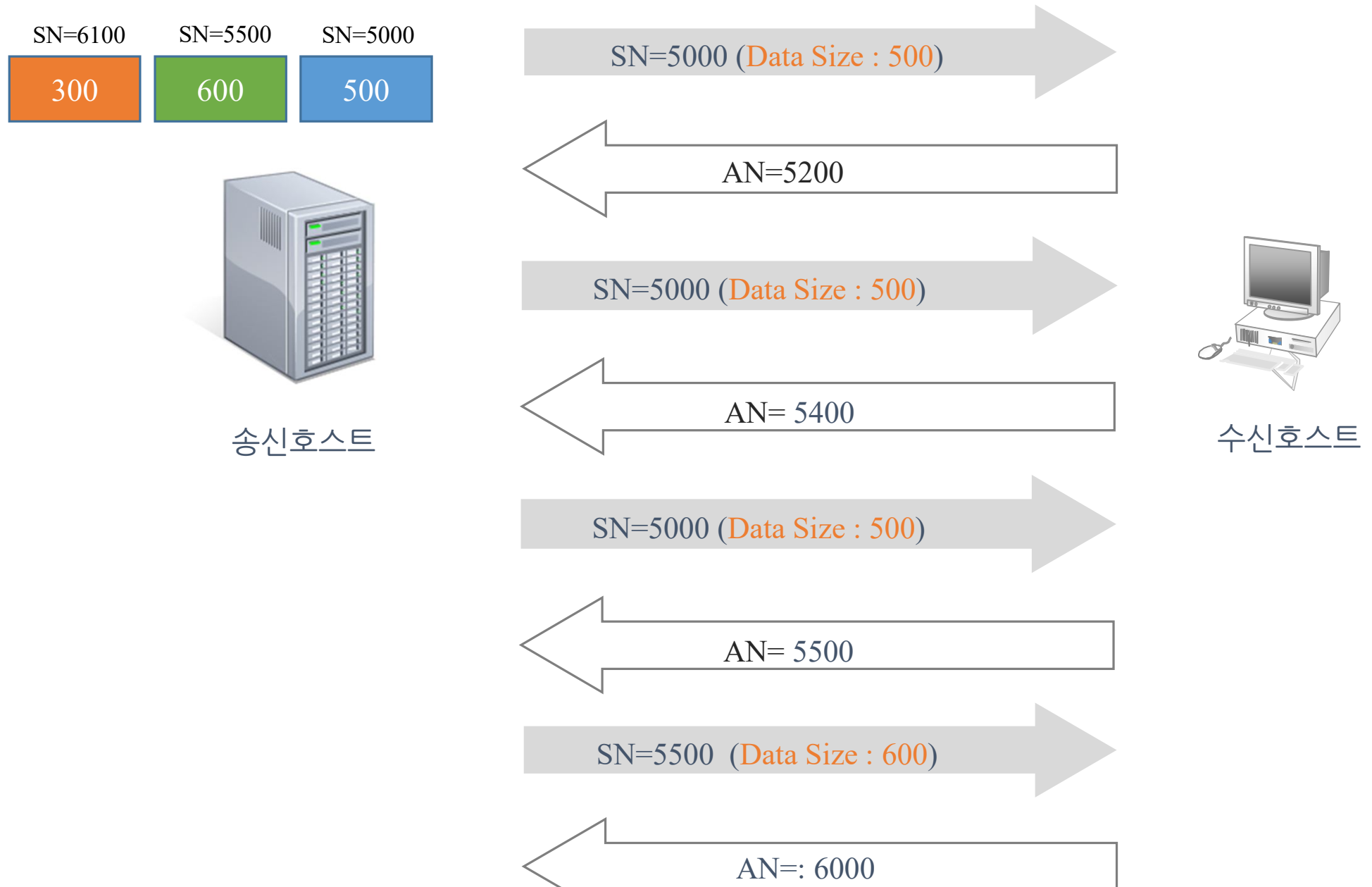
$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10$$

데이터 전송 중

① 정상적인 트래픽 전송 과정

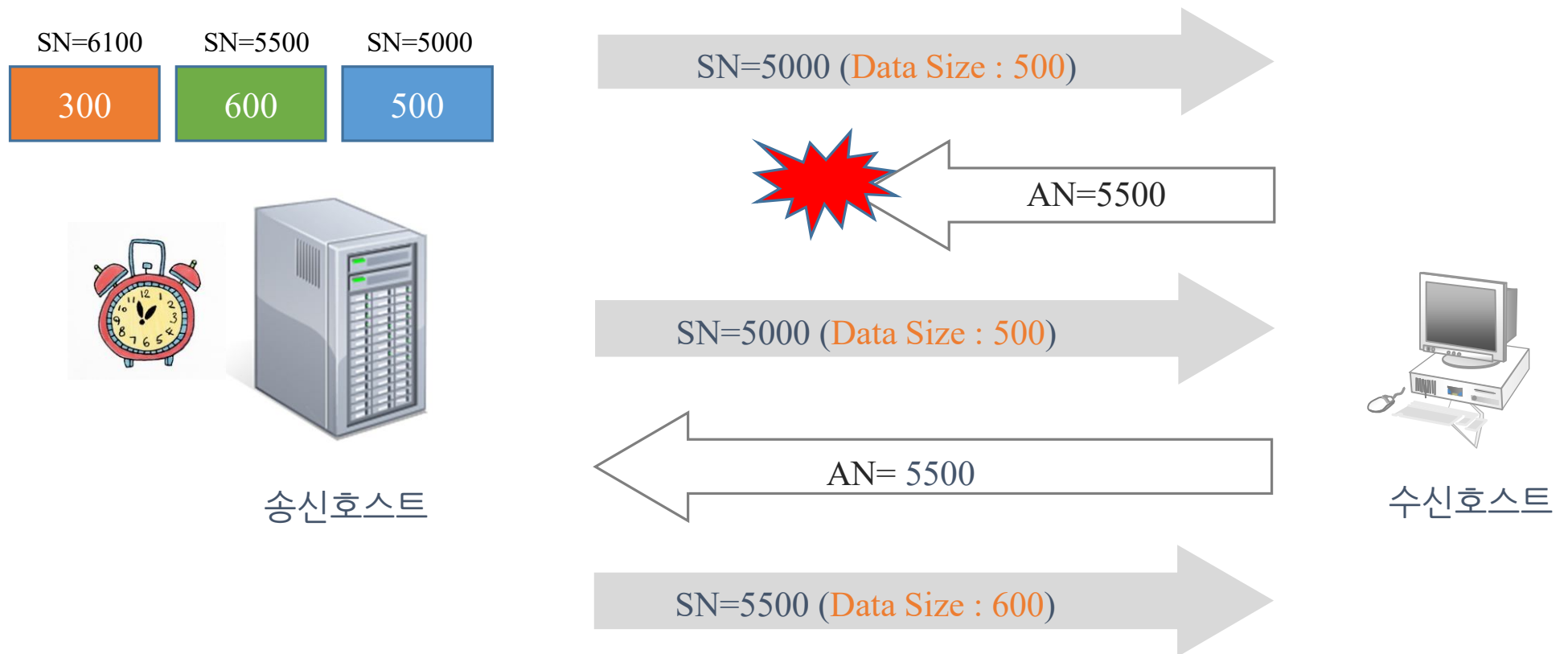


② 비정상적인 트래픽 전송 과정



② 비정상적인 트래픽 전송 과정

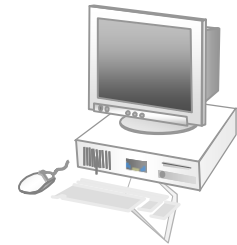
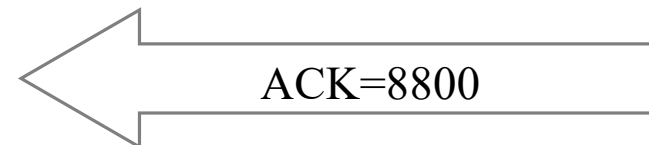
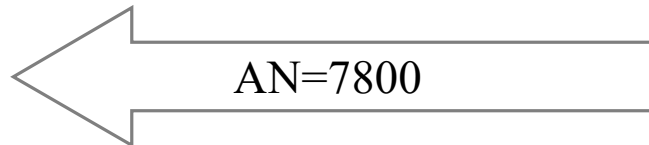
(예) 송신측에서 재전송 시도



3 트래픽 흐름제어



송신호스트

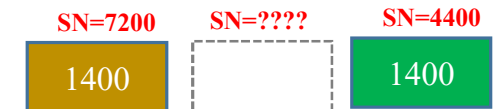
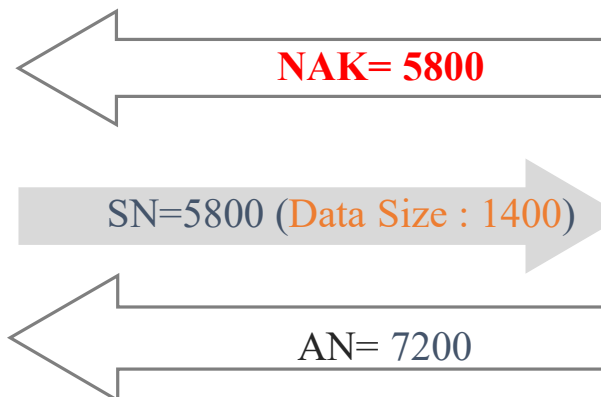
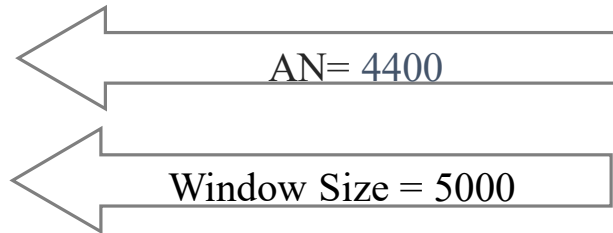
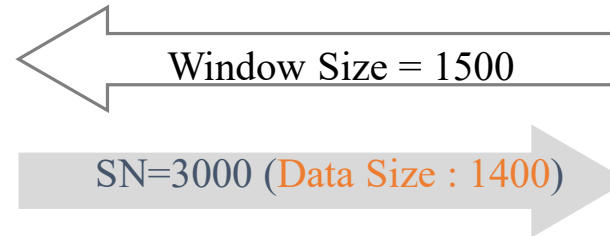


수신호스트

③ 트래픽 흐름제어



송신호스트

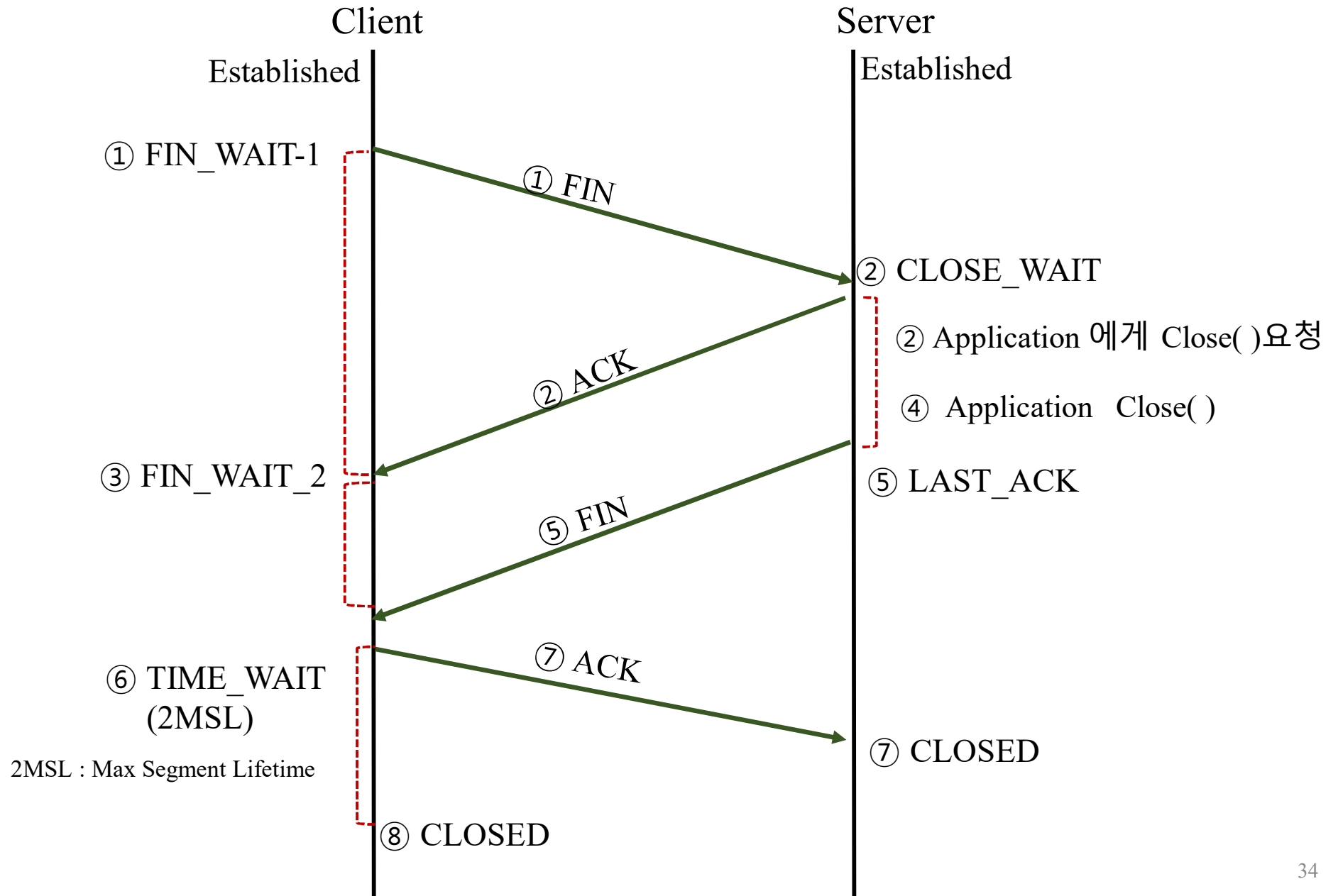


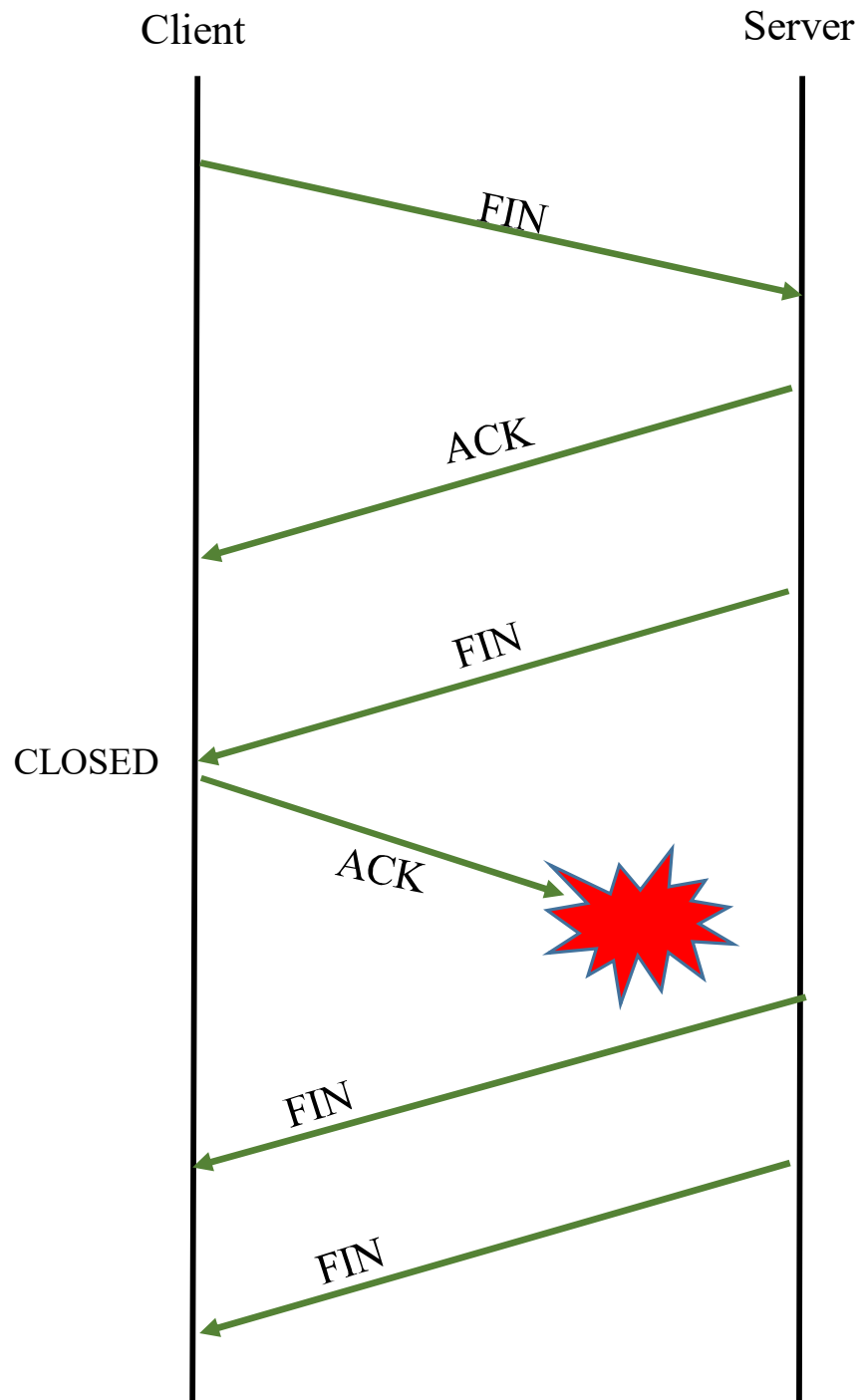
수신호스트

③ 트래픽 흐름제어



데이터 전송 후 (4-way Handshake)



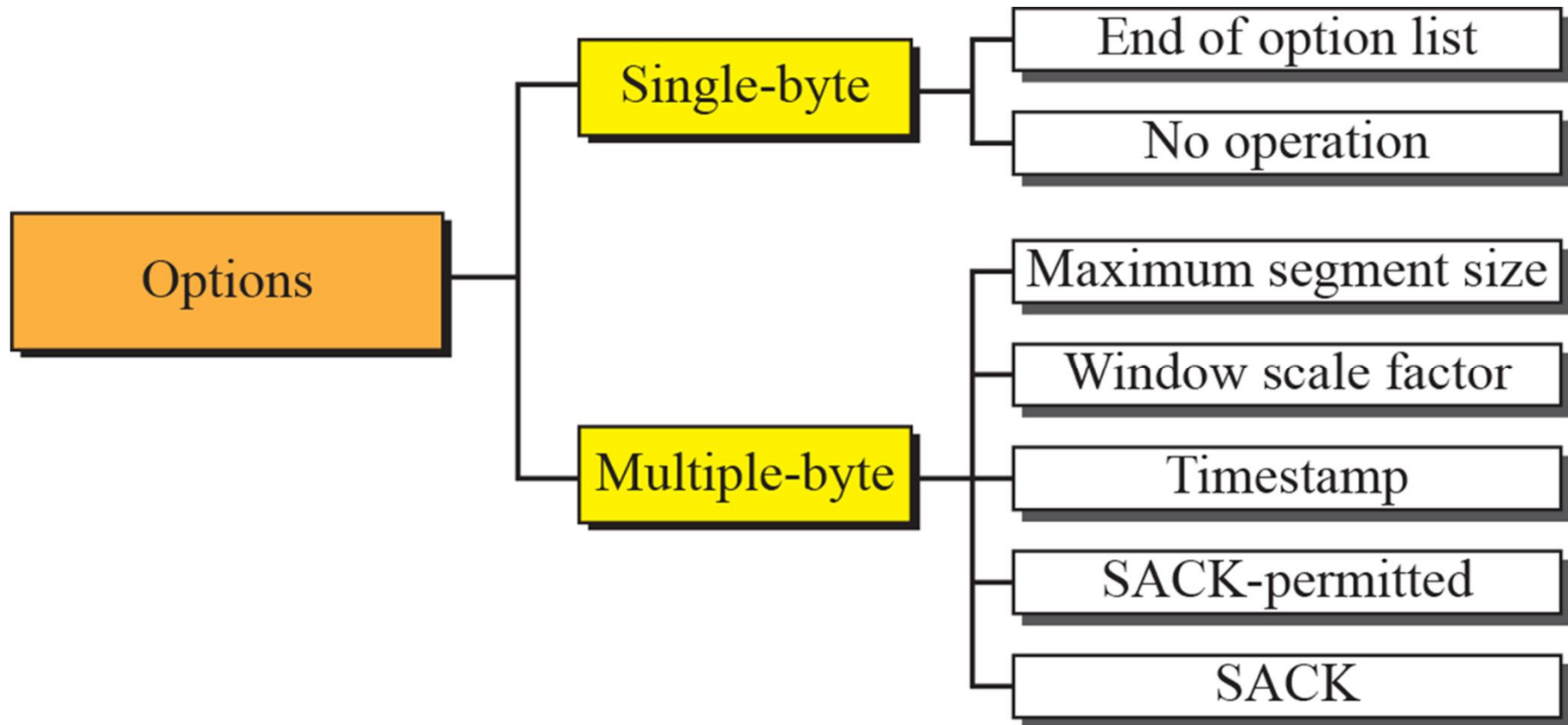


* Server에서는 소켓을 종료하지 못하게 됨

2) TCP Checksum 생성 방법

- ① IP 프로토콜 + 송신지 IP + 수신지IP + TCP Total length
- ② TCP 헤더의 합 (Checksum 필드제외)
- ③ TCP 데이터의 합
- ④ '**①+②+③**'
- ⑤ 자리 올림 값의 합산
- ⑥ 합산 결과를 이진화 후 1의 보수화

3) TCP Option Header



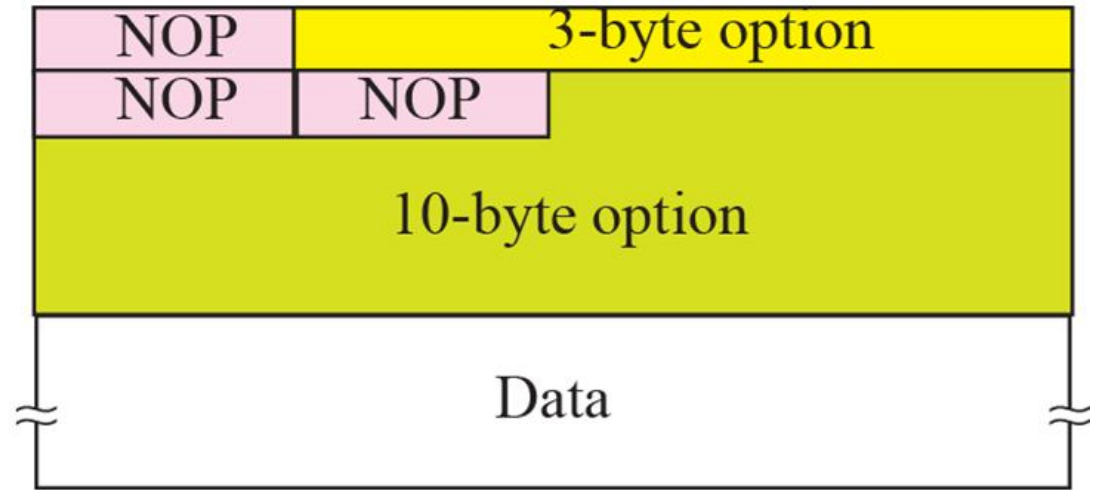
- 최대 40바이트의 길이까지 사용 가능
- 1 바이트 옵션과 다중 바이트 옵션이 제공

Type 1 byte	Length 1 byte	Meaning	
0		End of option list (EOL)	
1		No operation (NOP)	
2	4	Maximum Segment Size (MSS) 2 B	
3	3	Window Scale Factor (Shift count) 1 B	
8	10	Timestamp Value 4 B	Timestamp Echo Reply 4 B
11	6	Connection Count 4 B	
12	6	New Connection Count 4 B	
13	6	Connection Count Echo 4 B	

No-Operation option(NOP)



a. No operation option



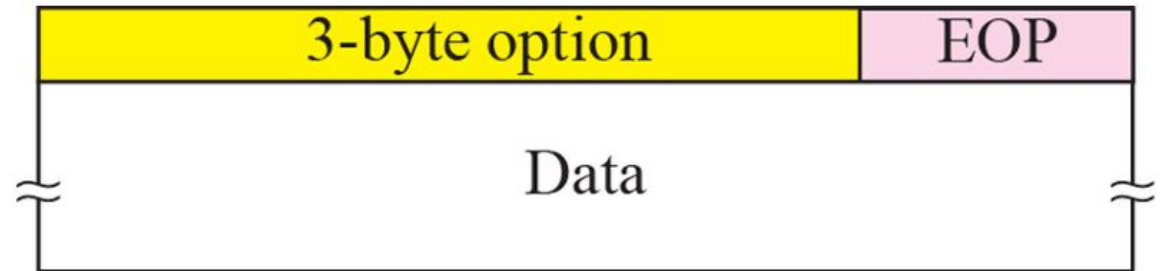
b. Used to align beginning of an option

- NOP: 빈 칸 채우기(padding) 용
 - TCP 헤더는 블록당 32bits(4byte)을 맞춰야 함
- 1byte나 2byte 등 원하는 byte만큼 띄우고 옵션을 시작할 수 있음
- 보통 뒤에는 EOP, 앞에는 NOP를 사용한다.
- NOP는 여러 번 사용 가능

End of Option(EOP)



a. End of option list



b. Used for padding

- EOP: 빈칸 채우기(padding) 용
- 3-byte option 뒤 빈 공간 1byte는 EOP로 채움
- EOP는 오직 한번만 사용된다.

MSS (Maximum Segment Size)

Kind: 2 00000010	Length: 4 00000100	Maximum segment size
1 byte	1 byte	2 bytes

- 목적지에서 수신할 수 있는 세그먼트 데이터 부분의 최대 크기
- 16비트이므로 최대 65535바이트의 값을 가질 수 있음
- 수신 호스트에 의해 결정되며 하위 계층의 MTU값에 의해 결정
- TCP의 연결 성립과정, SYN Flag와 함께 사용해 상대 시스템의 MSS를 파악된 이 후의 세그먼트에서는 사용 안 됨

Windows Scale Factor(윈도우 확장 인자)

- 윈도우 확장 인자 바이트는 1바이트
- 기본 헤더의 윈도우 최대 크기 (16비트)
 - 65535바이트이므로 이는 현실적으로 충분하지 않을 경우가 있음

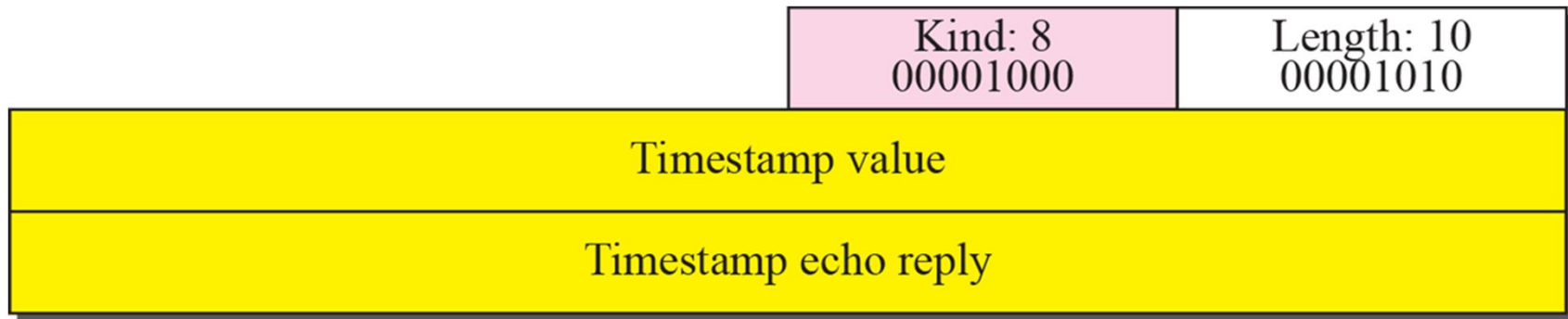
(예) 광섬유를 이용한 전송을 할 경우 65535바이트 이상으로 전송할 필요가 있음

- 이 때 이 옵션을 사용해 윈도우 크기를 확장할 필요가 있을 경우 사용
- 이 항목의 값이 2이면 공식에 의해 $65535 * 2^2$ 의 데이터를 전송할 수 있음

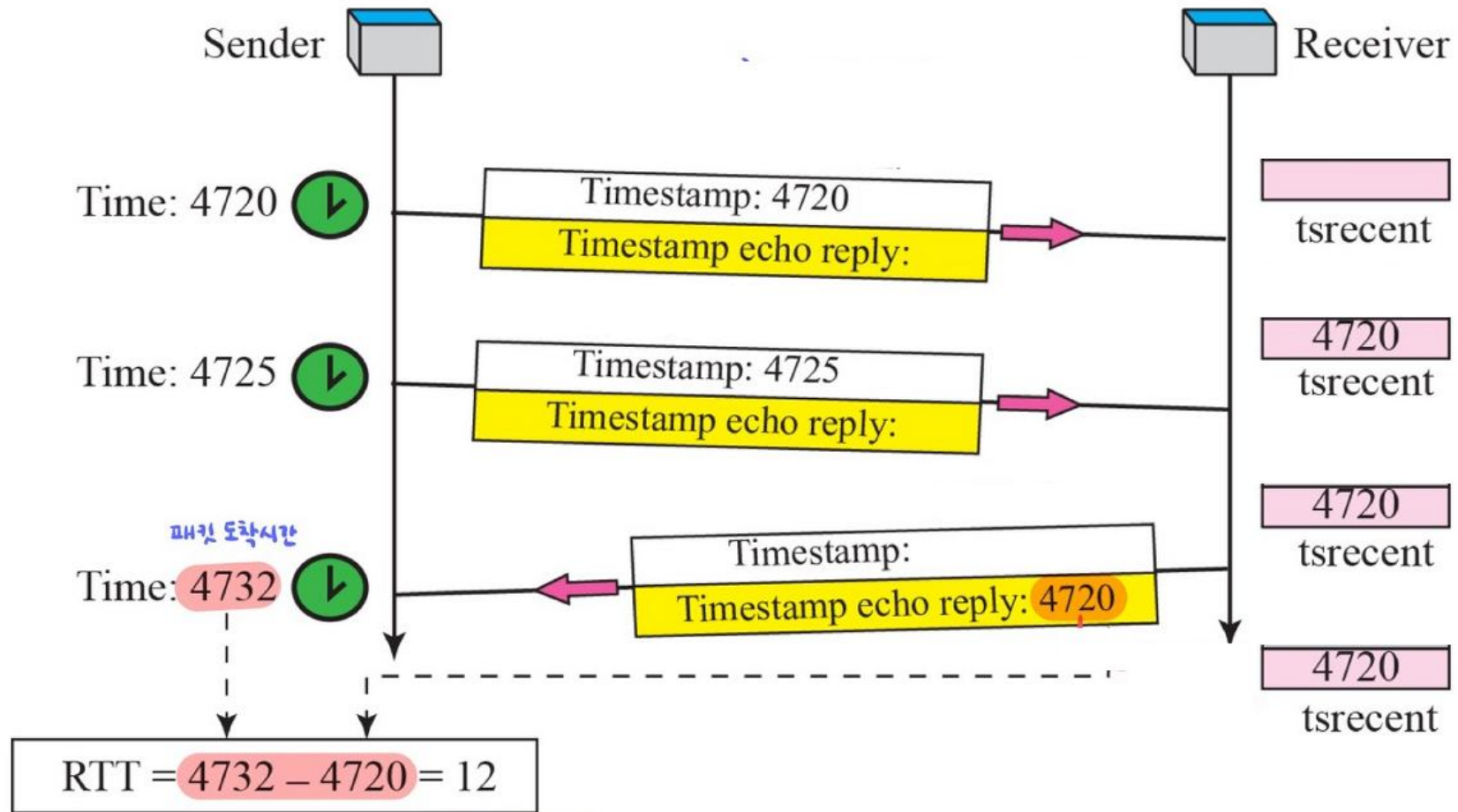
Kind: 3 00000011	Length: 3 00000011	Scale factor
1 byte	1 byte	1 byte

새로운 윈도우 크기 = 기본 헤더의 Windows 크기 * $2^{\text{윈도우확장인자}}$

Time stamp



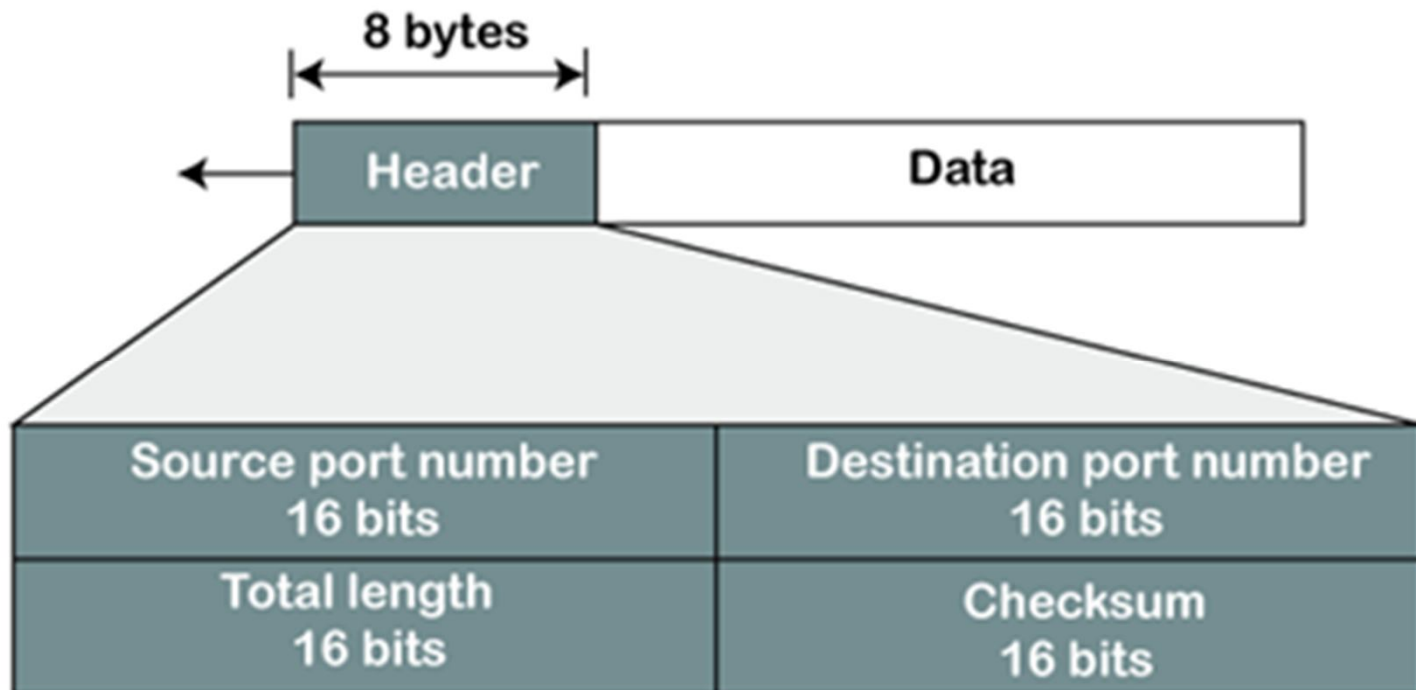
- Timestamp value: 출발시간, 도착시간 기록(RTT 측정 시 사용)
- TCP에서는 RTT (Round Trip Time)에 기초해 재전송 타임 아웃 길이를 결정



- TSval(Time Stamp value) : 송신 시각
- TSecr(Time Stamp echo reply) : 상대가 받은 Tsval을 그대로 복사해서 반환
- RTT(왕복시간) = 현재 TSval - TSecr
- * Delta Time : 패킷 간 간격

3.2 UDP Header 구조 분석

- User Datagram Protocol
- Data 전송을 위하여 사전에 필요한 Process가 없음
- Best-Effort Delivery
- 신뢰성 보장 못함



UDP Checksum 생성 방법

- ① IP 프로토콜 + 송신지 IP + 수신지IP + UDP Header length
- ② UDP 헤더의 합 (Checksum 필드제외)
- ③ UDP 데이터의 합
- ④ ‘**①+②+③**’
- ⑤ 자리 올림 값의 합산
- ⑥ 합산 결과를 이진화 후 1의 보수화

UDP

- Message-Oriented Transport Protocol
- 상위계층에서 payload 된 데이터를 단편화하지 않음

TCP

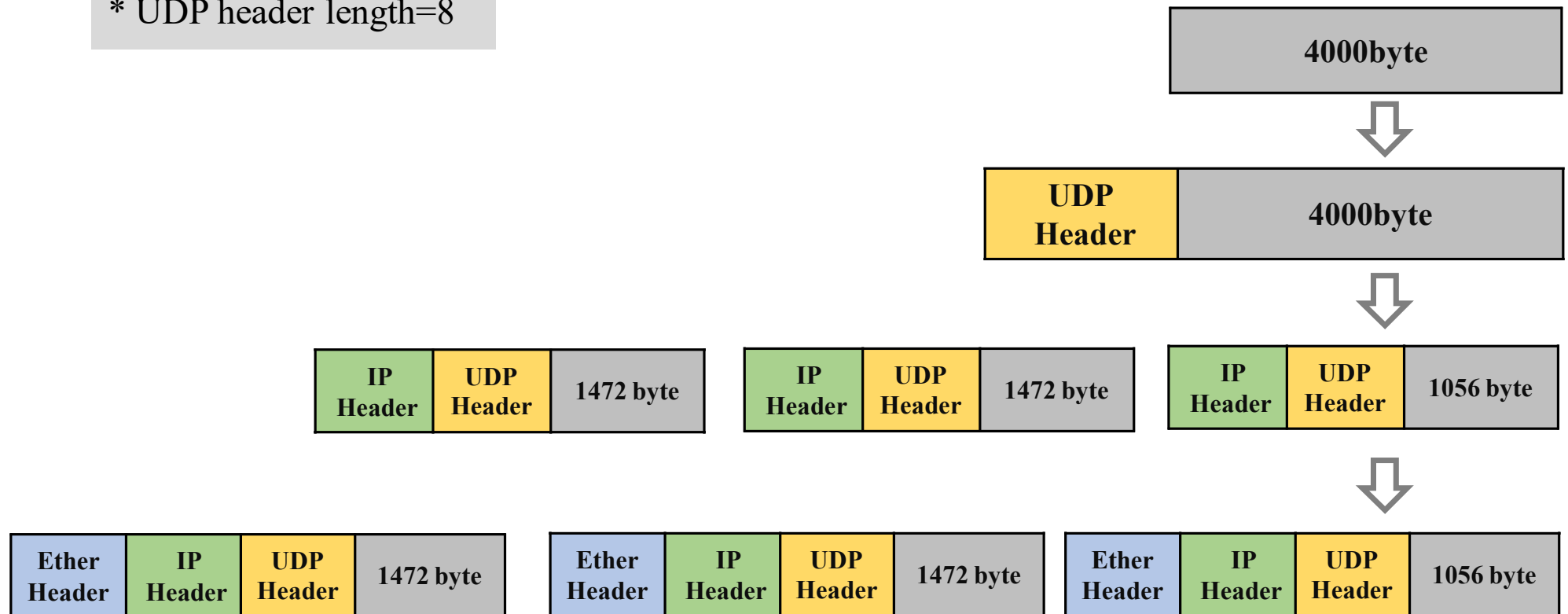
- Stream-Oriented Transport Protocol
- 상위계층에서 payload 된 데이터를 단편화

UDP 기반의 애플리케이션 데이터 전송

- Message-oriented transport protocol

* IP header length=20

* UDP header length=8

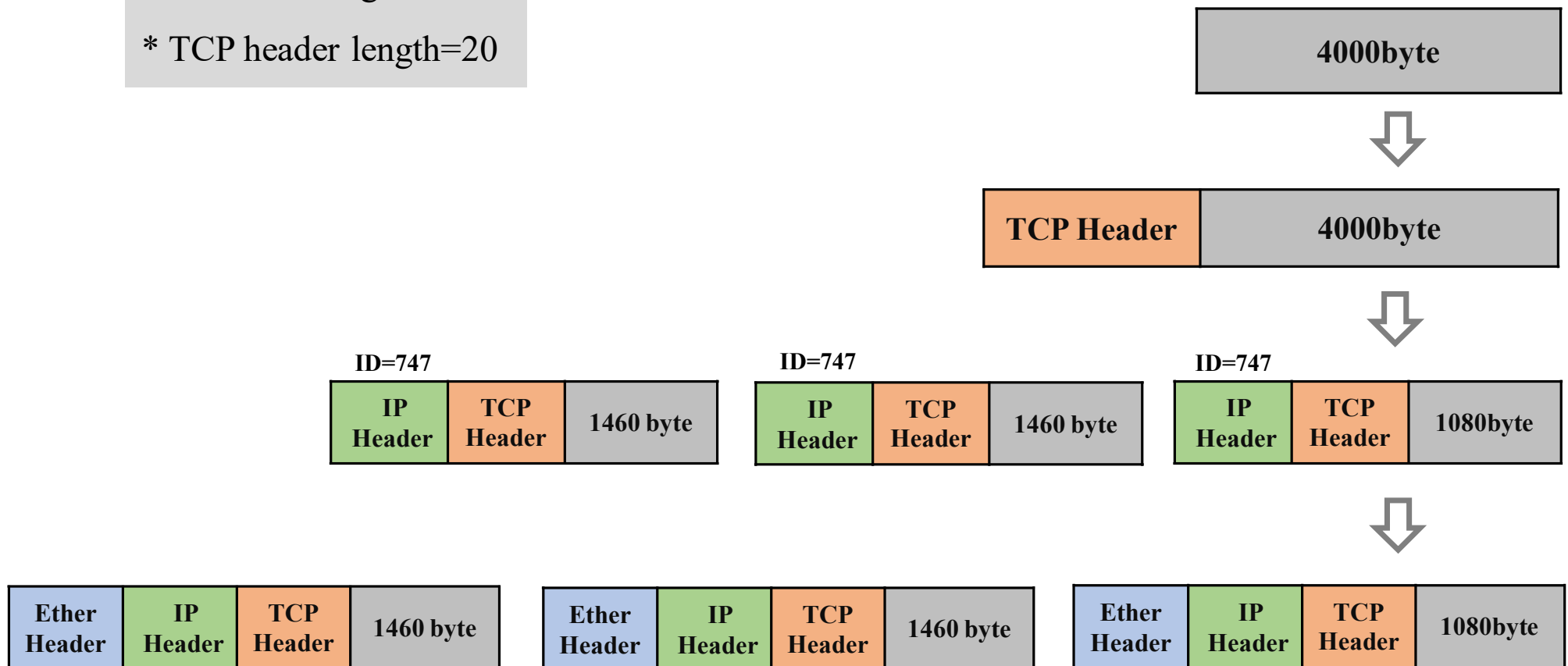


TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 5000byte인 경우

* IP header length=20

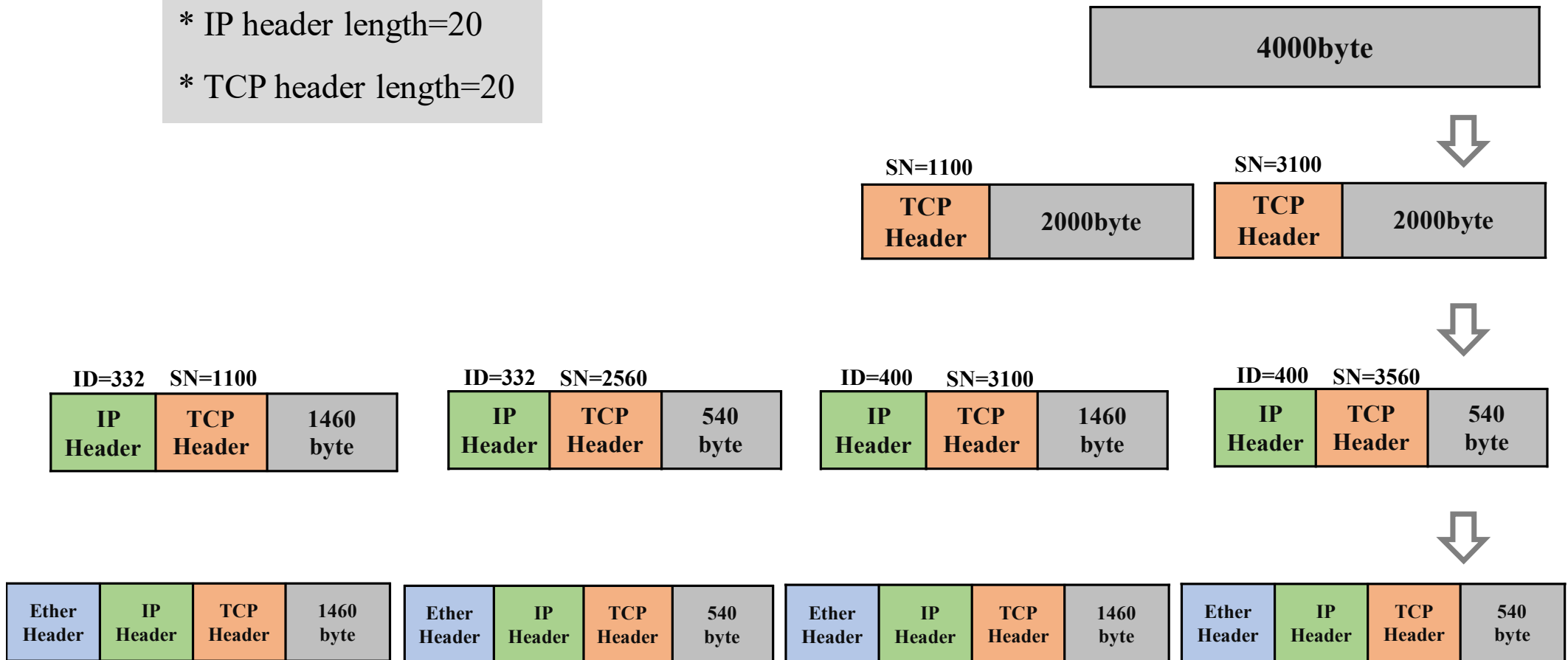
* TCP header length=20



TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 2000 byte인 경우

- * IP header length=20
- * TCP header length=20

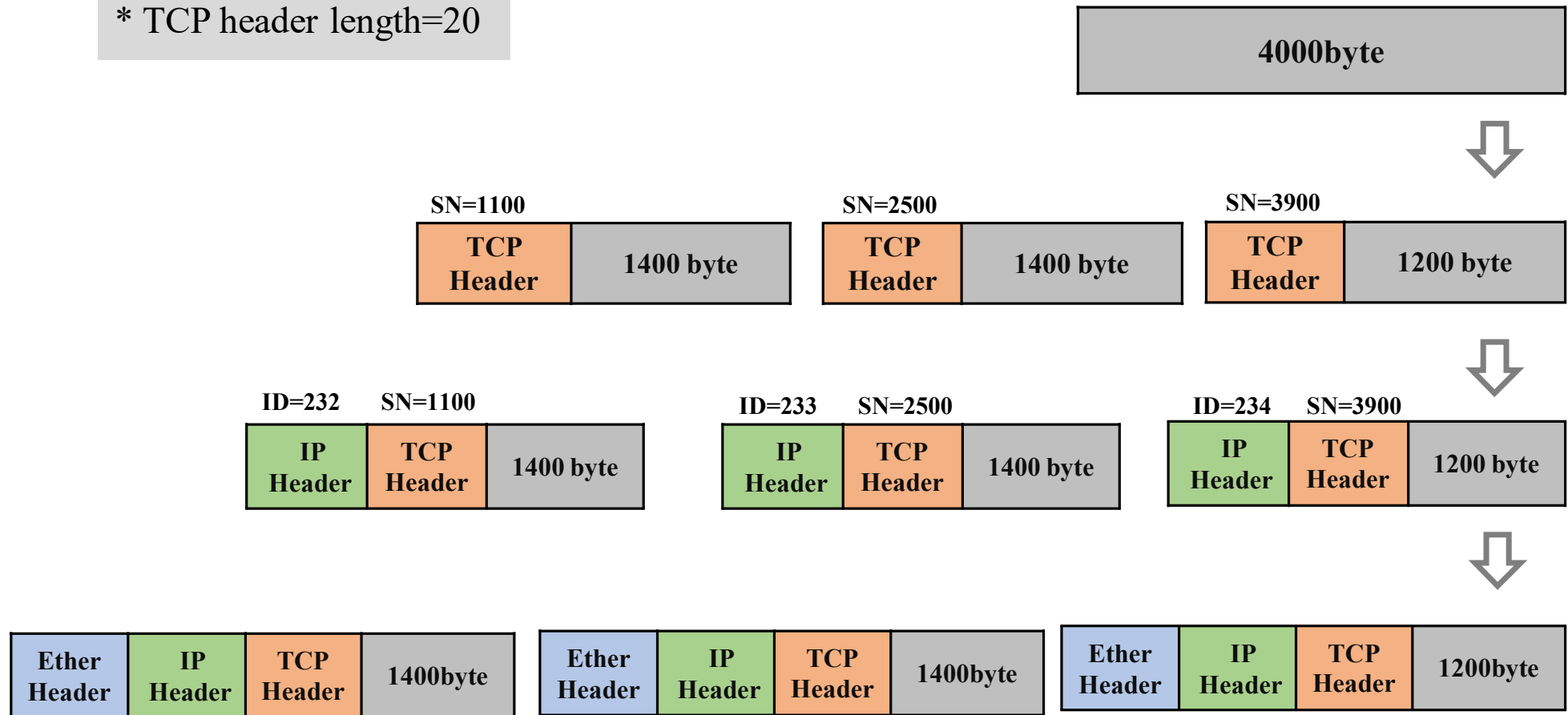


TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 1400 byte인 경우

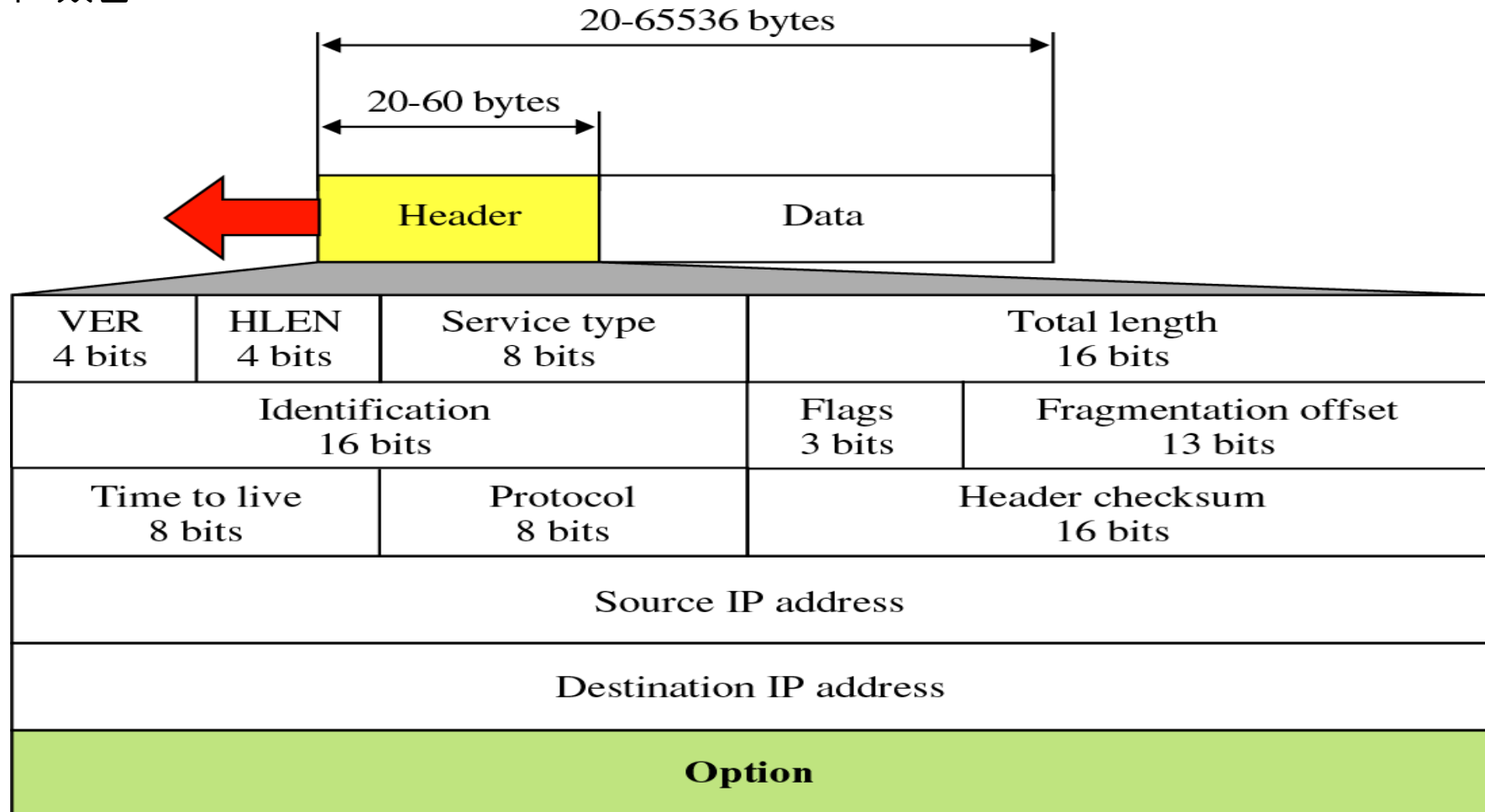
* IP header length=20

* TCP header length=20



3.3 IP(Internet Protocol) Header 구조 분석

- IP 헤더는 총 20Byte의 기본 길이와 옵션을 사용해 크기가 최대 60Byte까지 가질 수 있음



식별(Identification) 필드

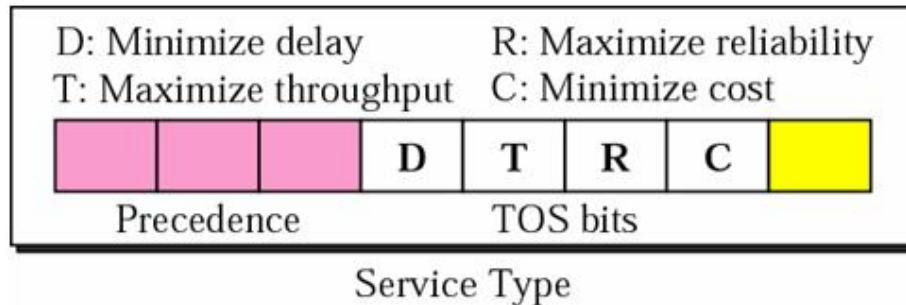
- IP 패킷의 식별 필드 값은 패킷의 유일함을 보장
- 식별 필드 값을 통해 캡처한 패킷이 재전송 패킷인지 아니면 네트워크 장애인 루프 상황에 만들어진 복제 패킷인지 판단 가능
- IP 패킷 헤더의 식별 필드 값 : 0~65,535
- 시스템에서 순차적으로 증가되어 사용
- 패킷의 수가 많으면 헤더의 식별 필드 값은 재사용될 수 있음

```
> Frame 1: 1514 bytes on wire (12112)
> Ethernet II, Src: HewlettP_bf:91:e8:00, Dst: 08:00:27:00:00:00
  ✓ Internet Protocol Version 4, Src: 10.10.10.1, Dst: 10.10.10.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes
    > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 1500
    Identification: 0x7474 (29812)
    > Flags: 0x2000, More fragments
    ...0 0000 0000 0000 = Fragment Offset
```

```
> Frame 2: 1514 bytes on wire (12112)
> Ethernet II, Src: HewlettP_bf:91:e8:00, Dst: 08:00:27:00:00:00
  ✓ Internet Protocol Version 4, Src: 10.10.10.1, Dst: 10.10.10.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes
    > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 1500
    Identification: 0x7474 (29812)
    > Flags: 0x20b9, More fragments
    ...0 0101 1100 1000 = Fragment Offset
```

Type of Service Field

- 데이터그램이 라우터에서 어떻게 처리되어야 하는지 정의



ToS bit

- 0000 : Normal (default)
- 0001 : Minimize cost
- 0010 : Maximize reliable
- 0100 : Maximize throughput
- 1000 : Minimize delay

- 총 8비트로 구성, 최초 3비트는 우선 순위, 나머지 4비트는 TOS 설정용으로 사용, 나머지 1비트는 예약
- 예를 들어 텔넷과 같은 경우는 D = 1로 설정되어 최대한 지연시간을 단축 시키도록 요구
- 현재 시스템에서는 TOS, 우선순위 항목에 대해서 지원하지 않고 기본 값 사용

MTU (Maximum Transfer Unit)

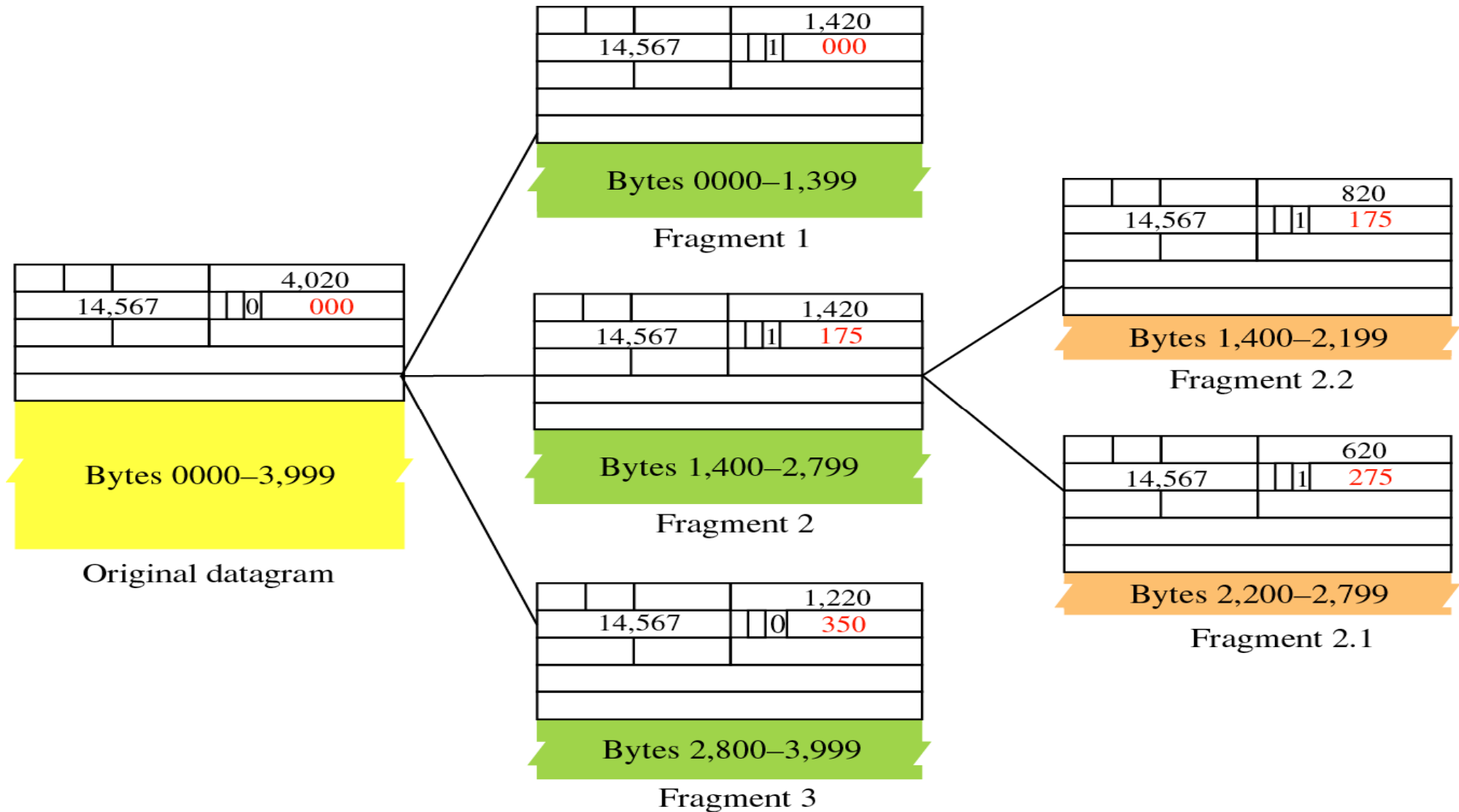
- 네트워크 기기가 전송할 수 있는 최대 전송 단위
- 네트워크 환경에 따라 각각의 크기는 다음
- 현재 대부분의 네트워크 환경이기 때문에 MTU는 1500바이트로 통용되고 있음

전송매체	MTU(bytes)
Internet IPv4 Path MTU	최소 68
Internet IPv6 Path MTU	최소 1280
Ethernet v2	1500
Ethernet LLC(Logical Link Control) SNAP (Subnetwork Access Protocol) PPPoE(P2P over Ethernet)	1492
Ethernet Jumbo Frames	1501~9216
WLAN(802.11)	7981
Token Ring(802.5)	4464
FDDI	4352

단편화(Fragmentation)

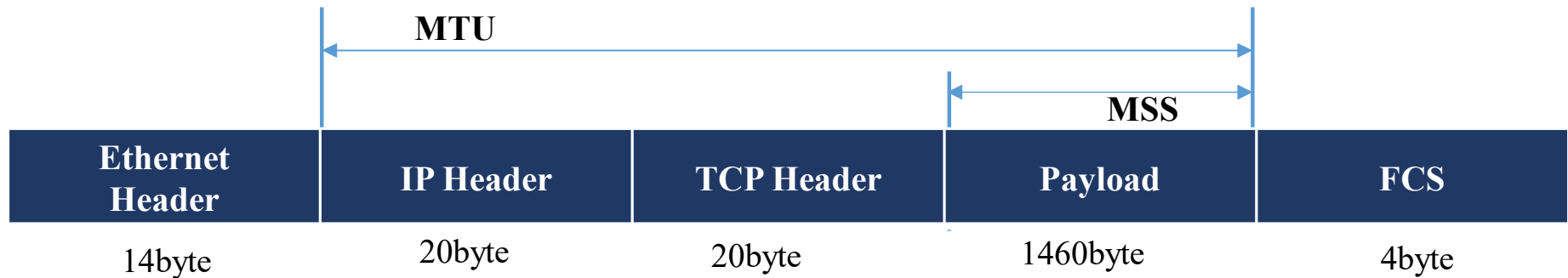
- MTU가 큰 네트워크에서 MTU가 작은 네트워크로 데이터그램이 전송될 경우 데이터그램은 나누어서 보내져야 함
- 데이터그램의 재조립은 최종목적지 호스트에 의해서만 수행
- 재조립으로 인해 발생하는 비효율성 때문에 전송 중 재조립 안됨
- 단편화와 관련된 필드 : Identification, Flag, Fragmentation offset

Fragmentation 예제



MTU (Maximum Transfer Unit)

- 해당 네트워크 프로토콜 마다 하나의 데이터그램을 송신할 때 보낼 수 있는 최대 사이즈
Ethernet - 1500, FDDI - 4352, TokenRing(16Mbps) - 17,914,
TokenRing(4Mbps) - 4464, X.25 - 576, PPP - 296



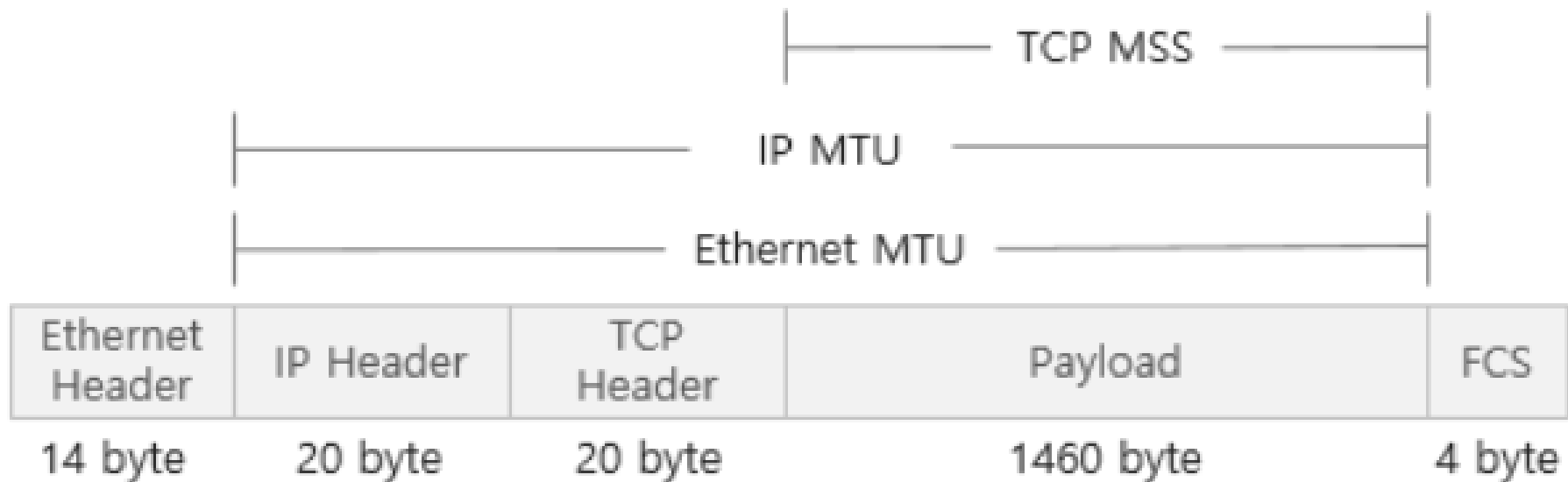
* MTU(Maximum Transfer Unit)

* MSS(Maximum Segmentation Size)

* MSS(Maximum Segment Size)

- TCP상에서 전송할 수 있는 사용자 데이터의 최대 크기

MSS = MTU - IP Header 크기 (최소 20 Byte) - TCP Header 크기 (최소 20 Byte)



[참고] 명령어 netsh

- 마이크로소프트의 유틸리티
- 로컬 또는 원격 구성의 네트워크 설정을 변경

```
C:W>netsh ?
```

```
사용법: netsh [-a 별칭 파일] [-c 컨텍스트] [-r 원격 컴퓨터]  
          [-u [DomainName\]UserName] [-p 암호 | *] [명령 | -f 스크립트 파일]
```

다음 명령을 사용할 수 있습니다.

이 컨텍스트에 있는 명령:

```
?           - 명령 목록을 표시합니다.  
add         - 항목 목록에 구성 항목을 추가합니다.  
advfirewall - 'netsh advfirewall' 컨텍스트의 변경 내용입니다.  
branchcache - 'netsh branchcache' 컨텍스트의 변경 내용입니다.  
bridge      - 'netsh bridge' 컨텍스트의 변경 내용입니다.  
delete      - 항목 목록에서 구성 항목을 삭제합니다.  
dhcpclient  - 'netsh dhcpclient' 컨텍스트의 변경 내용입니다.  
dnscclient  - 'netsh dnscclient' 컨텍스트의 변경 내용입니다.  
dump        - 구성 스크립트를 표시합니다.
```

관리자: 명령 프롬프트

```
C:W>
```

```
C:W>net int ipv4 set global ?
```

이 명령에 대한 구문:

```
NET
```

```
[ ACCOUNTS | COMPUTER | CONFIG | CONTINUE | FILE | GROUP | HELP |  
  HELPMSG | LOCALGROUP | PAUSE | SESSION | SHARE | START |  
  STATISTICS | STOP | TIME | USE | USER | VIEW ]
```

```
C:W>
```

관리자권한으로 실행

```
C:\>netsh interface show interface
```

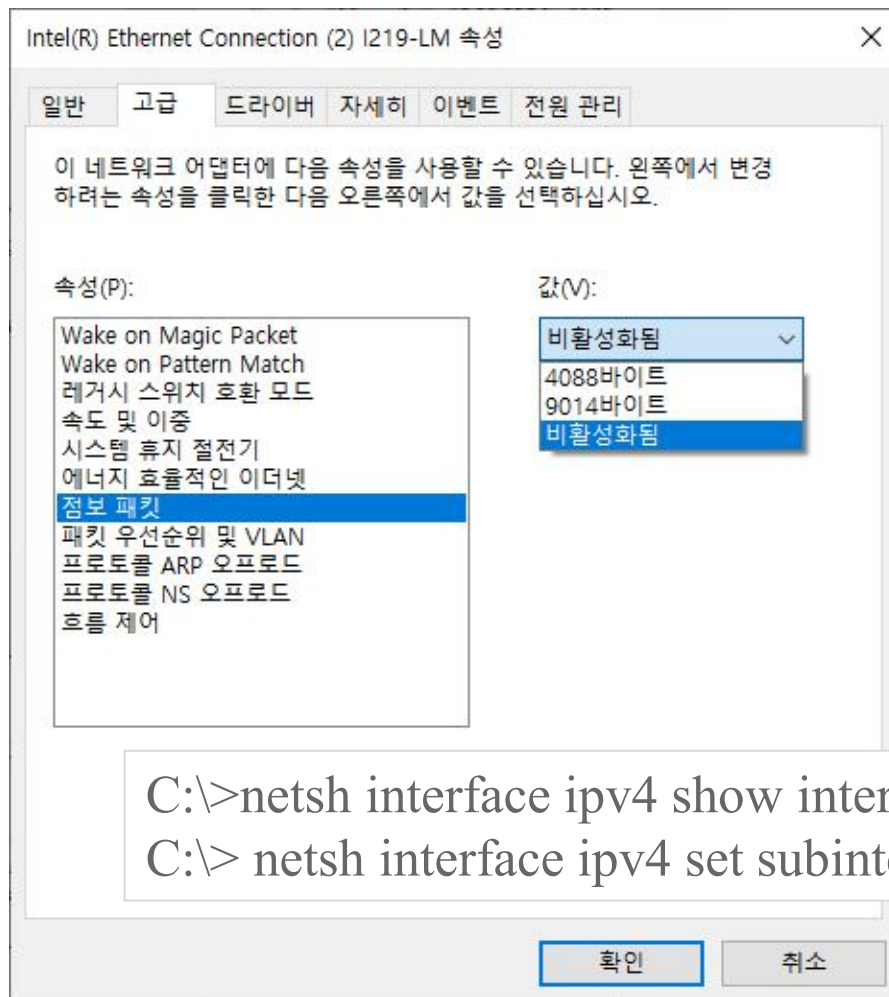
```
C:\>netsh interface ipv4 show global
```

```
C:\>netsh interface ipv4 show interfaces
```

* MTU 설정 변경

```
C:\> netsh interface ipv4 set subinterface "6" mtu=9000 store=persistent
```

```
C:\>netsh interface ipv4 show global
```



```
C:\>netsh interface ipv4 show interfaces
```

```
C:\> netsh interface ipv4 set subinterface "6" mtu=9000 store=persistent
```

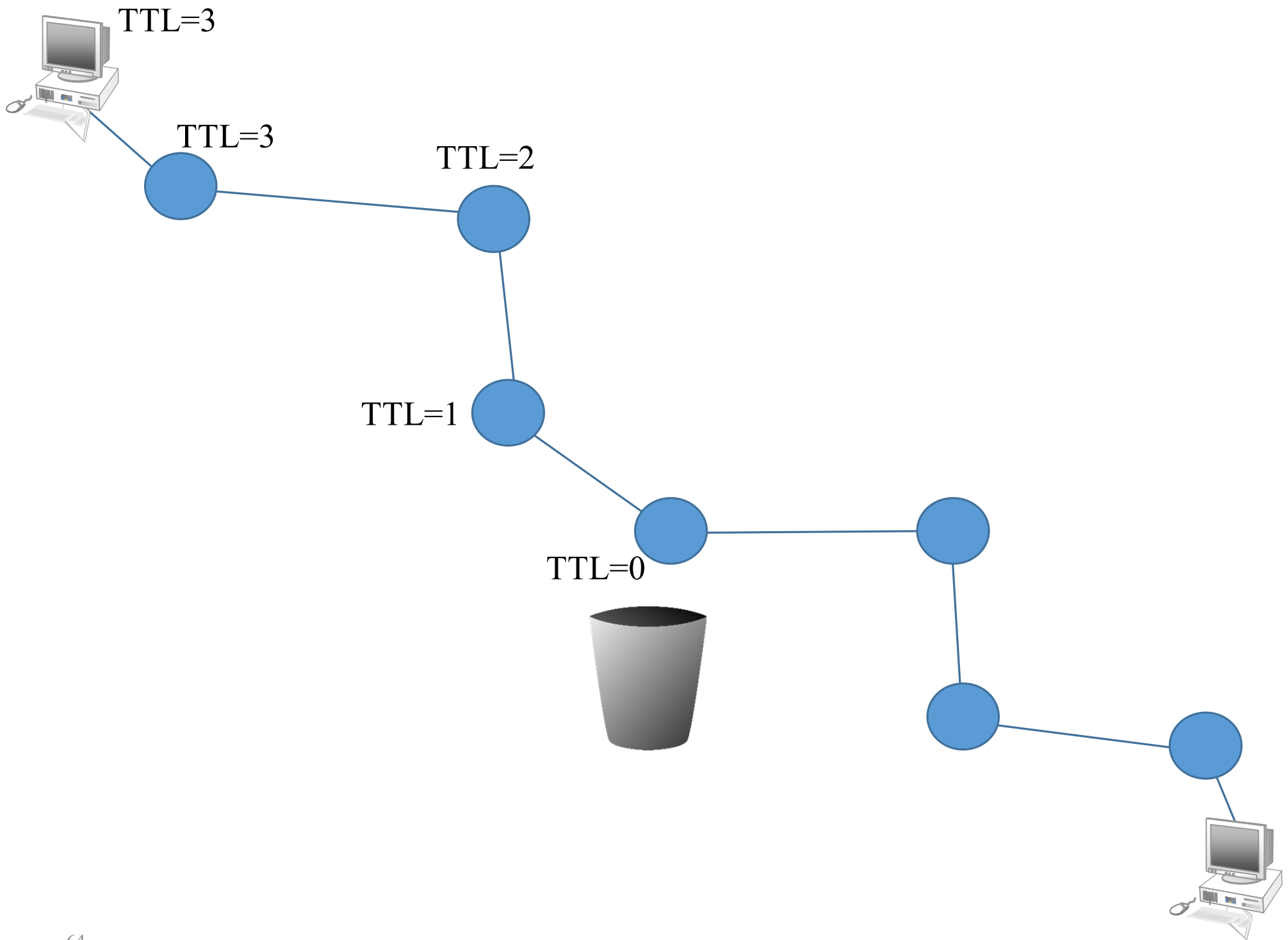
```
C:\W>netsh interface ipv4 show interfaces
```

색인	메트릭	MTU	상태	이름
1	75	4294967295	connected	Loopback Pseudo-Interface 1
6	35	1500	connected	이더넷 2
5	5	1500	disconnected	이더넷 3
9	5	1500	disconnected	이더넷 4
8	35	1500	connected	VMware Network Adapter VMnet8

TTL(Time To Live)

- 패킷 수명을 제한하기 위해 데이터그램이 통과하는 최대 홉(hop)수를 지정
- 패킷 전달과정에서 라우터와 같은 전송장비를 통과 할 때마다 TTL값 감소
- TTL이 0이 되면 라우터에서 폐기하여 불필요한 패킷이 네트워크에 방치되는 것을 방지
- OS종류와 버전에 따라 TTL 값이 다름

OS/Version	TCP TTL	UDP TTL
Linux	64	64
HP/UX 10.01	64	64
Solaris 2.z	255	255
Window Server 2008	128	128
Windows 10	64	64



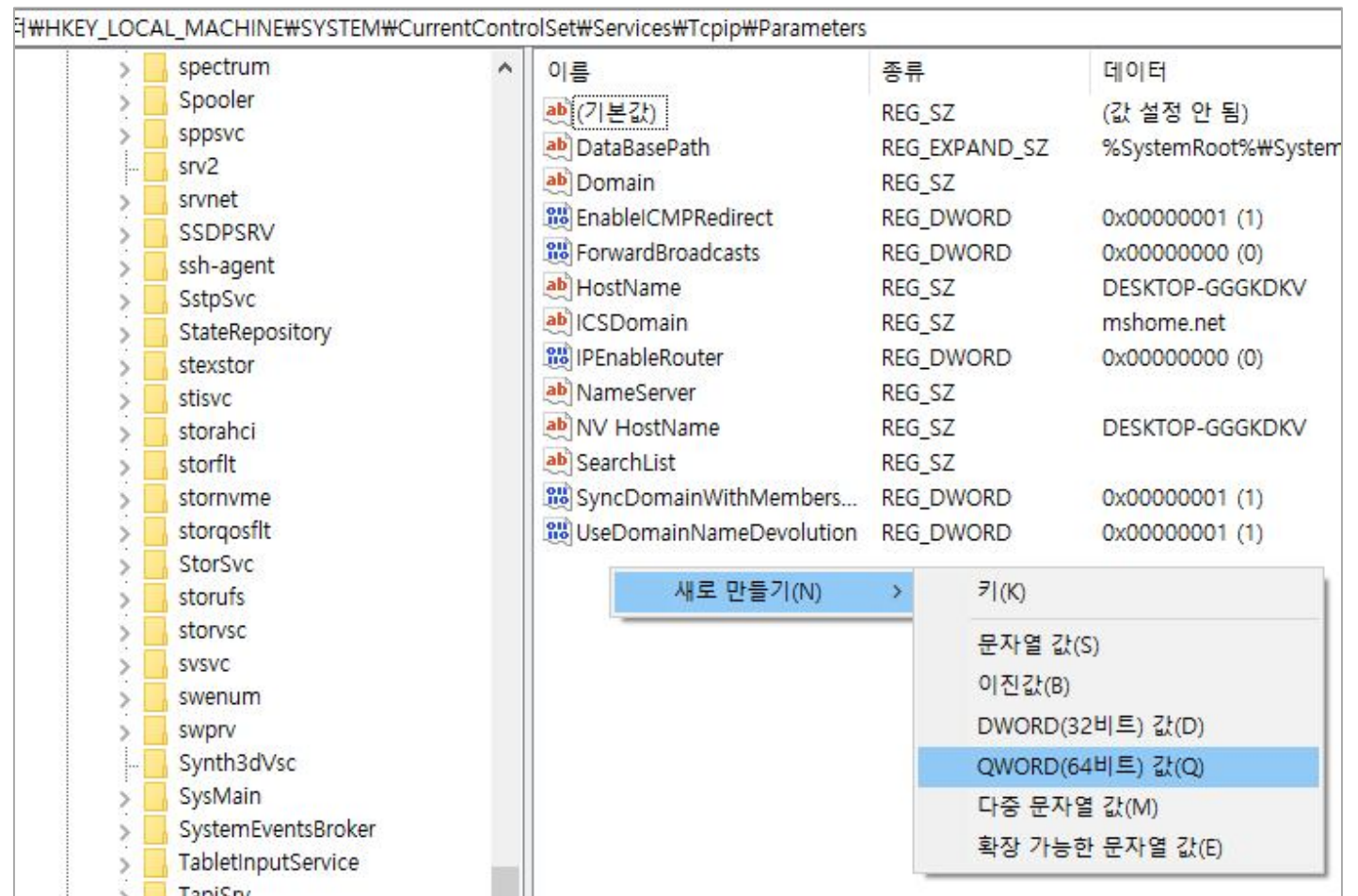
* TTL 값 변경

HKEY_LOCAL_MACHINE \SYSTEM\CurrentControlSet\Services\Tcpip\Parameters

Name: DefaultTTL

Type: REG_DWORD

Valid Range: 1-255



```
C:\>netsh interface show interface
```

```
C:\>netsh interface ipv4 show global
```

```
C:\>netsh interface ipv4 show interfaces
```

* TTL 설정 변경

```
C:\> netsh interface ipv4 set global defaultcurhoplimit=64
```

```
C:\>netsh interface ipv4 show global
```

IP Header checksum

- 헤더의 오류를 검증하기 위해 사용
- 계상방식은 version 필드 값부터 마지막 필드인 목적지 IP 필드값까지 모두 더함
 - Version 필드~ 목적지 IP 필드(Checksum 필드 제외)

```
Internet Protocol Version 4, Src: 10.40.219.42, Dst: 10.40.201.225
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 64
Identification: 0x1042 (4162)
> Flags: 0x40, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 254
Protocol: TCP (6)
Header Checksum: 0xb319 [correct]
[Header checksum status: Good]
[Calculated Checksum: 0xb319]
Source Address: 10.40.219.42
Destination Address: 10.40.201.225

0000 00 26 b9 48 ff 97 00 10 db ff 10 01 08 00 45 00
0010 00 40 10 42 40 00 fe 06 b3 19 0a 28 db 2a 0a 28
0020 c9 e1 c6 b0 00 50 d6 87 40 de 00 00 00 00 b0 02
0030 ff ff 92 d4 00 02 04 05 b4 01 03 03 05 01 01
0040 08 0a 13 ad f8 b8 00 00 00 00 04 02 00 00
```

Version/ToS : 4500
Total Length : 0040
ID : 1042
Flags : 4000
TTL/Protocol : fe06
SourceIP : 0a28 + db2a
DestinationIP : 0a28 + c9e1

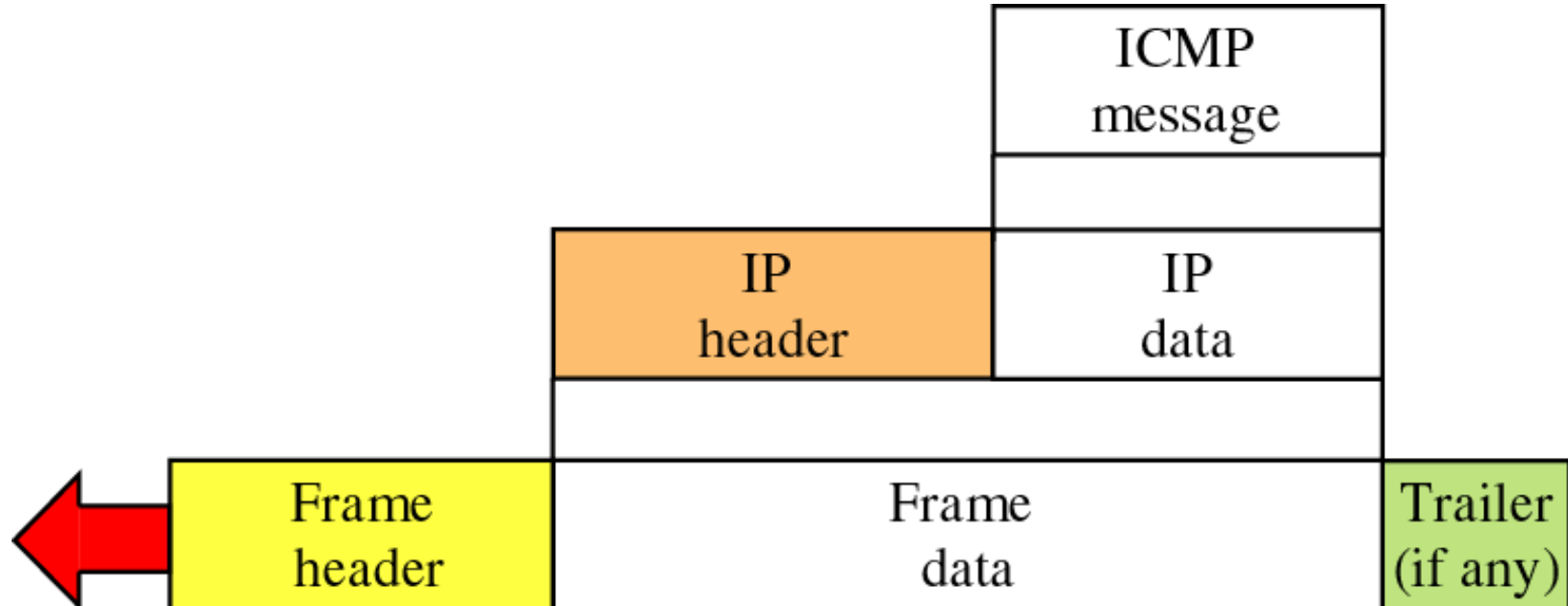
34ce3

$$3 + 4ce3 = 4ce6$$

$$\begin{aligned} 4ce6 &= 0100\ 1100\ 1110\ 0110 \text{ (2진화)} \\ &= 1011\ 0011\ 0001\ 1001 \text{ (1의보수)} \\ &= \mathbf{B319} \end{aligned}$$

3.4 ICMP Header 구조 분석

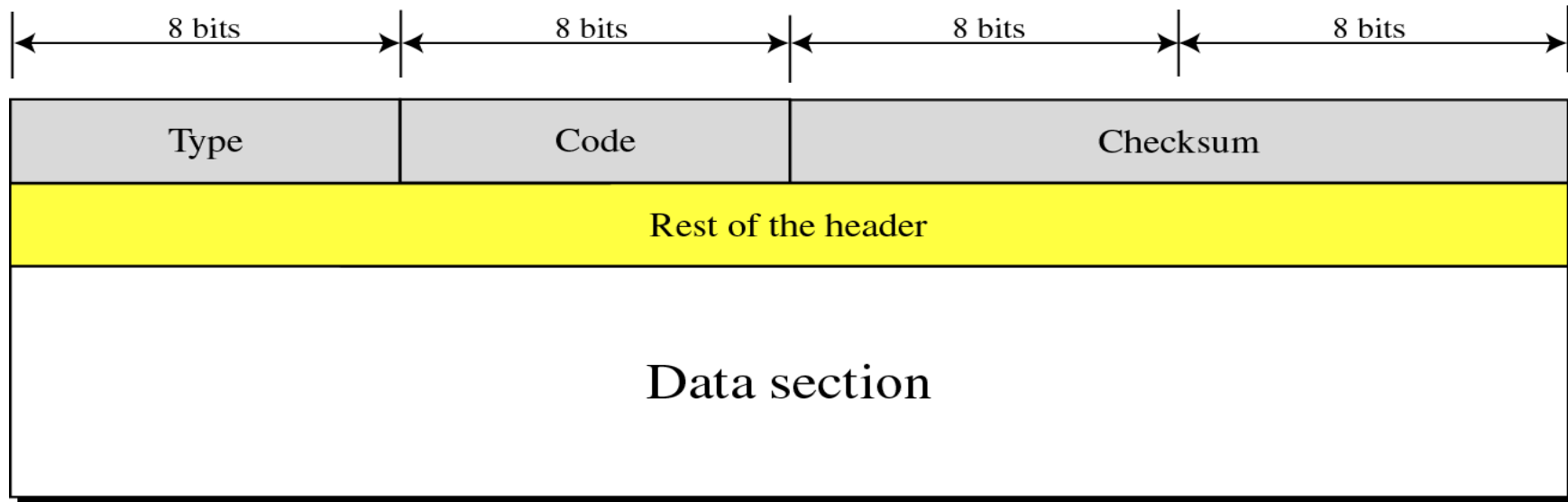
- Internet Control Message Protocol
- IP Protocol은 송신시스템과 수신시스템 사이의 패킷을 최적의 경로를 통해 전달하는 것이 주된 목적
- IP Protocol은 신뢰성이 없고 비연결형 Protocol
- IP Protocol은 에러 발생 원인이나 진단 기능 및 상황 정보를 지원하지 않음
➔ ICMP Protocol Support (IP의 단점 보완)



① ICMP 메시지 종류

- 오류 보고 메시지 (Error Reporting Message)
 - IP 패킷 처리 도중 발생한 문제 보고
- 질의 메시지 (Query Message)
 - 다른 호스트로부터 특정 정보 획득
 - 네트워크 문제 진단

2 ICMP Header



- 총 8Byte로 구성
- Type : ICMP의 업무, 즉 어떠한 용도로 사용되는 ICMP를 나타냄
- Code : ICMP Type의 세부 내용을 나타내며 이 부분은 Type과 조합을 이루어 ICMP 메시지의 목적과 용도를 나타냄
- Checksum : ICMP 메시지의 이상 유무 판단

③ ICMP Code

Type	Code	Description	Query	Error
0	0	Echo Reply (ping reply)	*	
3		Destination unreachable		
	0	Network unreachable – 지정된 Network이 없을 경우 라우터가 생성		*
	1	Host unreachable – 마지막 라우터가 호스트와 통신 불가		*
	2	Protocol unreachable – 목적지 호스트가 생성하며 IP 헤더의 프로토콜 필드로 식별된 상위 프로토콜을 사용할 수 없을 경우		*
	3	Port unreachable – 목적지 호스트가 생성		*
	4	Fragmentation needed for DE = 1 – MTU크기보다 데이터그램이 커서 전달할 수 없을 경우 라우터에서 생성		*
	5	Source Route Failed		*

3 ICMP Code

Type	Code	Description	Query	Error
4	0	Source quench		*
5		Redirect		
	0	Redirect for network		*
	1	Redirect for host		*
	2	Redirect for type of service and network		*
	3	Redirect for type of service and host		*
8	0	Echo request (ping 요청)	*	
9	0	Router advertisement	*	
10	0	Router solicitation	*	

* 에러 보고 메시지

타입	메시지 이름	설명
3	Destination Unreachable (목적지 도착 불가 메시지)	라우터가 패킷을 라우팅 할 수 없거나 호스트가 패킷을 전달할 수 없을 때 해당 패킷은 전달이 불가능하다고 판단한 라우터나 호스트가 폐기하고 출발지 호스트에 '목적지도착불가'메시지를 전달
4	Source Quench (발신지 억제)	- 송신자에게 링크 혼잡으로써 패킷이 폐기되었음을 알려줌 혼잡상황을 알려줄뿐 혼잡원인을 제공하지 않음 - 발신지 억제 메시지를 수신한 시스템 자체에 혼잡을 유발했다고 단정할 수는 없음
5	Redirection (재지정)	특정 목적지에 대해 더 나은 경로를 갖고 있는 다른 라우터가 있음을 알려줌
11	Time exceeded (시간초과)	TTL 만료로 패킷이 폐기 되었음을 알림

ICMP와 네트워크 관리 명령어

```
C:\>ping -?
```

```
사용법: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
          [-r count] [-s count] [[-j host-list] | [-k host-list]]
          [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
          [-4] [-6] target_name
```

옵션:

-t	중지될 때까지 지정한 호스트를 ping합니다. 통계를 보고 계속하려면 <Ctrl+Break>를 입력합니다. 중지하려면 <Ctrl+C>를 입력합니다.
-a	주소를 호스트 이름으로 확인합니다.
-n count	보낼 에코 요청의 수입니다.
-l size	송신 버퍼 크기입니다.
-f	패킷에 조각화 안 함 플래그를 설정(IPv4에만 해당)합니다.
-i TTL	Time To Live
-v TOS	서비스 종류(IPv4에만 해당. 이 설정은 더 이상 사용되지 않으며 IP 헤더의 서비스 종류 필드에 영향을 주지 않음)입니다.
-r count	count 홉의 경로를 기록합니다(IPv4에만 해당).
-s count	count 홉의 타임스탬프(IPv4에만 해당)입니다.
-j host-list	host-list에 따라 원본 라우팅을 완화합니다(IPv4에만 해당).
-k host-list	host-list에 따라 원본 라우팅을 강화합니다(IPv4에만 해당).
-w timeout	각 응답의 대기 시간 제한(밀리초)입니다.

```
C:\W>ping 203.241.132.31
```

```
Ping 203.241.132.31 32바이트 데이터 사용:
```

```
요청 시간이 만료되었습니다.
```

```
요청 시간이 만료되었습니다.
```

```
요청 시간이 만료되었습니다.
```

```
요청 시간이 만료되었습니다.
```

```
203.241.132.31에 대한 Ping 통계:
```

```
패킷: 보냄 = 4, 받음 = 0, 손실 = 4 (100% 손실),
```

```
C:\W>
```

```
C:\W>ping 192.168.10.5
```

```
Ping 192.168.10.5 32바이트 데이터 사용:
```

```
192.168.10.1의 응답: 대상 호스트에 연결할 수 없습니다.
```

```
192.168.10.1의 응답: 대상 호스트에 연결할 수 없습니다.
```

```
192.168.10.1의 응답: 대상 호스트에 연결할 수 없습니다.
```

```
192.168.10.1의 응답: 대상 호스트에 연결할 수 없습니다.
```

```
192.168.10.5에 대한 Ping 통계:
```

```
패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
```

```
C:\W>
```

```
C:\W>ping -l 5000 203.241.132.34
```

Ping 203.241.132.34 5000바이트 데이터 사용:
203.241.132.34의 응답: 바이트=5000 시간=11ms TTL=55
203.241.132.34의 응답: 바이트=5000 시간=10ms TTL=55
203.241.132.34의 응답: 바이트=5000 시간=9ms TTL=55
203.241.132.34의 응답: 바이트=5000 시간=8ms TTL=55

203.241.132.34에 대한 Ping 통계:

패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),

왕복 시간(밀리초):

최소 = 8ms, 최대 = 11ms, 평균 = 9ms

```
C:\W>
```

Internet Control Message Protocol

Type: 8 (Echo (ping) request)

Code: 0

Checksum: 0x414e [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence number (BE): 1 (0x0001)

Sequence number (LE): 256 (0x0100)

[Response frame: 1447]

Data (5000 bytes)

Data: 6162636465666768696a6b6c6d6e6f707172737475767761...

[Length: 5000]

```
0000 08 00 41 4e 00 01 00 01 61 62 63 64 65 66 67 68 ..AN....abcdefgh
0010 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 ijklmnopqrstuvw
0020 62
0030 72
0040 6b
0050 64
0060 74
0070 6d
```

(ip.src == 70.12.111.90) && (ip.dst == 203.241.132.34)

Source	Destination	Protocol	Length	Info
70.12.111.90	203.241.132.34	ICMP	602	Echo (ping) request id=
70.12.111.90	203.241.132.34	IPv4	1514	Fragmented IP protocol (
70.12.111.90	203.241.132.34	IPv4	1514	Fragmented IP protocol (
70.12.111.90	203.241.132.34	IPv4	1514	Fragmented IP protocol (

-l size : ICMP 패킷 크기 지정

```
C:\>ping -f 203.241.132.34
```

```
Ping 203.241.132.34 32바이트 데이터 사용:  
203.241.132.34의 응답: 바이트=32 시간=3ms TTL=55  
203.241.132.34의 응답: 바이트=32 시간=4ms TTL=55  
203.241.132.34의 응답: 바이트=32 시간=4ms TTL=55  
203.241.132.34의 응답: 바이트=32 시간=4ms TTL=55
```

```
203.241.132.34에 대한 Ping 통계:  
패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),  
왕복 시간(밀리초):  
최소 = 3ms, 최대 = 4ms, 평균 = 3ms
```

```
C:\>
```

-f : Packet에 Do not fragment flag를 설정

```
Internet Protocol Version 4, Src: 70.12.111.90, Dst: 203.241.132.34  
  0100 .... = Version: 4  
  .... 0101 = Header Length: 20 bytes (5)  
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)  
    Total Length: 60  
    Identification: 0x1a56 (6742)  
  ▸ Flags: 0x4000, Don't fragment  
    0... .... = Reserved bit: Not set  
    .1.. .... = Don't fragment: Set  
    ..0. .... = More fragments: Not set  
    ...0 0000 0000 0000 = Fragment offset: 0  
    Time to live: 128  
    Protocol: ICMP (1)  
    Header checksum: 0x0000 [validation disabled]  
    [Header checksum status: Unverified]  
    Source: 70.12.111.90  
    Destination: 203.241.132.34  
  ▸ Internet Control Message Protocol  
    Type: 8 (Echo (ping) request)
```

```
C:\>ping -f -l 5000 203.241.132.34
```

```
Ping 203.241.132.34 5000바이트 데이터 사용:  
패킷 조각화가 필요하지만 DF가 설정되어 있습니다.  
패킷 조각화가 필요하지만 DF가 설정되어 있습니다.  
패킷 조각화가 필요하지만 DF가 설정되어 있습니다.  
패킷 조각화가 필요하지만 DF가 설정되어 있습니다.
```

```
203.241.132.34에 대한 Ping 통계:  
패킷: 보냄 = 4, 받음 = 0, 손실 = 4 (100% 손실),
```

```
C:\>
```

-w timeout : 각 응답시간에 대한 대기 시간을 지정

```
C:\>ping -w 5000 203.241.132.34
```

```
Ping 203.241.132.34 32바이트 데이터 사용:
```

```
203.241.132.34의 응답: 바이트=32 시간=4ms TTL=55
```

```
203.241.132.34의 응답: 바이트=32 시간=5ms TTL=55
```

```
203.241.132.34의 응답: 바이트=32 시간=4ms TTL=55
```

```
203.241.132.34의 응답: 바이트=32 시간=4ms TTL=55
```

```
203.241.132.34에 대한 Ping 통계:
```

```
패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
```

```
왕복 시간(밀리초):
```

```
최소 = 4ms, 최대 = 5ms, 평균 = 4ms
```

```
C:\>
```

```
C:\W>ping -i 3 203.241.132.34
```

```
Ping 203.241.132.34 32바이트 데이터 사용:  
100.71.51.33의 응답: 전송하는 동안 TTL이 만료되었습니다.  
100.71.51.33의 응답: 전송하는 동안 TTL이 만료되었습니다.  
100.71.51.33의 응답: 전송하는 동안 TTL이 만료되었습니다.  
100.71.51.33의 응답: 전송하는 동안 TTL이 만료되었습니다.
```

```
203.241.132.34에 대한 Ping 통계:  
패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
```

```
C:\W>
```

-i TTL: TTL값을 지정된 값으로 설정

```
Internet Protocol Version 4, Src: 70.12.111.90, Dst: 203.241.132.34  
  0100 .... = Version: 4  
  .... 0101 = Header Length: 20 bytes (5)  
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)  
    Total Length: 60  
    Identification: 0x1ac2 (6850)  
  ▸ Flags: 0x0000  
    0... .. = Reserved bit: Not set  
    .0... .. = Don't fragment: Not set  
    ..0. .... = More fragments: Not set  
    ...0 0000 0000 0000 = Fragment offset: 0  
  ▸ Time to live: 3  
    ▸ [Expert Info (Note/Sequence): "Time To Live" only 3]  
    Protocol: ICMP (1)  
    Header checksum: 0x0000 [validation disabled]  
    [Header checksum status: Unverified]  
    Source: 70.12.111.90  
    Destination: 203.241.132.34  
  ▸ Internet Control Message Protocol  
    Type: 8 (Echo (ping) request)  
    Code: 0
```



```
C:\>tracert 203.241.132.34
```

최대 30홉 이상의

nic.samsung.co.kr [203.241.132.34](으)로 가는 경로 추적:

1	2 ms	1 ms	1 ms	192.168.35.1
2	5 ms	8 ms	4 ms	118.221.108.1
3	2 ms	3 ms	2 ms	100.71.51.33
4	2 ms	2 ms	2 ms	10.44.255.236
5	4 ms	8 ms	5 ms	10.222.16.242
6	4 ms	4 ms	4 ms	58.229.11.58
7	4 ms	7 ms	3 ms	1.255.24.110
8	4 ms	7 ms	4 ms	u18.ppp80.samsung.co.kr [157.197.80.18]
9	6 ms	4 ms	4 ms	112.106.53.2
10	4 ms	4 ms	4 ms	nic.samsung.co.kr [203.241.132.34]

추적을 완료했습니다.

```
C:\>
```

```
C:\W>tracert -d 203.241.132.34
```

최대 30홉 이상의 203.241.132.34(으)로 가는 경로 추적

1	1 ms	2 ms	1 ms	192.168.35.1
2	4 ms	10 ms	4 ms	118.221.108.1
3	2 ms	2 ms	2 ms	100.71.51.33
4	2 ms	2 ms	2 ms	10.44.255.236
5	3 ms	4 ms	4 ms	10.222.16.242
6	7 ms	4 ms	4 ms	58.229.11.58
7	5 ms	4 ms	4 ms	1.255.24.110
8	4 ms	3 ms	4 ms	157.197.80.18
9	4 ms	5 ms	4 ms	112.106.53.2
10	4 ms	4 ms	4 ms	203.241.132.34

추적을 완료했습니다.

```
C:\W>
```

-d : 호스트명을 명시하지 않음

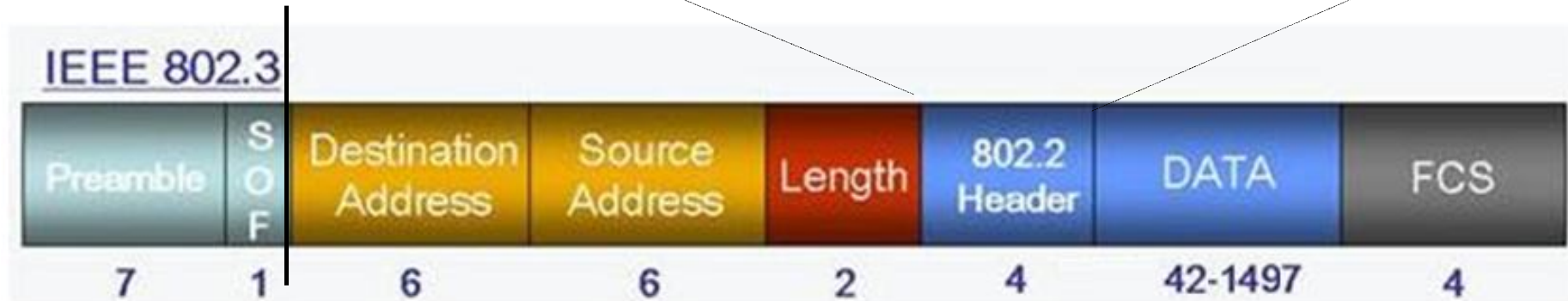
3.5 Ethernet Frame 구조 분석

① Ethernet Frame (DIX 2.0)



2 IEEE 802.3 & IEEE 802.2

DSAP address	SSAP address	Control	Information
8 bits	8 bits	8 or 16 bits	M*8 bits



3.6 ARP Header 구조 분석

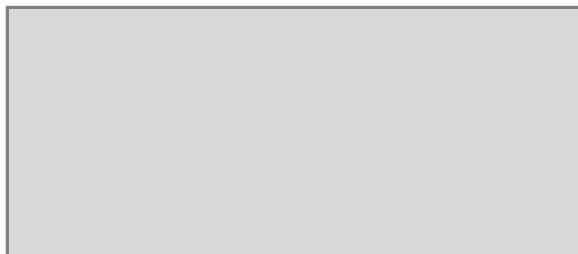
Hardware Type		Protocol Type
Hardware length	Protocol length	Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)		
Target protocol address (For example, 4 bytes for IP)		

Type: 0x0806

Preamble and SFD	Destination address	Source address	Type	Data	CRC
8 bytes	6 bytes	6 bytes	2 bytes		4 bytes

ARP Request Packet

ARP Cache Table



130.23.43.20

xB23455102210

System A

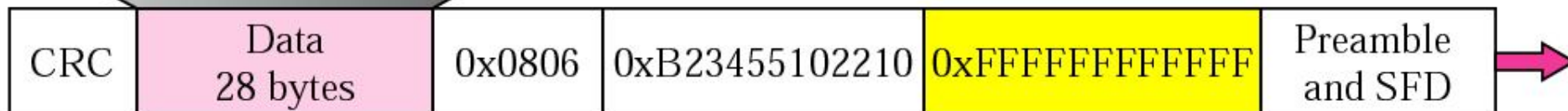
130.23.43.25



x46EF45983AB

System B

0x0001		0x0800
0x06	0x04	0x0001
0xB23455102210 130.23.43.20 0x000000000000 130.23.43.25		

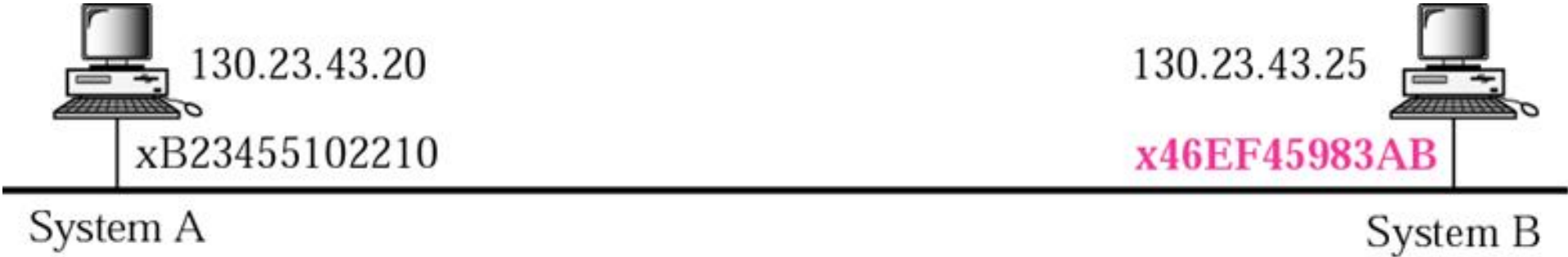


ARP Request

ARP Reply Packet

ARP Cache Table

130.23.43.20	B234-5510-2210
---------------------	-----------------------



0x0002		0x0800
0x06	0x04	0x0002
0x46EF45983AB 130.23.43.25 0xB23455102210 130.23.43.20		



ARP Reply (from B to A)

ARP 수행 후 캐시 테이블

ARP Cache Table

130.23.43.25 46EF-4598-3AB



130.23.43.20

xB23455102210

System A

ARP Cache Table

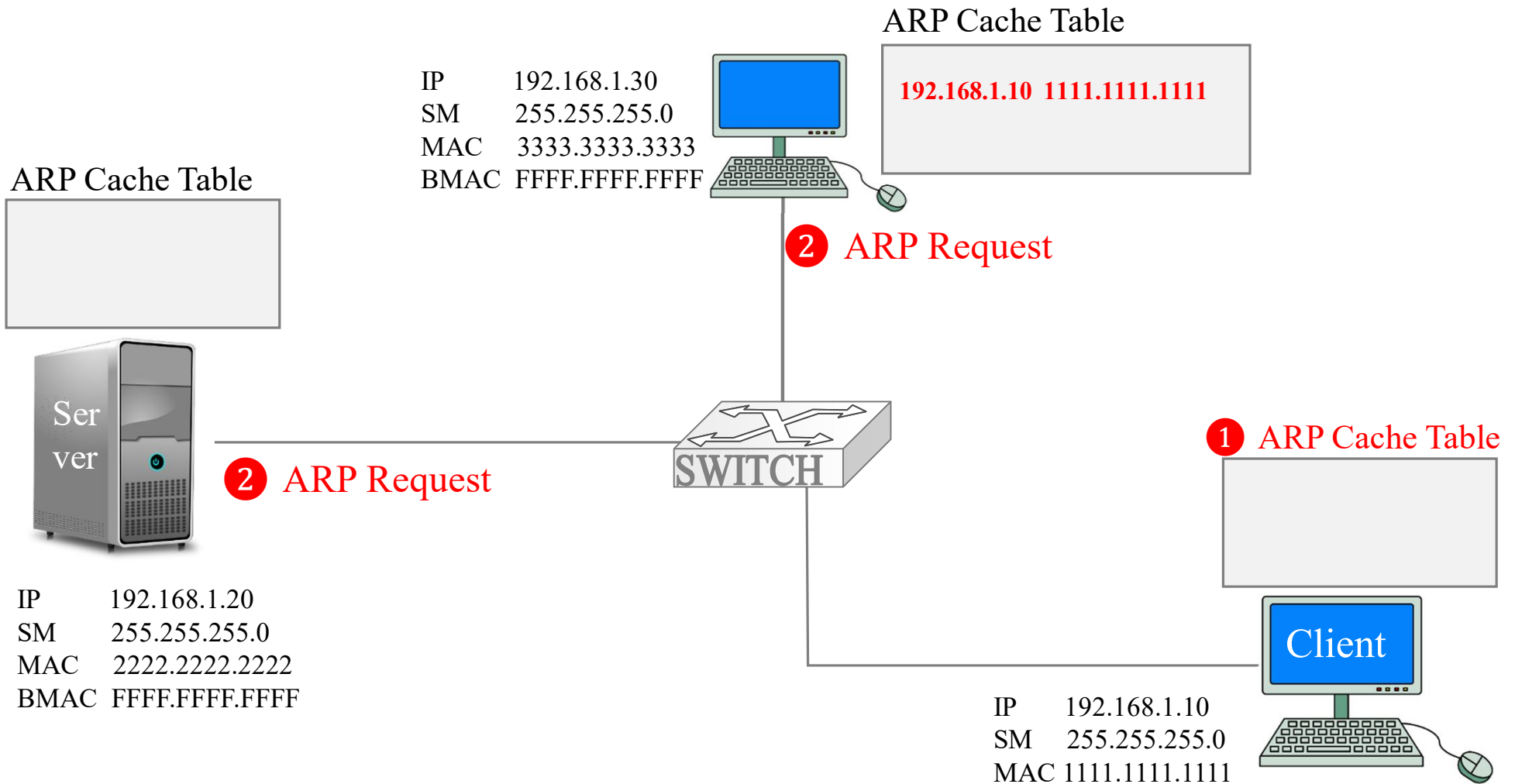
130.23.43.20 B234-5510-2210

130.23.43.25

x46EF45983AB

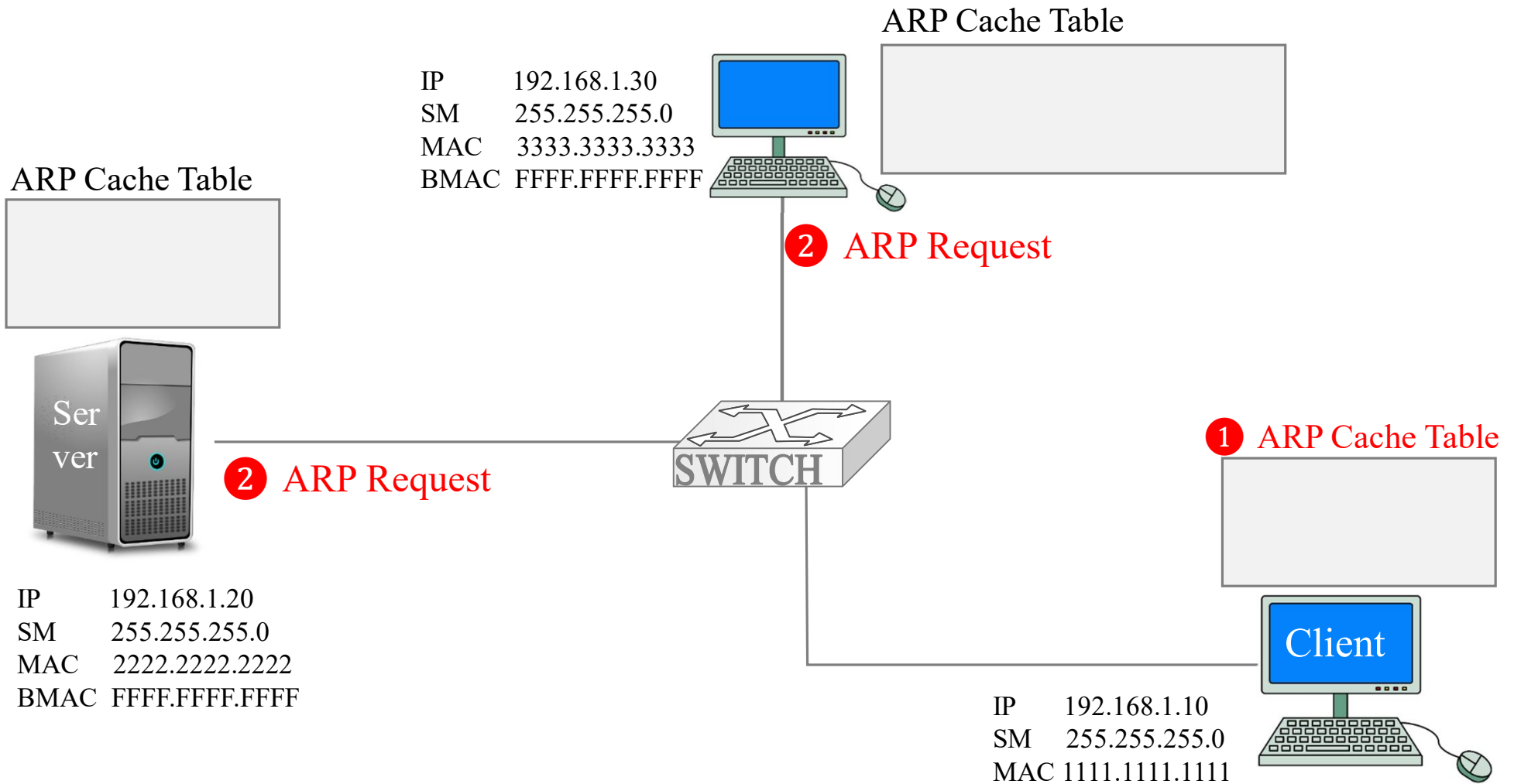


System B



ARP Request	수MAC	송MAC	Protocol	ARP Header	
←	FFFF.FFFF.FFFF	1111.1111.1111	0x0806	1111.1111.1111 / 192.168.1.10 0000.0000.0000 / 192.168.1.20	송신지 수신지

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이터
????	1111.1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com



ARP Request	수MAC	송MAC	Protocol	ARP Header	
←	FFFF.FFFF.FFFF	1111.1111.1111	0x0806	1111.1111.1111 / 192.168.1.10 0000.0000.0000 / 192.168.1.20	송신지 수신지

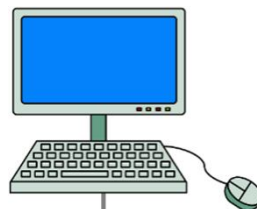
수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이터
????	1111.1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com

ARP Cache Table



IP 192.168.1.20
SM 255.255.255.0
MAC 2222.2222.2222
BMAC FFFF.FFFF.FFFF

② ARP Request

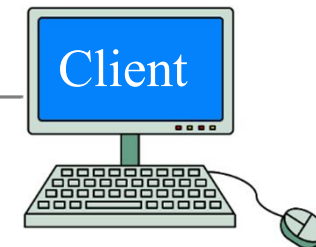


IP 192.168.1.30
SM 255.255.255.0
MAC 3333.3333.3333
BMAC FFFF.FFFF.FFFF

② ARP Request



① ARP Cache Table



IP 192.168.1.10
SM 255.255.255.0
MAC 1111.1111.1111

ARP Request



수MAC	송MAC	Protocol	ARP Header
FFFF.FFFF.FFFF	1111.1111.1111	0x0806	1111.1111.1111 / 192.168.1.10 0000.0000.0000 / 192.168.1.20

송신지
수신지

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이터
????	1111.1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com

	ARP Header	Protocol	송MAC	수MAC	
송신지 수신지	2222.2222.2222 / 192.168.1.20 1111.1111.1111 / 192.168.1.10	0x0806	2222.2222.2222	1111.1111.1111	➡ ARP Reply

ARP Cache Table

① 192.168.1.10 1111.1111.1111



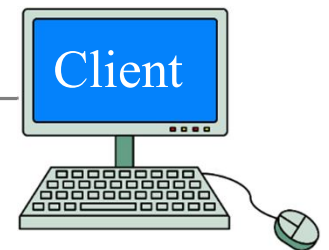
IP 192.168.1.20
SM 255.255.255.0
MAC 2222.2222.2222
BMAC FFFF.FFFF.FFFF



ARP Cache Table

--

② ARP Reply



IP 192.168.1.10
SM 255.255.255.0
MAC 1111.1111.1111

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이터
222.2222.2222	1111.1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com

	ARP Header	Protocol	송MAC	수MAC	
송신지 수신지	2222.2222.2222 / 192.168.1.20 1111.1111.1111 / 192.168.1.10	0x0806	2222.2222.2222	1111.1111.1111	➡ ARP Reply

ARP Cache Table

① 192.168.1.10 1111.1111.1111



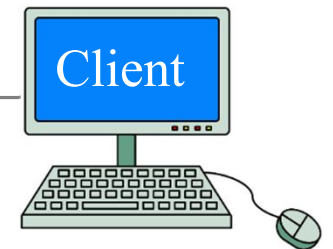
IP 192.168.1.20
SM 255.255.255.0
MAC 2222.2222.2222
BMAC FFFF.FFFF.FFFF



ARP Cache Table

③ 192.168.1.20 2222.2222.2222

② ARP Reply



IP 192.168.1.10
SM 255.255.255.0
MAC 1111.1111.1111

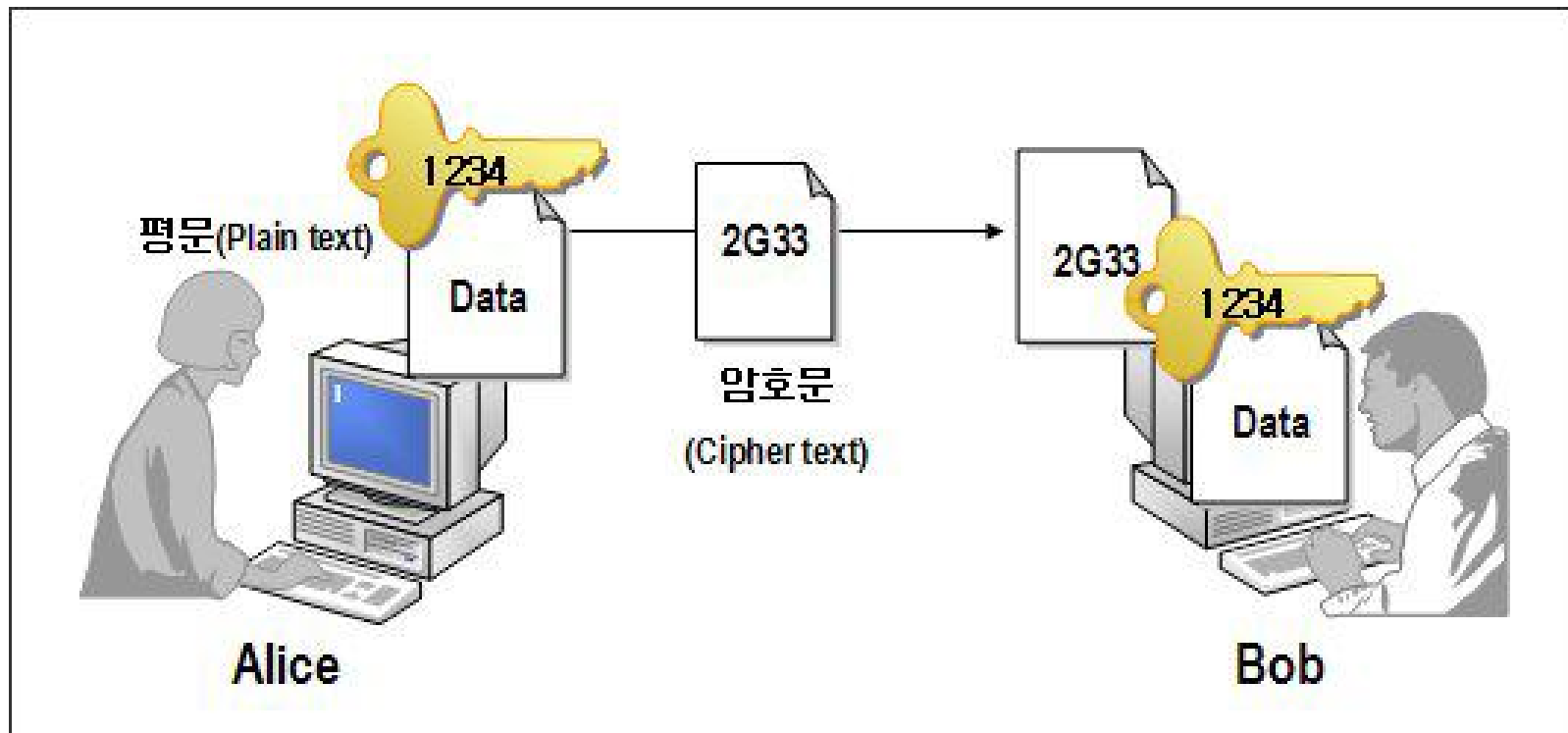
수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이터
222.2222.2222	1111.1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com

3.7 SSL/X.509 Header 구조 분석

1) 키 기반의 암호화 기술

① Symmetric Key(대칭키)

- Encryption Key = Decryption Key

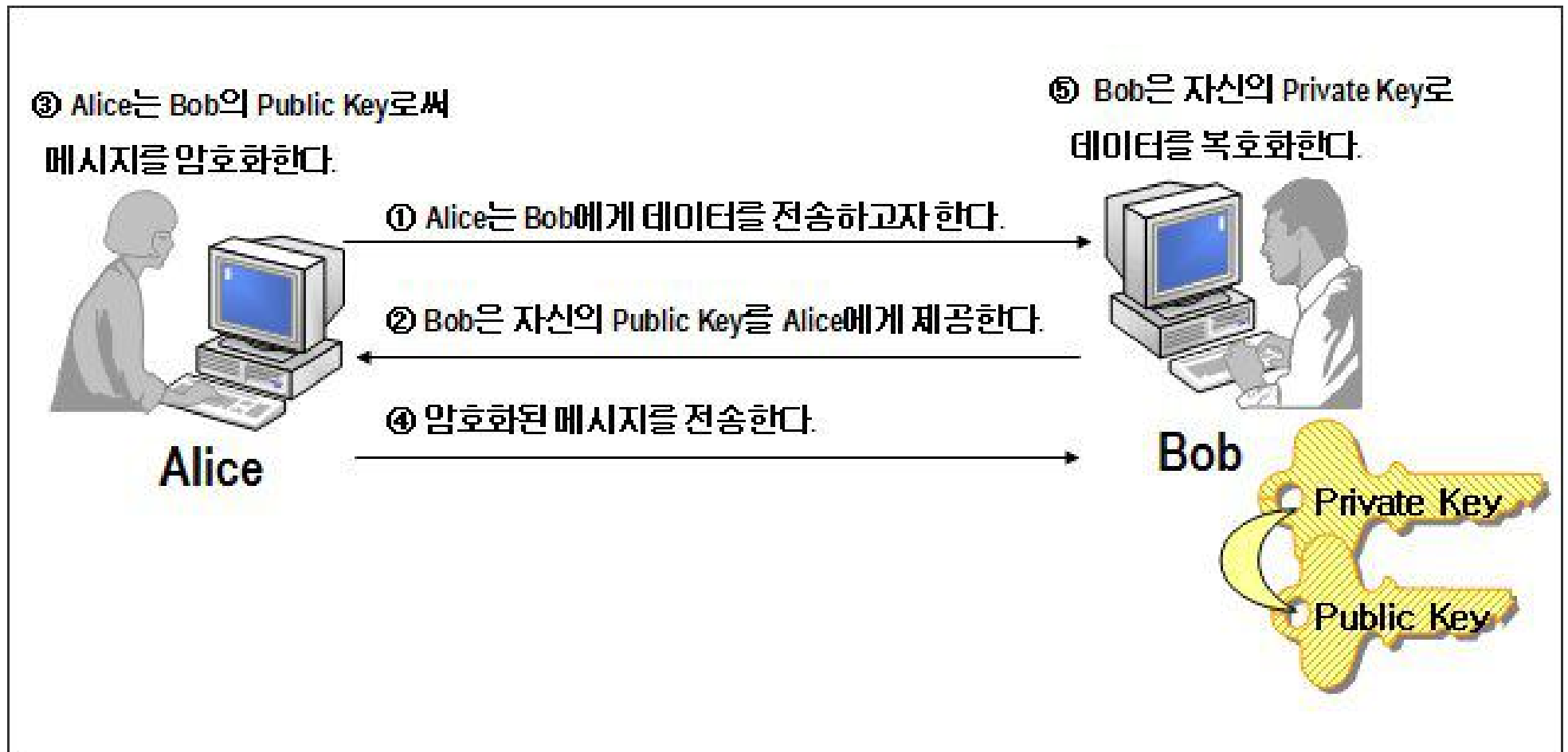


- DES, 3DES, AES, SEED, ARIA, RC5 알고리즘 등이 해당됨

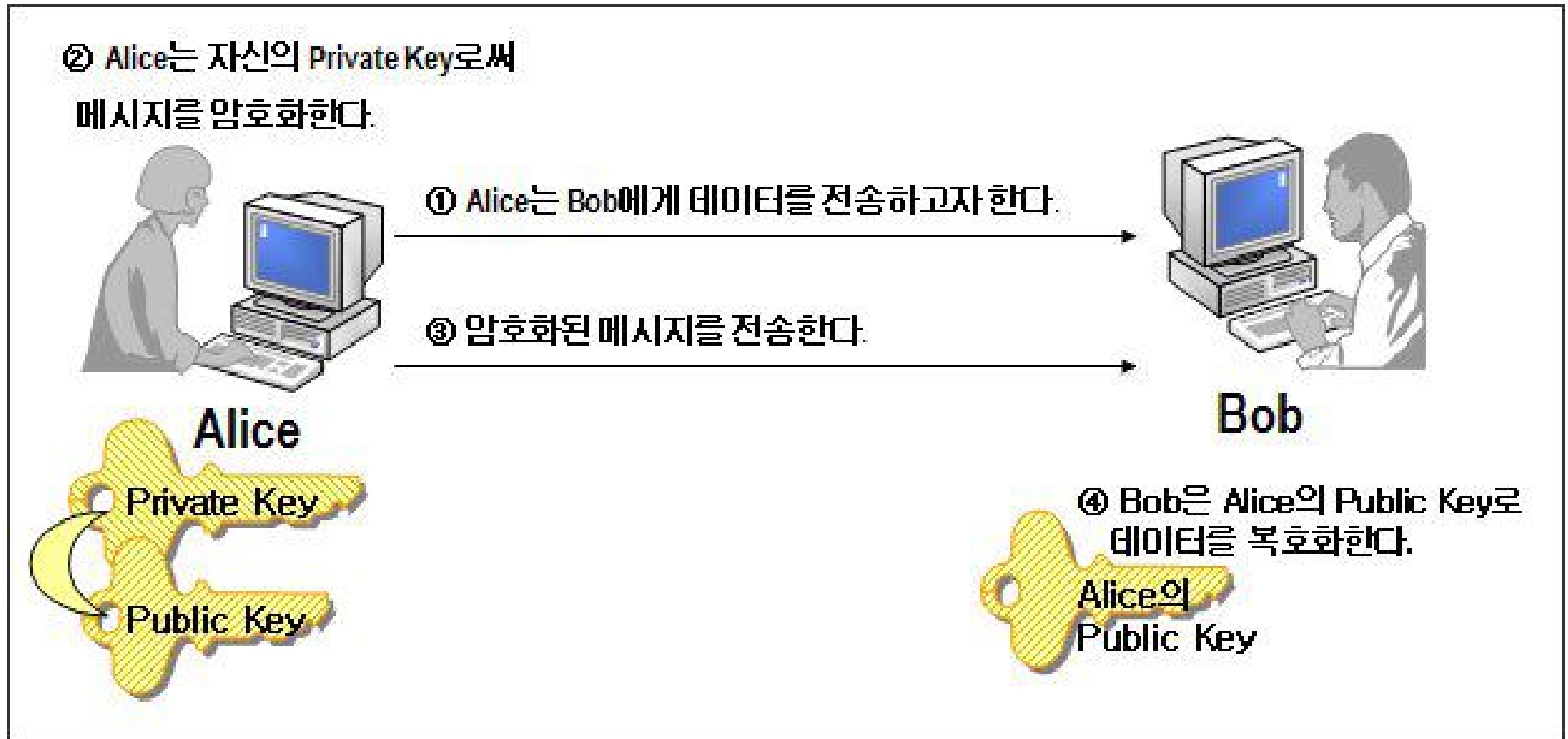
Public Key(공용키)

- Encryption Key $\neq$ Decryption Key
 - 비대칭키, 공용키, 페어(Pair)키에 해당함.
- Public Key와 Private Key로 구성
 - Public Key(공용키) = 공개가 되는 키
 - Private Key(개인키) = 오직 발행주체만이 가지는 비밀키
- RSA 프로토콜

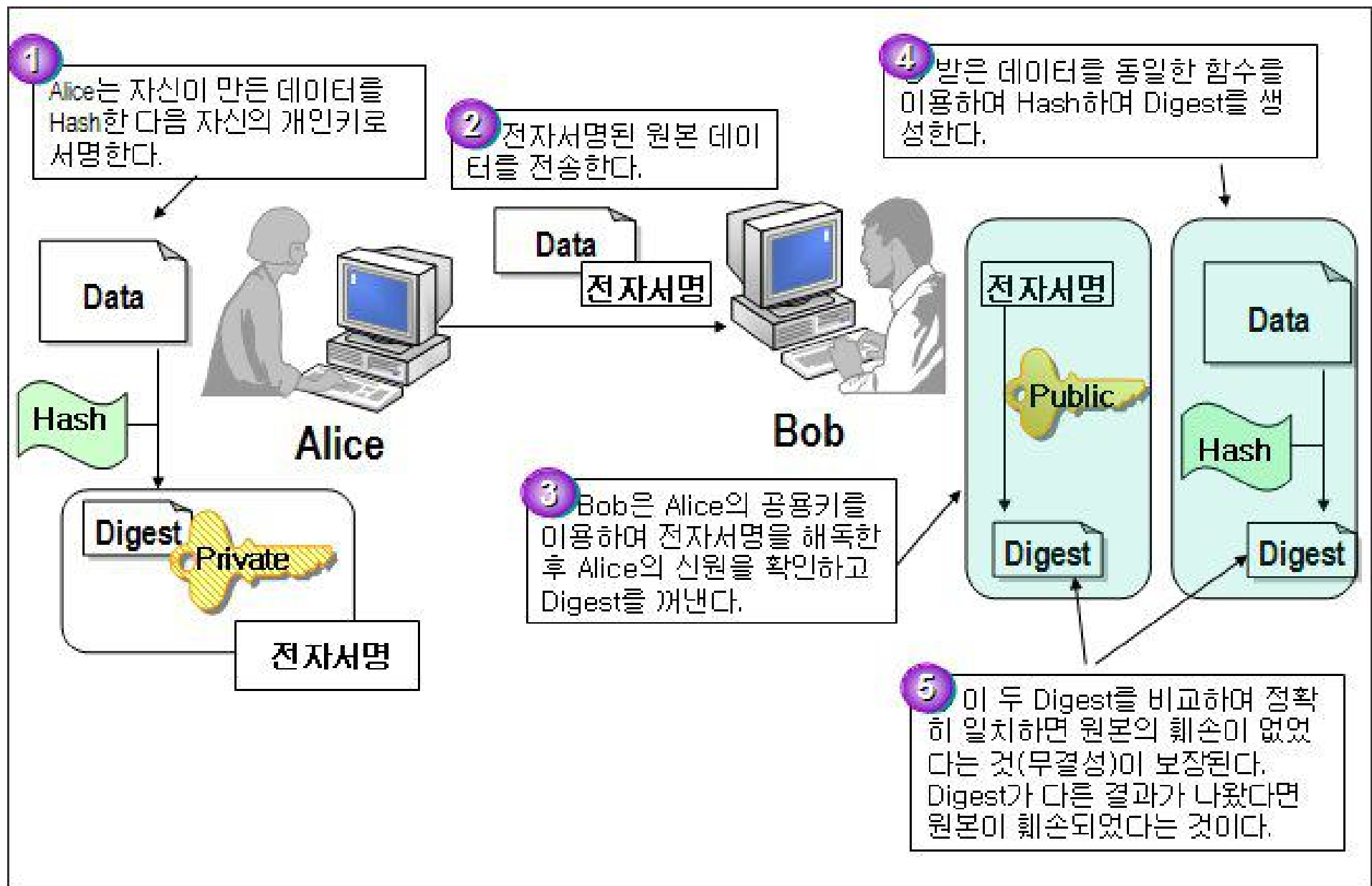
② Public Key 암호화(Confidentiality 제공)



③ Private Key 암호화(Authenticity 제공)



④ Hash Function & Digest (Integrity제공)

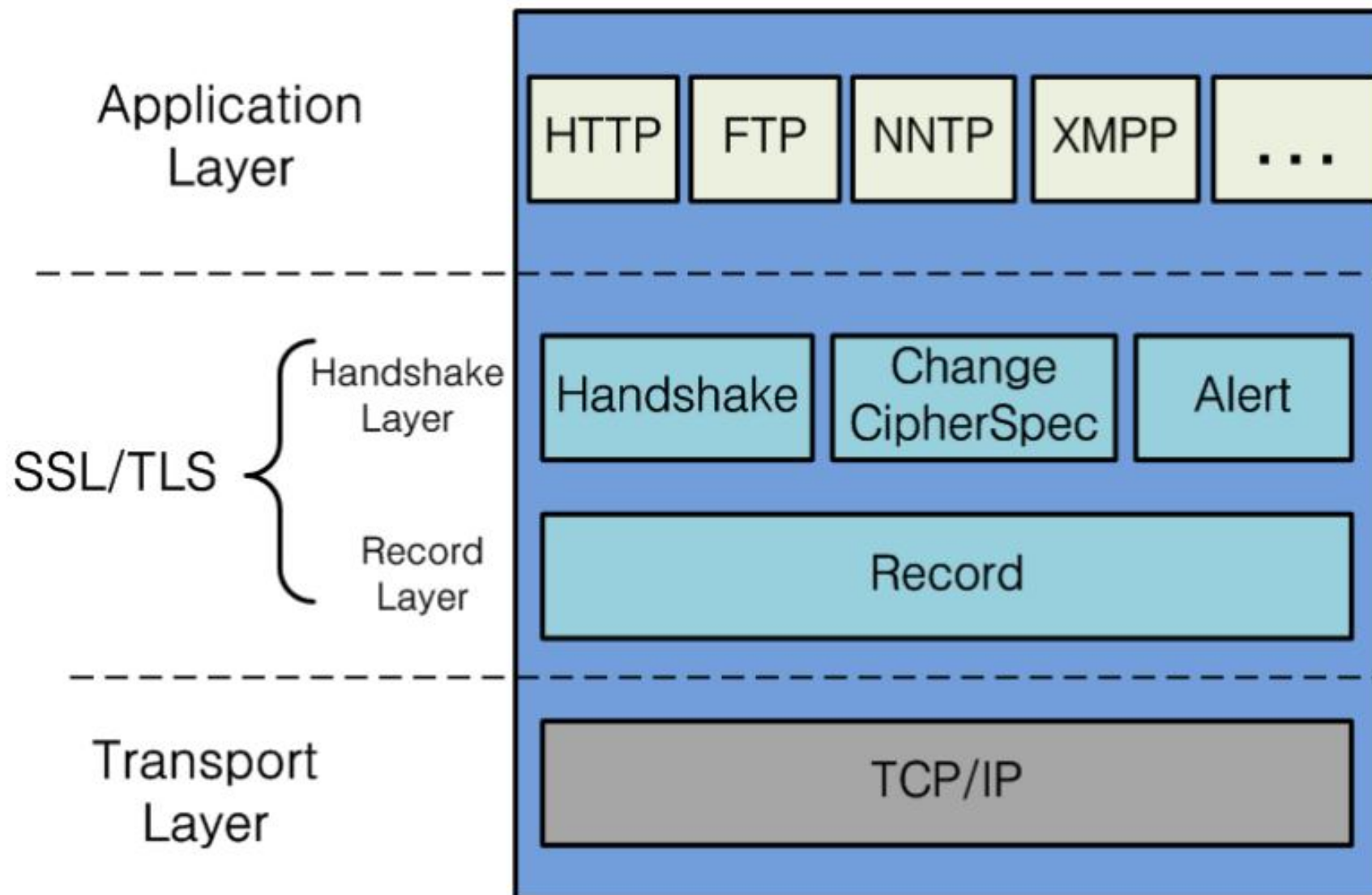


2) SSL/TLS Header 구조 분석

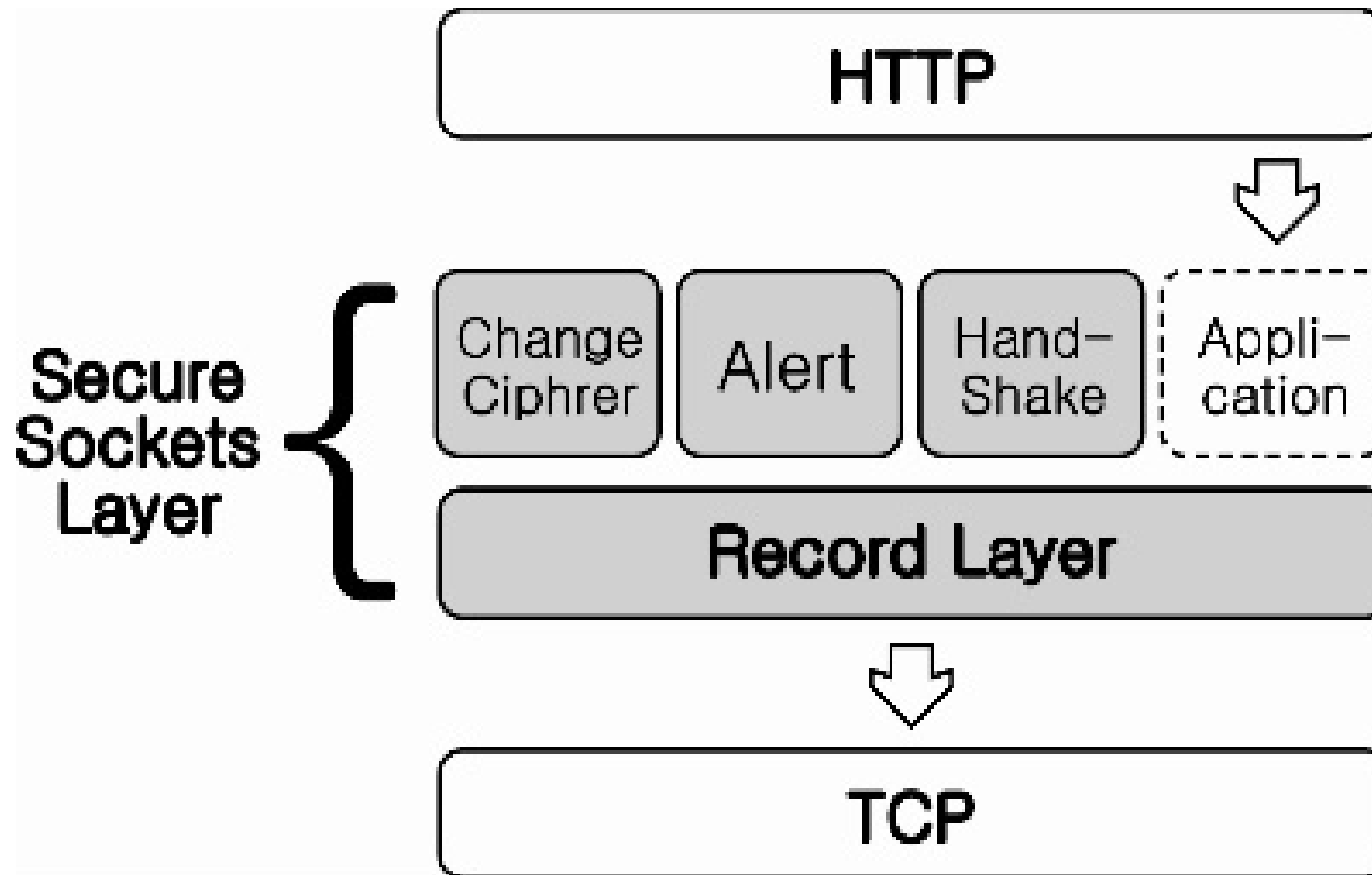
- 응용계층을 보호하는 프로토콜
- 웹 브라우저에 기본적으로 탑재되어 있음
- IETF에서 TLS 1.0 발표, SSL 3.1 이라고도 함
- TLS 프로토콜은 SSL 프로토콜을 사용할 수 있도록 구성
- SSL/TLS 프로토콜이라는 용어 사용

SSL(Secure Socket Layer)	TLS(Transport Layer Security)
응용계층을 보호하는 프로토콜	전송 계층 상위에서 동작하여 응용계층 암호화 HTTP 패킷 보호가 주용도

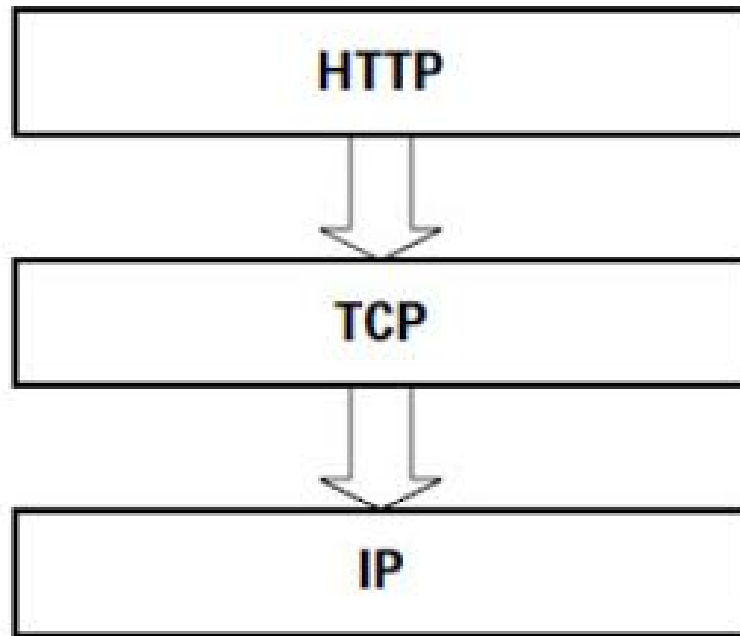
SSL/TLS Protocol stack



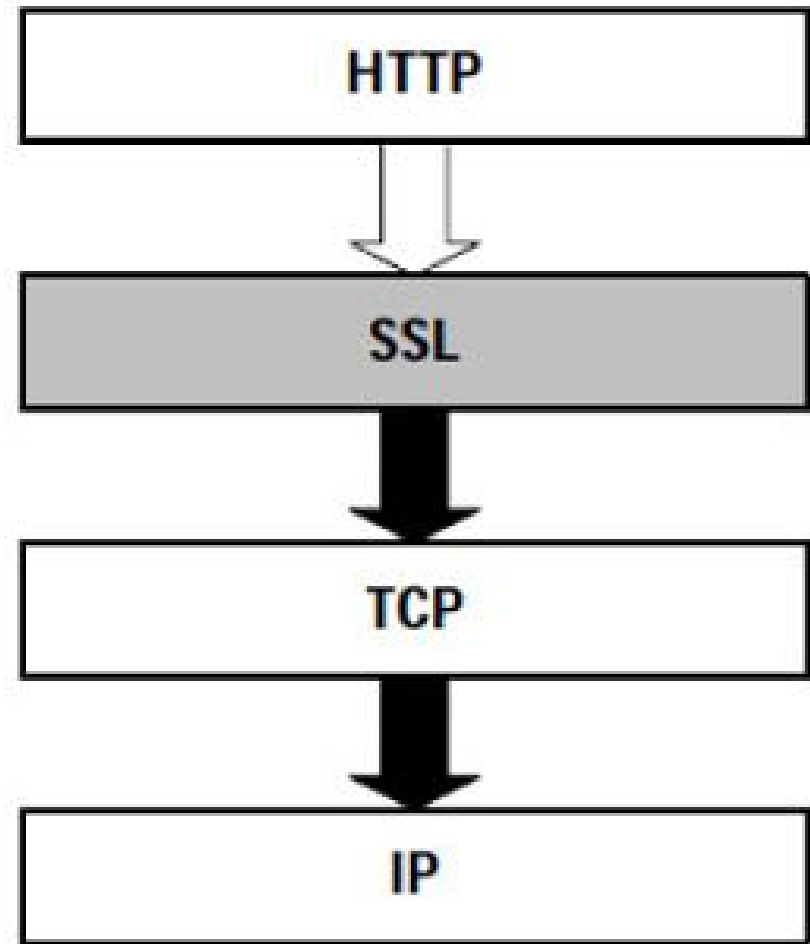
SSL/TLS Protocol stack



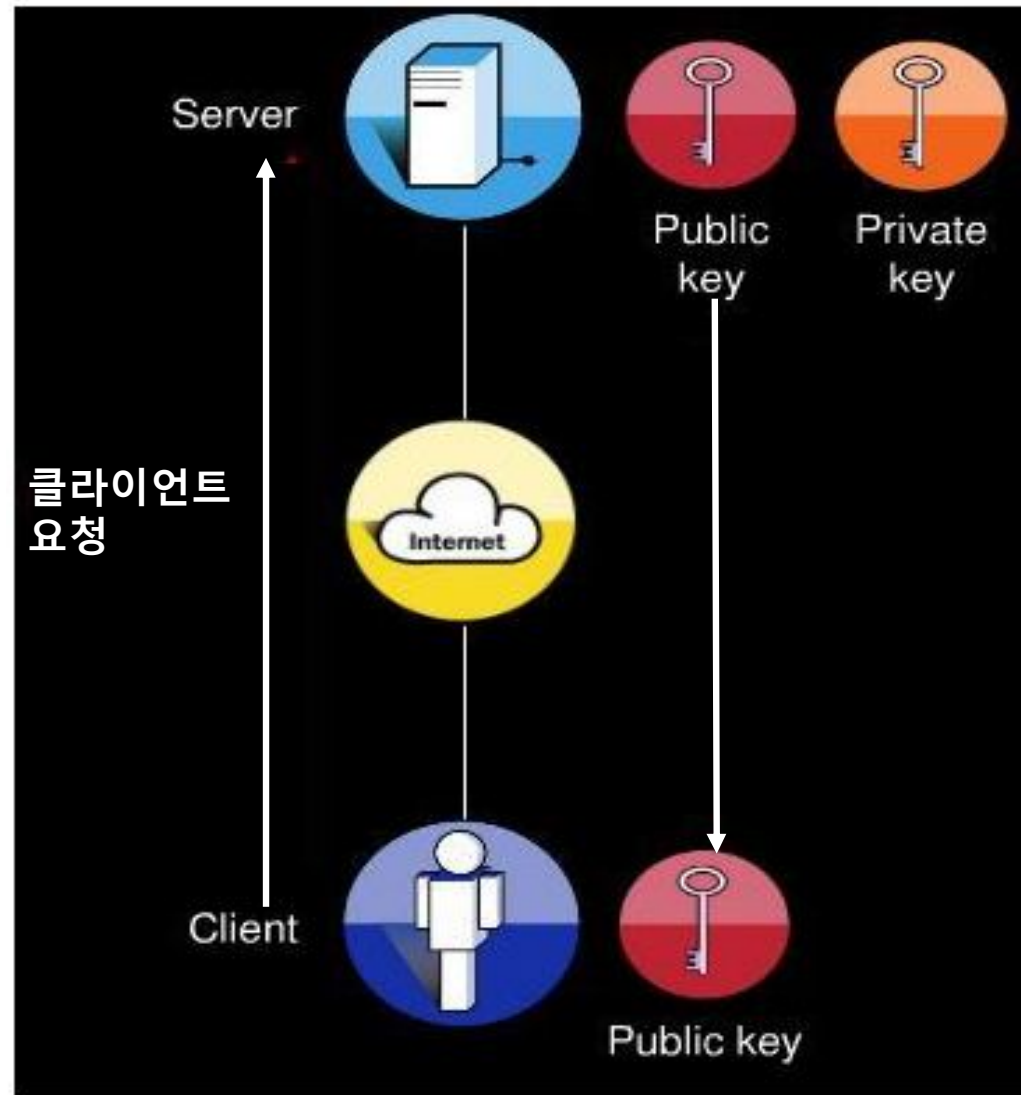
HTTP



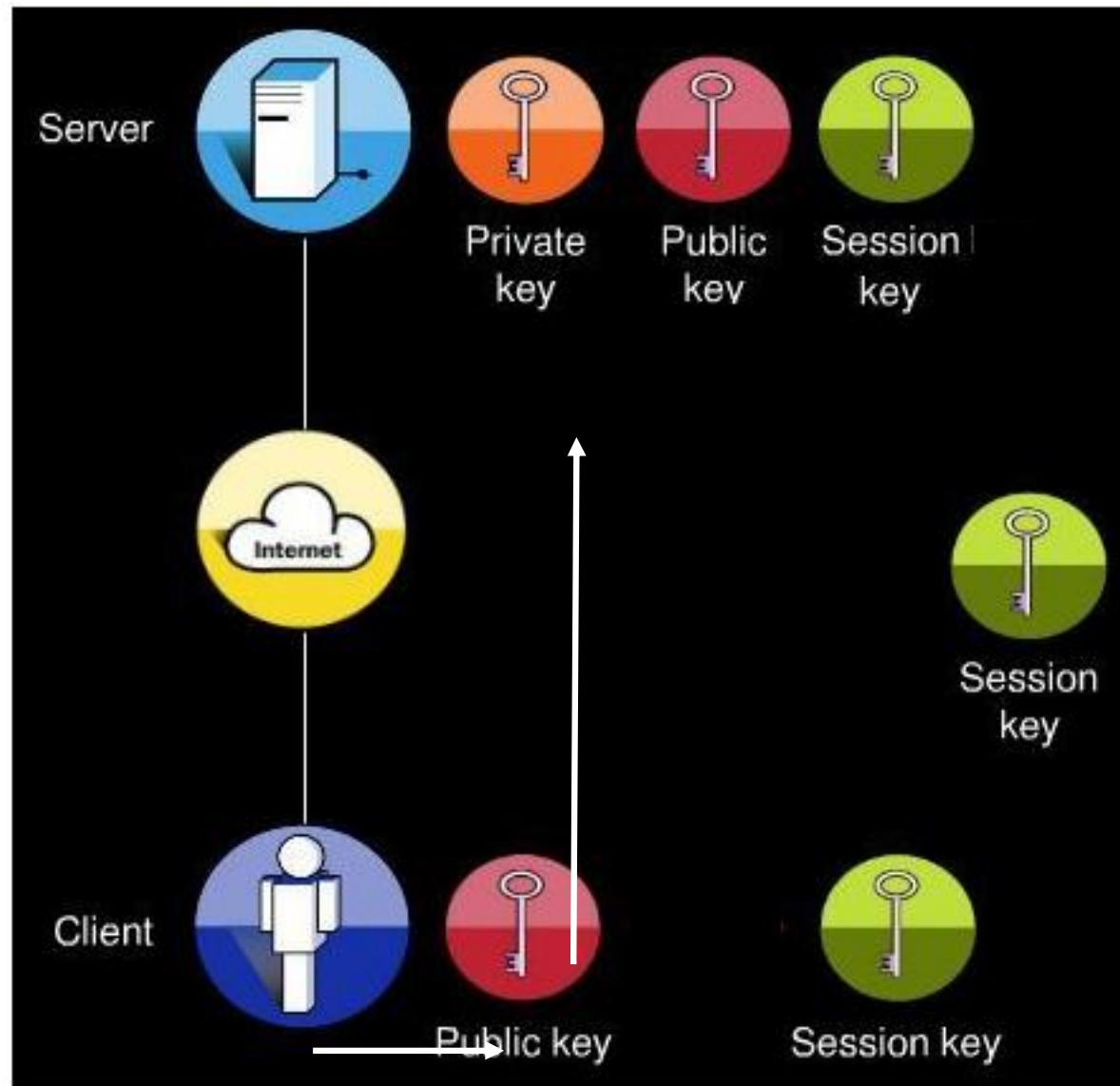
HTTPS



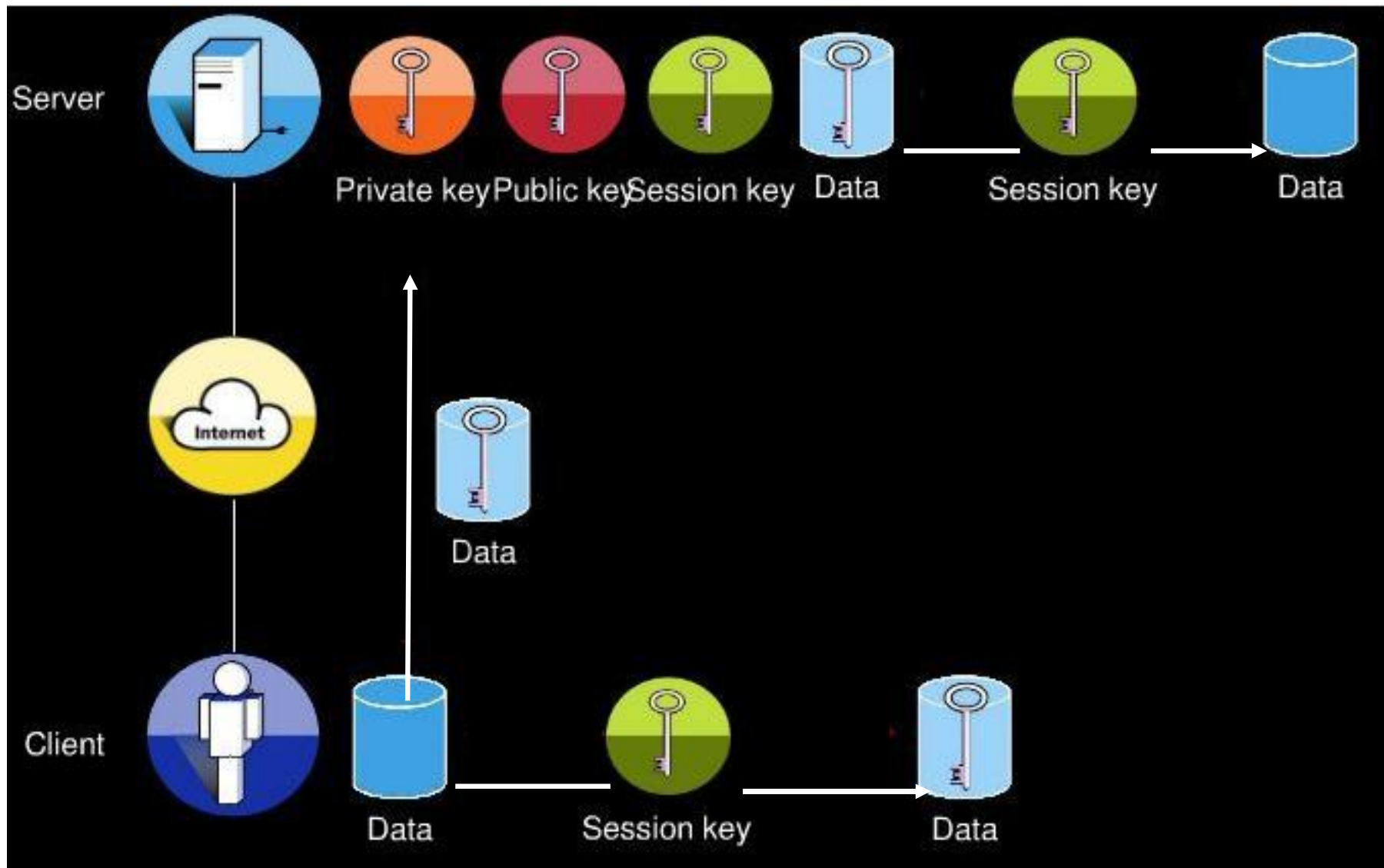
3) SSL/TLS Processing



Client가 server에 접속하면 서버인증서를 전송 받는다.



- Client는 받은 서버 인증서를 분석하여 신뢰할 수 있는 인증서 인지 검토
- 서버의 공개 키를 추출
- Client가 세션키로 사용할 임의의 메시지는 서버의 공개키로 암호화하여 서버에 전송



- 서버에서는 자신의 개인키로 세션키를 복호화
- 세션키를 사용하여 대칭키 암호 방식으로 메시지를 암호화하여 클라이언트와 통신
 - Pre-Master Secret Key : 암호화된 세션키
 - Master Secret key : 복호화된 세션키

